

eCOA User Requirements Specification

May 16, 2026

Contents

1	Product Requirements	2
1.1	Administrator Settings	3
1.2	Audit Log Visibility and Presentation	6
1.3	DIARY-PRD-configuration-precedence: Configuration Precedence	10
1.4	Device Linking	12
1.5	Diary Entry Rules	16
1.6	Daily eDiary: HHT Epistaxis Data Capture	24
1.7	Evidence Records	29
1.8	DIARY-PRD-help-resources: Help and Resources	38
1.9	Mobile Application Foundation	40
1.10	Participant Tasks and Notifications	48
1.11	DIARY-PRD-notification-behavior: Notification Behavior	62
1.12	Participant Workflows	64
1.13	DIARY-PRD-platform-operations-monitoring: Platform Operations and Monitoring	73
1.14	Sponsor Portal Authentication	75
1.15	Questionnaire Management	83
1.16	Questionnaires Overview	88
1.17	Participant Questionnaire Workflow	92
1.18	Questionnaire Versioning, Localization, and Sponsor Eligibility	101
1.19	Role-Based Access Control	106
1.20	Requirements Management	111
1.21	Score Calculation	114
1.22	Service Level Agreement	117
1.23	User Account Management	128
1.24	DIARY-PRD-data-backup-and-archival: Data Backup and Archival	143
1.25	DIARY-PRD-ancillary-platform-services: Ancillary Platform Services	144
2	Operational Requirements	145
3	Development Requirements	146
4	Topic Index	147

Product Requirements

1.1 Administrator Settings

This file groups the platform-side requirements that establish the **Administrator Settings** surface — a read-only view available to **Administrators** that displays the current values of sponsor-configured platform parameters. The PRD requirement defines the surface; the GUI requirement specifies its presentation. The sponsor-specific inventory of which **Configuration Parameters** are surfaced and how they are grouped lives in the corresponding hht_diary_callisto file (spec/prd-administrator-settings.md).

DIARY-PRD-administrator-settings: Administrator Settings Surface

Level: prd | **Status:** Legacy | **Implements:** -

Overview

The **Sponsor Portal** maintains a body of sponsor-configured platform parameters that govern System behavior. These parameters are defined in individual requirements throughout the specification and are typically static for the duration of a deployment. **Administrators** require visibility into the current values of these parameters to support troubleshooting, audit response, and operational verification, without needing to consult the specification or contact CureHHT personnel. The **Administrator Settings** surface consolidates these values into a single read-only view scoped to the **Administrator** role.

Definitions

Configuration Parameter: A named, sponsor-configured value that governs a specific System behavior, defined in a single source requirement and surfaced to **Administrators** for visibility.

Configuration Category: A grouping of related **Configuration Parameters** presented together within the **Administrator Settings** surface.

Administrator Settings: The read-only surface within the **Sponsor Portal** that displays **Configuration Parameters** to **Administrators**, organised by **Configuration Category**.

Assertions

Availability

A. The **System** SHALL provide an **Administrator Settings** surface within the **Sponsor Portal**.

Content

- B. The **System** SHALL display each **Configuration Parameter** in **Administrator Settings** with its name, current value, and unit of measurement where applicable.
- C. The **System** SHALL display each **Configuration Parameter** under exactly one **Configuration Category**.
- D. The **System** SHALL display the current value of each **Configuration Parameter** as defined by the source requirement that establishes it.

Read-Only Behavior

- E. The **System** SHALL NOT permit modification of any **Configuration Parameter** value from the **Administrator Settings** surface.

Configuration

- F. The **System** SHALL support sponsor-configurable selection of which **Configuration Parameters** are surfaced in **Administrator Settings** per deployment.
- G. The **System** SHALL support sponsor-configurable definition of **Configuration Categories** per deployment.

Rationale

Sponsor-configured parameters (session timeouts, code expiries, threshold values, reminder schedules) shape the system's runtime behavior but are not visible to the **Administrator** who is operationally responsible for the deployment. Without a surface that exposes these values, troubleshooting and audit response require the **Administrator** to either read the specification or escalate to CureHHT, neither of which scales. Consolidating the visible parameters under the **Administrator** role, with one **Configuration Category** per logical grouping and one source-requirement reference per parameter, gives the **Administrator** a single place to confirm "what is this deployment configured to do?" without granting modification capability. Modification stays out of band (the parameters are set during sponsor onboarding and changed via CureHHT change-control) because their values are tightly coupled to compliance review; a runtime modification surface would bypass that review.

End Administrator Settings Surface | **Hash:** 318dd00f

DIARY-GUI-administrator-settings: Administrator Settings Interface

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-administrator-settings

Overview

The **Administrator Settings** surface is reached from the **Settings** action in the **Administrator Dashboard** header. The surface presents **Configuration Parameters** as a grouped list, with each **Configuration Category** as a section heading and each

Configuration Parameter as a row beneath it. The interface communicates the read-only nature of the view so the **Administrator** does not expect to edit values directly.

Assertions

Display

- A. The interface SHALL display the **Administrator Settings** surface as a full-page view that replaces the **Administrator Dashboard** top-level tab content.
- B. The interface SHALL display a header on the **Administrator Settings** surface containing the title **Settings** and a back action that returns the **Administrator** to the previously active top-level tab.
- C. The interface SHALL display each **Configuration Category** as a labeled section.
- D. The interface SHALL display each **Configuration Parameter** as a row within its **Configuration Category** section, showing the parameter name, the current value, and the unit of measurement where applicable.
- E. The interface SHALL display **Configuration Parameters** in the order defined by the deployment configuration.

Read-Only Indication

- F. The interface SHALL display a notice on the **Administrator Settings** surface indicating that values are read-only and that changes require contacting CureHHT personnel.
- G. The interface SHALL NOT present any input control, edit action, or save action on the **Administrator Settings** surface.

Rationale

Settings is a low-frequency surface — an **Administrator** opens it during troubleshooting or audit response, not during routine workflow — so it is reached via a single header action rather than a tab. Presenting it as a full-page view that replaces the tab content (rather than a modal or a side-panel) reflects the surface's information density (multiple categories, many parameters per category) and lets the **Administrator** scan the entire configuration without scrolling between panels. The explicit read-only notice and the absence of any input control communicate the read-only contract at the GUI level: an **Administrator** who reaches this screen should not waste time looking for an edit affordance, and should know who to contact (CureHHT personnel) when a value needs to change. The back action returns to the previously active tab rather than always defaulting to **Users** because Administrators frequently reach Settings from the audit log during an investigation and should be returned to the same investigation context.

End Administrator Settings Interface | **Hash:** f10caf2f

1.2 Audit Log Visibility and Presentation

This file holds the platform-side audit-log GUI templates: the common presentation shared across all roles, and the role-specific refinements for Study Coordinator, CRA, and Administrator audit log views. The per-role visibility scope (which entries each role is authorized to see) is enforced by the callisto-side CAL-PRD-audit-log-view-scope, which the common GUI template refines cross-repo.

DIARY-GUI-audit-log-common: Audit Log View — Common Presentation

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** CAL-PRD-audit-log-view-scope

Overview

The **Audit Log View** uses a consistent presentation across all roles. This requirement defines the shared display, ordering, content, and empty state behavior. Role-specific GUI requirements refine this base by adding role-specific columns, controls, and entry points.

Assertions

Display

- A. The interface SHALL display the **Audit Log View** as a table containing at minimum the following columns: Timestamp, Action, User, Details.
- B. The interface SHALL display Audit Log Entries in reverse chronological order, with the most recent entry first.

Content

- C. When an **Audit Log Entry** corresponds to an **Action** that included a free text input from the user (such as a reason, justification, or note), the interface SHALL include that text in the Details column.
- D. The interface SHALL display in the Details column a human-readable summary of the **Action** that includes, where applicable, the previous state of the affected record and the parameter or option selected as part of the **Action**.

Empty State

- E. When the **Audit Log View** contains no Audit Log Entries within the active scope or filter selection, the interface SHALL display a message indicating no results were found.

Action Display

- F. The interface SHALL display the **Action** name in the Action column.

G. When an **Action** has parameters that distinguish meaningful variants (such as **Questionnaire Type**), the interface SHALL append the parameter to the **Action** name in the Action column for human readability.

Detail View

H. The interface SHALL provide a **More details** button on each **Audit Log Entry** that, when selected, displays the full **Audit Log Entry** record as raw text.

Rationale

A consistent audit log presentation across roles reduces training overhead, allows reviewers comparing entries across roles to read them with the same mental model, and concentrates accessibility, formatting, and column-ordering decisions in one place. Reverse chronological ordering surfaces the most recent activity first, matching the typical investigative workflow of “what just happened?”. The Details column carries the free-text reason that originated with the Action so reviewers do not have to drill into the raw record to understand intent. The raw-text **More details** affordance preserves the full record for inspectors who need every field, while keeping the table dense enough to scan quickly.

End Audit Log View — Common Presentation | **Hash:** f9158ab1

DIARY-GUI-audit-log-study-coordinator: Study Coordinator Audit Log View

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-GUI-audit-log-common

Overview

The **Study Coordinator Audit Log View** presents the **Study Coordinator**’s own Actions. **Participant**-related Actions are displayed alongside a **Participant ID** column.

Assertions

Display

A. The interface SHALL display the **Participant ID** column in addition to the columns defined in DIARY-GUI-audit-log-common.

Controls

B. The interface SHALL provide a **Participant ID** search input that filters the displayed Audit Log Entries by **Participant ID** in real time.

Rationale

A Study Coordinator's investigative workflow centers on individual participants — confirming what was done for a specific participant during a specific visit, or assembling documentation in response to a monitoring request. Surfacing **Participant ID** as a first-class column and providing a dedicated search input collapses the most common filter operation to a single keystroke. Limiting the view to the Coordinator's own actions reinforces the separation-of-duties model encoded in the per-role scope: Coordinators are accountable for their own audit trail and do not need (and should not have) visibility into peer actions.

End Study Coordinator Audit Log View | **Hash:** 5311961d

DIARY-GUI-audit-log-cra: CRA Audit Log View

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-GUI-audit-log-common

Overview

The **CRA Audit Log View** is accessed by selecting a **Site** from the **CRA's** list of assigned Sites and presents **Study Coordinator** Actions for the selected **Site**. A **Study Coordinator** filter and a **Participant ID** search input enable focused review.

Assertions

Display

- A. The interface SHALL display the **Participant ID** and **Site** columns in addition to the columns defined in DIARY-GUI-audit-log-common.
- B. The interface SHALL display the name of the selected **Site** in the **Audit Log View** header.

Controls

- C. The interface SHALL provide a **Study Coordinator** selector that filters the displayed Audit Log Entries by the selected **Study Coordinator**.
- D. The interface SHALL provide a default selection in the **Study Coordinator** selector that displays Audit Log Entries for all Study Coordinators at the selected **Site**.
- E. The interface SHALL provide a **Participant ID** search input that filters the displayed Audit Log Entries by **Participant ID** in real time.

Rationale

CRAs perform site monitoring — verifying that Study Coordinator activity at a specific Site is compliant and consistent with protocol. The single-Site scope makes the unit of monitoring explicit (a CRA reviews one Site at a time, not their entire assignment portfolio in a single view), the Site name in the header keeps the active scope visible, and the Coordinator selector with an “all Coordinators” default supports both audit-wide review and Coordinator-specific drill-down without forcing the CRA to apply filters manually. The Participant ID search supports the same per-participant investigation pattern the Study Coordinator view supports, but scoped to the Site under review.

End CRA Audit Log View | **Hash:** 108a69db

DIARY-GUI-audit-log-administrator: Administrator Audit Log View

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-GUI-audit-log-common

Overview

The Administrator audit log view is presented as a dedicated tab inside the Administrator dashboard, alongside the User Account management tab where Administrator actions originate.

Assertions

A. The interface SHALL present the **Administrator Audit Log View** on a dedicated tab within the **Administrator** dashboard, alongside the **User Account** management tab.

Rationale

Administrator actions are scoped to User Account management; placing the audit log view on the same dashboard as the User Account management surface keeps the Administrator’s investigative loop tight — review an account, switch to the audit tab, inspect the history of changes to that account, switch back. A separate top-level audit log surface would force navigation away from the Administrator’s working context for what is fundamentally an inline review activity.

End Administrator Audit Log View | **Hash:** 1f44ba79

1.3 DIARY-PRD-configuration-precedence: Configuration Precedence

Level: prd | **Status:** Legacy | **Implements:** -

Overview

The platform defines a baseline set of behaviors and default values that apply to every deployment. Each sponsor deployment may, where the platform requirement explicitly permits configuration, override those defaults with values appropriate to that study's protocol. This requirement establishes the precedence rule that resolves any apparent conflict between a platform requirement and a sponsor configuration: when both apply to the same behavior, the sponsor configuration governs. The rule is structural — it ensures that sponsor-specific values such as thresholds, reason lists, reminder times, and enabled features take effect deterministically without requiring each individual platform requirement to restate the precedence relationship. The rule applies only within the bounds set by the platform requirement itself; sponsor configuration cannot grant capabilities the platform does not expose, and cannot override platform behavior that is not declared as configurable.

Definitions

Platform Requirement: A requirement that defines behavior, structure, or default values applicable to every deployment of the **System**. Identified by a requirement ID of the form REQ-pXXXXX or GUI-pXXXXX.

Sponsor Configuration Requirement: A requirement that specifies the values applied to a configurable parameter exposed by a **Platform Requirement** for a specific sponsor deployment. Identified by a requirement ID of the form REQ-CAL-pXXXXX for the sponsor-specific deployment, with analogous prefixes for other deployments.

Configurable Parameter: A value, threshold, list, label, enablement flag, or option that a **Platform Requirement** explicitly declares as configurable per deployment.

Configuration Conflict: A state in which a **Sponsor Configuration Requirement** specifies a value for a **Configurable Parameter** that differs from the default value or behavior described in the corresponding **Platform Requirement**.

Assertions

A. The **System** SHALL apply the value specified in a **Sponsor Configuration Requirement** in place of the default value or behavior described in the corresponding **Platform Requirement** when a **Configuration Conflict** exists.

B. The **System** SHALL apply a **Sponsor Configuration Requirement** only to a **Configurable Parameter** that is explicitly declared as configurable by a **Platform Re-**

requirement.

C. The **System** SHALL NOT permit a **Sponsor Configuration Requirement** to override behavior of a **Platform Requirement** that is not declared as configurable.

D. The **System** SHALL NOT permit a **Sponsor Configuration Requirement** to grant a capability that is not defined by a **Platform Requirement**.

E. When no **Sponsor Configuration Requirement** exists for a given **Configurable Parameter**, the **System** SHALL apply the default value or behavior described in the corresponding **Platform Requirement**.

F. When a **Platform Requirement** specifies that a behavior takes effect only if a corresponding **Configurable Parameter** is configured, and no **Sponsor Configuration Requirement** configures it, the **System** SHALL NOT apply that behavior.

Rationale

A multi-sponsor platform must absorb deployment-specific differences (reason lists, reminder cadences, lock thresholds, enabled feature flags) without forking the code-base or rewriting platform requirements per sponsor. The conventional pattern — a single config file silently overrides defaults — is unsuitable for an FDA-regulated platform because it loses traceability: when a behavior is wrong in production, the auditor cannot reconstruct from the requirements which value the sponsor actually configured and why. This requirement encodes a structural precedence rule so every sponsor overlay (REQ-CAL-pXXXXX and equivalents) becomes a first-class, citable artifact. The two-way containment — sponsor configuration cannot exceed the platform's exposed configurability, and cannot grant capabilities the platform does not implement — keeps the platform requirements authoritative for the set of behaviors that exist at all, and keeps sponsor requirements authoritative for the specific values within that set. The absence-of-configuration rules (E, F) make the default-vs-opt-in semantics explicit so that a missing sponsor requirement is never ambiguous.

End Configuration Precedence | **Hash:** f5372beb

1.4 Device Linking

This file defines the participant-side linking flow: the platform-level rules for error handling and rate limiting on linking-code entry, the **Join the Study** screen the participant uses to enter a code and consent to the **Clinical Trial Privacy Policy**, and the Successful Linking Confirmation that completes the transition from **User** to **Participant**. Sponsor-side configuration of the rate-limit values and error message text lives in the `hht_diary_callisto` repo (CAL-PRD-linking-code-entry-errors-configuration).

DIARY-PRD-linking-code-entry-errors: Linking Code Entry Error Handling

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Security best practices require generic error messages that do not reveal whether a linking code exists, is expired, or has been used. This prevents malicious actors from enumerating valid codes or determining code status. Rate limiting protects against brute-force attacks while a configurable cooldown balances security with usability.

Definitions

Linking Code Validation Failure: Any unsuccessful attempt to enter a linking code, regardless of the specific reason for failure.

Rate Limit Threshold: The configurable maximum number of consecutive **Linking Code Validation Failures** permitted within a single rate limit window before further attempts are temporarily blocked.

Rate Limit Cooldown: The configurable duration after which the rate limit counter resets and the **User** may attempt linking again.

Assertions

Error Messaging

- A. When a **Linking Code Validation Failure** occurs, the System SHALL display a single generic error message regardless of the specific reason for failure.
- B. The error message SHALL NOT reveal the specific reason for failure.
- C. When a **Linking Code Validation Failure** occurs, the System SHALL provide guidance to contact the site for a new code.

Rate Limiting

D. When the number of consecutive **Linking Code Validation Failures** reaches the **Rate Limit Threshold**, the System SHALL block further linking attempts and display the **Rate Limit** error message until the **Rate Limit Cooldown** has elapsed.

E. The **Rate Limit Cooldown** SHALL be calculated from the first failed attempt in the current window.

Configuration

F. The System SHALL support sponsor-configurable **Rate Limit Threshold** per deployment.

G. The System SHALL support sponsor-configurable **Rate Limit Cooldown** per deployment.

H. The System SHALL support sponsor-configurable error message text per deployment.

Rationale

Linking codes are short, human-typeable secrets that confer access to a clinical-trial participant identity; the platform's threat model has to assume that someone may attempt to guess or enumerate valid codes. Two mitigations compose: a generic error message that does not distinguish "code does not exist" from "code expired" from "code already used" (assertions A and B — distinguishing them would tell an attacker exactly which guesses are worth pursuing), and a rate limit that caps the number of attempts in a window. The site-contact guidance (assertion C) is the legitimate-user recovery path that survives the genericity rule — a real participant who has mistyped their code gets the same message as an enumerator, but they also get the actionable next step (contact the site). The cooldown anchored at the first failed attempt in the window (assertion E) prevents the trivial workaround of attempting just below the threshold, waiting one second, and continuing — the window is a fixed time interval from the first miss. Sponsor-configurable threshold, cooldown, and text let each deployment tune the security/usability trade-off and customize the message.

End Linking Code Entry Error Handling | **Hash:** 2c1deb9d

DIARY-GUI-join-study-screen: Join the Study Screen

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-linking-code-entry-errors

Overview

The **Join the Study** screen is presented to the User when initiating the linking workflow. It captures the **Mobile Linking Code** and the Participant's explicit consent to the **Clinical Trial Privacy Policy** before any link is established.

Definitions

Linking Consent: The explicit acknowledgement by the Participant that they have read and consent to the Clinical Trial Privacy Policy. This consent must be captured before the linking code submission is permitted.

Assertions

- A. The **Join the Study** screen SHALL display a **Mobile Linking Code** entry field formatted according to the configured code format.
- B. The screen SHALL display a **Linking Consent** checkbox with text confirming the Participant has read, understood, and consents to the **Clinical Trial Privacy Policy**.
- C. The **Linking Consent** text SHALL include a link that opens the **Clinical Trial Privacy Policy**.
- D. The **Submit** action SHALL be disabled until both a complete **Mobile Linking Code** is entered and the **Linking Consent** checkbox is checked.
- E. The System SHALL retain the **Linking Consent** acknowledgement, including the **Clinical Trial Privacy Policy** version, against the **Participant** record upon successful link.

Rationale

The **Join the Study** screen has to do two things at once: capture the linking code (the technical secret that binds device to participant) and capture **Linking Consent** (the regulatory acknowledgement that the participant has read the **Clinical Trial Privacy Policy**). Co-locating them on a single screen rather than splitting across two screens reflects that both are required before linking can proceed, and a two-step split would invite the participant to complete one without the other. The Submit-disabled-until-both-present rule is the GUI-side enforcement of the dual gate — there is no path forward without both. Retaining the consent record with the policy version against the participant record is the audit-trail requirement: the platform must be able to demonstrate which version of the policy each participant consented to, separately from the current version, for the lifetime of their participation. The configurable code format on the entry field is the platform-side accommodation for sponsor-specific code shapes (length, separator characters) without imposing a single format across deployments.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

End Join the Study Screen | **Hash:** 4cf97153

DIARY-GUI-linking-confirmation: Successful Linking Confirmation

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-linking-code-entry-errors

Overview

When a Participant successfully enters a valid Mobile Linking Code on the Join the Study screen, the interface needs to clearly confirm that the link was established and transition the Participant from personal use mode into linked use mode.

Assertions

- A. When the **Participant** successfully submits a valid **Mobile Linking Code**, the interface SHALL display an **Acknowledgement Dialog** confirming that the device has been linked to the study.
- B. When the **Participant** acknowledges the dialog, the interface SHALL navigate the **Participant** to the **User Profile** screen.

Rationale

A successful link transitions the participant from a personal-use **User** to a clinical-trial **Participant** — a state change that affects what data syncs, which notifications fire, and which screens display the sponsor logo. The Acknowledgement Dialog makes the transition explicit so the participant has a clear before/after rather than discovering the change passively. Routing to the **User Profile** screen after acknowledgement (rather than back to the **Main Screen**) lands the participant on the screen where the new linked state is most visible: the Clinical Trial section now displays the **Participation Status Badge** instead of the unlinked-state guidance, which serves as the visible confirmation of the new state.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

Screen reference

See: Successful Linking Confirmation

End Successful Linking Confirmation | **Hash:** 495858e9

1.5 Diary Entry Rules

This file groups the platform-level validation rules that apply to every diary entry: the two-tier (justification, lock) time-based restrictions, the duration reasonableness check, and the overlapping-event detection-and-resolution flow. Sponsor-side configuration overlays (CAL-PRD-entry-time-restrictions-configuration, CAL-PRD-entry-duration-live) live in the hht_diary_callisto repo and refine the PRDs in this file with specific threshold values.

DIARY-PRD-entry-time-restrictions: Time-Based Entry Restrictions

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Clinical trial data integrity requires that participant-reported data be contemporaneous. As time passes after an event, recall accuracy degrades and the risk of unreliable data increases. A two-tier restriction model balances the need for late data capture (with documented justification) against the need to prevent entries that are too old to be clinically reliable. The justification tier preserves ALCOA+ compliance by requiring documented reasons for non-contemporaneous entries. The lock tier enforces a hard boundary beyond which no data modification is permitted.

Definitions

Justification Threshold: The configurable elapsed time from midnight (00:00) of the event date in the **Participant's** local timezone after which the **Participant** must provide a reason before creating or editing an entry for that date.

Lock Threshold: The configurable elapsed time from midnight (00:00) of the event date in the **Participant's** local timezone after which the date is fully locked and no creation, editing, or deletion of entries is permitted.

Entry Justification: A reason selected from a sponsor-defined predefined list that the **Participant** must provide when creating or editing an entry that has exceeded the **Justification Threshold**.

Assertions

Justification Tier

A. When the elapsed time for an event date exceeds the **Justification Threshold** but has not exceeded the **Lock Threshold**, the System SHALL require the **Participant** to select an **Entry Justification** before saving a new entry or an edit to an existing entry for that date.

- B. The System SHALL NOT require an **Entry Justification** when the **Participant** deletes an entry within the justification window.
- C. The System SHALL NOT allow free-text entry for **Entry Justification**.
- D. The System SHALL store the selected **Entry Justification** with the entry record.

Lock Tier

- E. When the elapsed time for an event date exceeds the **Lock Threshold**, the System SHALL NOT permit creation of new entries for that date.
- F. When the elapsed time for an event date exceeds the **Lock Threshold**, the System SHALL NOT permit editing of existing entries on that date.
- G. When the elapsed time for an event date exceeds the **Lock Threshold**, the System SHALL NOT permit deletion of existing entries on that date.

Configuration

- H. The System SHALL support sponsor-configurable **Justification Threshold** per deployment.
- I. The System SHALL support sponsor-configurable **Lock Threshold** per deployment.
- J. The **Lock Threshold** SHALL be greater than or equal to the **Justification Threshold**.
- K. The System SHALL support sponsor-configurable definition of the **Entry Justification** predefined list per deployment.
- L. When neither threshold is configured for a deployment, the System SHALL NOT apply time-based entry restrictions.
- M. The **Lock Threshold** SHALL apply only to event dates on or after the trial start date.

Rationale

The two-tier model exists because two distinct integrity concerns argue for two distinct boundaries. Recall accuracy degrades gradually after an event, so for some window after the event date the data is still useful but the participant should explicitly acknowledge they are entering it late (this is the **Justification Threshold** — a soft boundary that captures a categorical reason on the record). Beyond a longer window, the data is no longer clinically reliable regardless of justification, so the platform makes it impossible to modify (the **Lock Threshold** — a hard boundary that disables create/edit/delete). Predefined-list justifications (rather than free-text) match the analyzability goal: the audit trail aggregates over a controlled vocabulary of reasons rather than over unbounded prose. Deletion within the justification window does not require justification because the participant is removing data they reported, not modifying it; the audit trail still captures the deletion event and its own reason via the standard delete flow. The $\text{Lock} \geq \text{Justification}$ invariant ensures the two tiers compose correctly (a Justification-required state always precedes a Lock state for the same date). The “only after trial start date” qualifier on the **Lock Threshold** means

pre-trial historical entries (covered by the **Diary Start Day**) are not subject to the lock, since those entries are explicitly retrospective and the lock semantics don't apply.

End Time-Based Entry Restrictions | **Hash:** 73c1e562

DIARY-PRD-entry-duration-check: Duration Reasonableness Check

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Unusually short or long event durations may indicate data entry errors. Confirmation prompts at configurable thresholds help ensure data accuracy while allowing legitimate outlier events to be recorded.

Definitions

Duration Reasonableness Check: A configurable validation rule that prompts the **Participant** to confirm the recorded duration of an event when it falls outside expected boundaries.

Short Duration Threshold: The configurable minimum duration below which a confirmation prompt is triggered.

Long Duration Threshold: The configurable maximum duration above which a confirmation prompt is triggered.

Assertions

Short Duration Check

- A. When the duration of an event is less than the **Short Duration Threshold**, the System SHALL prompt the **Participant** to confirm the duration before saving.
- B. The confirmation prompt SHALL provide a confirm option that saves the entry as entered and a reject option that returns the **Participant** to edit the duration.

Long Duration Check

- C. When the duration of an event is greater than the **Long Duration Threshold**, the System SHALL prompt the **Participant** to confirm the duration before saving.
- D. The confirmation prompt SHALL provide a confirm option that saves the entry as entered and a reject option that returns the **Participant** to edit the duration.

Prompt Timing

- E. The System SHALL display the confirmation prompt before the save operation completes.

Configuration

- F. The System SHALL support sponsor-configurable **Short Duration Threshold** per deployment.
- G. The System SHALL support sponsor-configurable **Long Duration Threshold** per deployment.
- H. The System SHALL support sponsor-configurable enablement of each check independently per deployment.
- I. When a check is not enabled for a deployment, the System SHALL NOT display the corresponding confirmation prompt.

Rationale

Duration outliers are a common data-entry-error signal: a typo or wrong-timezone selection can produce a one-second or twelve-hour nosebleed that the participant did not actually have. A soft confirmation prompt at configured boundaries catches these without rejecting legitimately-outlying events (a multi-hour nosebleed can be real for certain HHT participants). The two checks are independently enableable because a deployment may want only one direction (e.g. only the short-duration check, if long durations are clinically plausible in the protocol). The pre-save timing rule (assertion E) ensures the participant sees the prompt at the moment of decision rather than discovering after the fact that their entry was flagged. Reject-and-edit is the inverse of confirm-and-save and gives the participant a direct path to correct a typo without re-entering the event from scratch.

End Duration Reasonableness Check | **Hash:** 4c885aa4

DIARY-PRD-entry-overlap-resolution: Overlapping Event Detection and Resolution

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Nosebleed events cannot physically overlap — a participant cannot have two simultaneous, independent nosebleeds. When a new record's time range intersects with an existing record, the system must detect the conflict and guide the participant through resolution before the record is committed. This ensures data integrity and prevents logically impossible entries from reaching the clinical dataset.

Definitions

Overlap: A state in which the time range of a new or edited entry intersects with the time range of one or more existing entries for the same **Participant**.

Conflicting Record: An existing entry whose time range intersects with the entry being created or edited.

Resolution: The **Participant's** choice of how to handle an **Overlap** between two entries.

Assertions

Detection

- A. The System SHALL evaluate for potential **Overlap** when the **Participant** sets the start time of a new or edited entry, and SHALL confirm the **Overlap** when the **Participant** sets the end time.
- B. The System SHALL evaluate **Overlap** by comparing the actual start and end timestamps of the new or edited entry against all existing entries for the same **Participant**.
- C. The System SHALL evaluate **Overlap** against both completed entries and ongoing entries.

Pre-Resolution State

- D. When an **Overlap** is detected, the System SHALL save the new entry to persistent storage immediately upon creation.
- E. When an **Overlap** is detected, the System SHALL NOT display the new entry in the **Participant's** diary history until **Resolution** is complete.

Resolution Sequencing

- F. The System SHALL present the **Participant** with one **Conflicting Record** at a time, starting with the oldest.
- G. After each **Resolution** step, the System SHALL evaluate the resulting entry for further **Overlaps**.
- H. The System SHALL repeat the **Resolution** flow until no **Overlaps** remain.
- I. When all **Overlaps** are resolved, the System SHALL commit all resulting changes as a single transaction.

Resolution Options

- J. For each **Overlap**, the System SHALL offer three resolution options: keep the new entry and discard the **Conflicting Record**, keep the **Conflicting Record** and discard the new entry, or merge both entries into one.

Merge Behavior

- K. When the **Participant** selects merge, the System SHALL construct the merged entry using the earliest start time and the latest end time across both entries.
- L. When either entry in a merge is ongoing, the merged entry SHALL be ongoing.
- M. When the **Participant** selects merge, the System SHALL assign the higher severity value from the two original entries to the merged entry.

Post-Resolution Confirmation

- N. After any resolution choice, the System SHALL present the resulting entry to the **Participant** for review and confirmation before committing.
- O. The **Participant** SHALL be able to edit the resulting entry before confirming.

Rationale

Two nosebleeds cannot physically overlap; allowing overlapping records into the dataset would be silent data corruption. The two-stage detection (predictive on start-time, confirmatory on end-time) gives the participant early notice that an overlap is forming without interrupting them mid-record, then escalates to mandatory resolution at the moment the conflict becomes a hard reality. Saving the new entry to persistent storage immediately on creation (assertion D) is the durability guarantee that the participant's work is not lost if the application crashes mid-resolution; hiding it from the diary history until resolution completes (assertion E) keeps the participant's view consistent with what has actually been resolved. One-conflict-at-a-time resolution starting with the oldest, repeated until no overlaps remain, is the only sequencing that handles the cascade case (a new record overlaps two existing records that themselves overlap each other once merged). Three resolution options cover the realistic choices: the new record was right and the old one was wrong (keep-new), the old record was right and the new one is a duplicate (keep-existing), or both were partial captures of one continuous event (merge). Merge using earliest-start / latest-end with the higher severity captures the union-by-time and max-by-severity that best represents one continuous event reconstructed from two partial captures. Pre-commit review is the final integrity gate — the participant sees the resulting entry as the platform will save it and can edit any field before confirming.

Screen reference

See: Resolve Conflict — Resolution Screen

End Overlapping Event Detection and Resolution | **Hash:** bce7cb5b

DIARY-GUI-entry-overlap-resolution: Overlapping Event Resolution Flow

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-entry-overlap-resolution

Overview

When an overlap is detected, the **Participant** needs early warning and a clear resolution path. Displaying a warning as soon as the start time indicates a potential conflict gives the **Participant** awareness without interrupting their recording flow. Once the

full time range is established, the resolution screen provides a side-by-side comparison and straightforward actions to resolve the conflict.

Definitions

Resolution Screen: The screen presented to the **Participant** when an **Overlap** is confirmed, displaying the new entry and the **Conflicting Record** side by side with available resolution actions.

Assertions

Early Warning

- A. When a potential **Overlap** is detected after the **Participant** sets the start time, the interface SHALL display a warning indicating that overlapping events have been detected and the number of conflicting records.
- B. The warning SHALL NOT prevent the **Participant** from continuing the recording flow.

Resolution Trigger

- C. When the **Participant** sets the end time and the **Overlap** is confirmed, the interface SHALL navigate the **Participant** to the **Resolution Screen**.

Resolution Screen

- D. The interface SHALL present the **Resolution Screen** displaying the new entry and the **Conflicting Record** side by side.
- E. Each entry on the **Resolution Screen** SHALL display the time range, duration, and severity.
- F. When the **Conflicting Record** is older than the **Justification Threshold**, the System SHALL require an **Entry Justification** before saving the resolution.
- G. The **Resolution Screen** SHALL present a Keep New action and a Keep Existing action. The Resolution Screen SHALL present a Merge Records action only when the Conflicting Record's event date has not exceeded the **Lock Threshold**.
- H. The Merge Records action SHALL display a description indicating that both records will be combined into one event.

Post-Resolution

- I. When the Merge Records action is unavailable due to the **Lock Threshold**, the Resolution Screen SHALL display a message indicating that the conflicting record is locked and cannot be merged.
- J. After the **Participant** selects any resolution option (Keep New, Keep Existing, or Merge Records), the interface SHALL navigate to the standard **Record Nosebleed** screen pre-filled with the resulting entry data.

K. The **Record Nosebleed** screen SHALL allow the **Participant** to review and edit any field of the resulting entry before confirming.

Multiple Conflicts

L. When multiple **Overlaps** exist, the interface SHALL present one **Resolution Screen** at a time.

M. When all **Overlaps** are resolved and the **Participant** confirms the final entry, the interface SHALL return the **Participant** to the Main Screen.

Rationale

The early warning on start-time is a non-blocking signal: the participant sees that something is going to need resolution but is not pushed into the resolution flow yet, because the end-time may still make the overlap go away (e.g. the participant moves the end time earlier than the conflicting record's start). Only when the end-time confirms the conflict does the GUI commit the participant to the **Resolution Screen**. The side-by-side display makes the comparison legible — same time-range / duration / severity rows for both entries — so the participant can make an informed choice between Keep New, Keep Existing, and Merge. The Merge-availability check against the **Lock Threshold** is required because merging would produce edits to the Conflicting Record's date, which the lock prohibits; surfacing the unavailability with an explanatory message (rather than silently disabling) keeps the participant oriented. Routing the post-resolution result through the standard **Record Nosebleed** screen (pre-filled) gives the participant a final edit-and-confirm opportunity on the same surface they used to enter the original record — a single interaction model end-to-end rather than a resolution-specific screen the participant has to learn separately. One-at-a-time presentation for multiple overlaps matches the PRD-level resolution sequencing rule, and the return-to-Main-Screen on completion closes the recording journey at its natural endpoint.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

Screen reference

See: Resolve Conflict — Resolution Screen

End Overlapping Event Resolution Flow | **Hash:** a78fddfa

1.6 Daily eDiary: HHT Epistaxis Data Capture

This file defines the daily eDiary data-capture flow for **Epistaxis Events**: the platform-level data capture standard, the participant-facing recording flow (Start, Max Intensity, End), and the deletion flow (in-flight during recording and post-save from event history). The questionnaire-instrument definition for the Daily Epistaxis Record itself lives in `prd-questionnaire-overview.md` as `DIARY-PRD-questionnaire-daily-epistaxis`; this file is about the eDiary flow over that instrument.

DIARY-PRD-epistaxis-capture-standard: HHT Epistaxis Data Capture Standard

Level: prd | **Status:** Legacy | **Implements:** -

Overview

The HHT Epistaxis Data Capture Standard defines the requirements for participant-reported nosebleed event collection in HHT clinical trials. Epistaxis frequency, duration, and severity are primary outcome measures for evaluating treatment efficacy. Timezone-aware timestamps ensure precise duration calculation regardless of participant location.

Definitions

Daily Status: The participant's self-reported summary of nosebleed activity for a given day. Valid values are Had Nosebleed, No Nosebleed, and Don't Remember.

Intensity: A participant-reported measure of nosebleed severity captured on a defined six-level scale ordered from least to most severe.

Incomplete Record: An **Epistaxis Event** that has been saved but is missing one or more required fields: start time, **Intensity**, or end time.

Assertions

- A. The System SHALL require a participant to record a **Daily Status** for each calendar day within the diary period.
- B. The System SHALL enforce mutual exclusivity of the three **Daily Status** values — a participant SHALL NOT record more than one **Daily Status** per diary entry.
- C. When a participant records a **Daily Status** of Had Nosebleed, the System SHALL require the participant to provide start time and end time for the **Epistaxis Event** with timezone information.

- D. The System SHALL calculate the duration of an **Epistaxis Event** from the recorded start time and end time, accounting for timezone differences.
- E. The System SHALL require the participant to record an **Intensity** value for each **Epistaxis Event**.
- F. The System SHALL present notes options from a predefined list (defined in CAL-PRD-entry-time-restrictions-configuration) for each **Epistaxis Event**; the available options SHALL be sponsor-configurable per study protocol.
- G. The System SHALL classify an **Epistaxis Event** as an **Incomplete Record** when it has been saved with one or more required fields not recorded.

Rationale

The Daily Status / Epistaxis Event two-level structure exists because the clinical question is two-leveled: “Did anything happen today?” (a daily-grain answer that can be Had Nosebleed, No Nosebleed, or Don’t Remember) and, when something did happen, “What were the events?” (per-event start, end, intensity). Mutually exclusive **Daily Status** values prevent contradictory records (a participant cannot simultaneously have had a nosebleed and not had one). Required start, end, and **Intensity** on each **Epistaxis Event** capture the three primary outcome measures (frequency by count, duration by interval, severity by intensity); allowing any of these to be missing without an explicit incomplete-record state would silently downgrade dataset quality. Timezone-aware capture is essential because participants travel and durations computed from naive timestamps across a DST boundary or international move can be off by an hour or more. The **Incomplete Record** classification lets the platform distinguish “in progress, missing data” from “fully captured” so the participant can be prompted to finish and the dataset can flag what is not yet complete.

End HHT Epistaxis Data Capture Standard | **Hash:** b38359f8

DIARY-GUI-epistaxis-record: Record Nosebleed Event

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-epistaxis-capture-standard

Overview

The nosebleed recording flow walks the participant through three steps — **Start** time, **Max Intensity**, **End** time — visible at all times in a persistent **Progress Indicator** at the top of every screen. The participant can revisit any completed step. Validation at the End Time step prevents impossible records (end before start, end after current real time) and prompts for confirmation on entries that look like they may have been recorded too quickly after the event.

Definitions

Progress Indicator: The persistent bar displayed at the top of every screen in the recording flow showing the three steps — Start, Max Intensity, and End — and their completion state.

Time Picker: The time selection component presenting the current time as default, a date selector, a timezone selector, and minute adjustment controls of -15, -5, -1, +1, +5, and +15 minutes.

Assertions

Flow Structure

- A. The interface SHALL display the **Progress Indicator** on every screen throughout the recording flow showing the Start, Max Intensity, and End steps.
- B. The interface SHALL highlight the active step in the **Progress Indicator** and show the recorded value for each completed step.
- C. The interface SHALL allow the participant to navigate to any previously completed step by tapping it in the **Progress Indicator**.
- D. The interface SHALL present a Back navigation action on every screen throughout the recording flow.

Nosebleed Start Time

- E. The interface SHALL present a **Time Picker** for the participant to set the nosebleed start time.
- F. The interface SHALL present a Set Start Time button that confirms the start time and advances to the Max Intensity screen.

Max Intensity

- G. The interface SHALL present the six intensity options as a selectable grid: Spotting, Dripping, Dripping quickly, Steady stream, Pouring, and Gushing.
- H. The interface SHALL NOT advance to the End Time screen until the participant has selected one of the intensity options.
- I. When the participant selects an intensity option, the interface SHALL highlight the selected option and automatically advance to the End Time screen.

Nosebleed End Time

- J. The interface SHALL present a **Time Picker** for the participant to set the nosebleed end time.
- K. When the participant selects Set End Time, the interface SHALL save the record and return the participant to the home screen.
- L. The System SHALL validate that end time is greater than or equal to start time.
- M. The System SHALL restrict end time selection to times not exceeding the current

real time.

N. The System SHALL require confirmation when creating entries less than 2 minutes old.

Rationale

The three-step structure (Start, Max Intensity, End) matches the clinical model of an event (when it began, how bad it got, when it ended); making the **Progress Indicator** persistent across every screen of the flow keeps the participant oriented and allows direct navigation to any completed step without back-tracking. The **Time Picker**'s minute-adjustment chips (-15/-5/-1/+1/+5/+15) target the realistic adjustment granularity for recording a recently-completed event and avoid the precision-mismatch of a free-text time entry. The auto-advance on intensity selection (vs. requiring a separate Next tap) shaves friction from the most common path (set start, pick intensity, set end) while still letting the participant revisit via the **Progress Indicator** if they want to change their intensity choice. End-time validation (no earlier than start, no later than now) catches the two impossible-time inputs at the GUI layer rather than relying on later data validation. The two-minute confirmation prompt is a soft-stop against premature finalization — a participant who is still actively having a nosebleed when they set the end time is reminded to check before committing.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

End Record Nosebleed Event | **Hash:** fa583a47

DIARY-GUI-epistaxis-delete: Nosebleed Event Delete

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-epistaxis-capture-standard

Overview

Deletion of a nosebleed event can happen in two contexts: in-flight during the recording flow (before the record has been saved) and post-save from event history. Both paths require a **Reason Dialog — Predefined List** before the deletion is applied, and the participant may cancel at any point.

Assertions

Delete During Recording Flow

- A. The interface SHALL present a delete action on every screen throughout the recording flow.
- B. When the participant selects the delete action during the recording flow, the interface SHALL display a **Reason Dialog — Predefined List** before proceeding.
- C. When the participant confirms the delete reason, the interface SHALL discard the in-progress record and return the participant to the home screen.
- D. When the participant cancels the Reason Dialog, the interface SHALL return the participant to the screen they were on.

Delete From Event History

- E. When the participant selects the delete action on a saved **Epistaxis Event**, the interface SHALL display a **Reason Dialog — Predefined List** before proceeding.
- F. When the participant confirms the delete reason, the interface SHALL remove the event from the participant's diary and return the participant to the previous screen.
- G. When the participant cancels the Reason Dialog, the **Epistaxis Event** SHALL remain unchanged.

Rationale

The delete action is exposed on every screen of the recording flow because the participant may realize partway through that they started the record by mistake (wrong date, wrong event, accidental tap) and there is no benefit to forcing them to complete the flow before deleting. Routing both in-flight delete and post-save delete through a **Reason Dialog — Predefined List** keeps the audit trail's deletion-reason capture consistent across both contexts — a deletion at any point produces a categorized reason record. Predefined-list (rather than free-text) reasons match the dataset analyzability goal: deletion reasons are the kind of metadata downstream analysis aggregates over, and a controlled vocabulary makes that analysis tractable. The cancel path on either delete returns the participant to where they were (in-flight: the same screen; post-save: the previous screen, with the record unchanged) so an accidental delete-tap is fully recoverable.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

Screen reference

See: Delete Record — Reason Dialog

End Nosebleed Event Delete | **Hash:** cd171380

1.7 Evidence Records

Data-provenance commitments establishing the **when**, **how**, and **who** of clinical trial diary entries through third-party timestamp attestation, device fingerprinting, patient authentication, and identity verification. These obligations are sponsor-visible (PRD-level audience) and provide the cryptographic basis for regulatory defensibility of long-retention clinical records.

DIARY-PRD-evidence-timestamp-attestation: Third-Party Timestamp Attestation Capability

Level: prd | **Status:** Legacy | **Implements:** -

TODO (URS-Phase-3 reconciliation): Original Refines target REQ-p01085 (Compliance Systems) is in URS-replaced prd-clinical-trials.md. Re-parent to the URS-derived compliance REQ when that lands.

Assertions

- A. The system SHALL provide third-party timestamp attestation for clinical trial diary data.
- B. Timestamp attestation SHALL create independently verifiable proof that data existed at the time of recording.
- C. Timestamps SHALL be issued by entities independent of the clinical trial system.
- D. Timestamp proofs SHALL be cryptographically verifiable.
- E. Timestamp verification SHALL NOT require trust in any single party.
- F. Timestamps SHALL NOT be forgeable or backdatable.
- G. Timestamp proofs SHALL remain valid throughout the data retention period of 15-25 years.
- H. The system SHALL enable any timestamp to be independently verified by third parties.
- I. Timestamp verification SHALL produce cryptographic proof of minimum timestamp age.
- J. Timestamps SHALL be bound to specific data content such that any data modification invalidates the proof.
- K. Timestamp proof files SHALL be self-contained for regulatory review.
- L. Timestamp proof files SHALL be portable for regulatory review.

Rationale

Self-asserted timestamps can be questioned during regulatory audits. Independent third-party attestation provides incontrovertible evidence of when data was recorded, strengthening FDA 21 CFR Part 11 compliance and trial defensibility. This requirement establishes the framework for creating tamper-evident temporal proofs that remain verifiable throughout the mandated 15-25 year data retention period for clinical trial records.

End Third-Party Timestamp Attestation Capability | **Hash:** f2ab1f17

DIARY-PRD-evidence-bitcoin-timestamp: Bitcoin-Based Timestamp Implementation

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-evidence-timestamp-attestation-A+B+F

Assertions

- A. The system SHALL use Bitcoin blockchain via OpenTimestamps protocol as the primary third-party timestamp mechanism.
- B. The system SHALL aggregate daily diary entries into single timestamp proofs.
- C. The system SHALL submit timestamp proofs to multiple independent calendar servers for redundancy.
- D. The system SHALL automatically complete timestamp proofs after Bitcoin confirmation.
- E. The system SHALL store timestamp proof files locally associated with timestamped data.
- F. The system SHALL support offline verification of timestamp proofs without network access.
- G. The system SHALL create daily aggregated proofs for all diary entries.
- H. Timestamp proofs SHALL complete within 24 hours of submission.
- I. Timestamp proof verification SHALL succeed without external network access.
- J. Timestamp proof files SHALL be portable for independent regulatory verification.

Rationale

This requirement establishes Bitcoin blockchain as the cryptographic timestamp mechanism for clinical trial data integrity. The OpenTimestamps protocol leverages Bitcoin's immutability and security properties to create tamper-evident proof of data existence at specific points in time. Bitcoin's substantial attack cost (\$5-20 billion) and absence

of historical breaches provide the strongest available guarantee for regulatory compliance. The aggregation mechanism minimizes operational overhead while maintaining cryptographic proof of all diary entries. Offline verification ensures regulatory audits can proceed without dependency on external services, and proof portability supports long-term archival requirements under FDA 21 CFR Part 11.

End Bitcoin-Based Timestamp Implementation | **Hash:** 94499ad5

DIARY-PRD-evidence-timestamp-verification: Timestamp Verification Interface

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-evidence-timestamp-attestation-H+K

Assertions

- A. The system SHALL provide verification capability for all timestamp proofs.
- B. The system SHALL enable users to verify timestamp proofs on-demand for any timestamped data.
- C. The system SHALL enable regulators to verify timestamp proofs on-demand for any timestamped data.
- D. The verification interface SHALL clearly indicate verification results as valid, invalid, or pending.
- E. The verification interface SHALL display the attested timestamp using Bitcoin block time.
- F. The verification interface SHALL enable verification without requiring specialized technical knowledge.
- G. The system SHALL provide export capability for verification evidence suitable for regulatory submissions.
- H. The system SHALL make verification available for any diary entry with timestamp proof.
- I. The verification interface SHALL communicate results clearly to non-technical users.
- J. The system SHALL generate verification reports that are exportable for regulatory documentation.
- K. The system SHALL clearly indicate the reason for failure when verification fails.

Rationale

Timestamp proofs provide cryptographic evidence of when data existed, but this evidence is only valuable if stakeholders can independently verify it. FDA 21 CFR Part 11 requires that electronic records be readily retrievable and verifiable by regulatory

inspectors. This requirement ensures that users, auditors, and regulators can confirm the integrity and timing of timestamped data without requiring specialized blockchain expertise. The verification interface bridges the gap between complex cryptographic proofs and regulatory accessibility requirements.

End Timestamp Verification Interface | **Hash:** 9956bd94

DIARY-PRD-evidence-timestamp-archival: Timestamp Proof Archival

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-evidence-timestamp-attestation-G

TODO (URS-Phase-3 reconciliation): Additional Refines targets REQ-p00012-C, REQ-p00012-E, REQ-p00012-F (Clinical Data Retention Requirements) were in URS-replaced prd-clinical-trials.md. Re-parent to the URS-derived clinical-data-retention REQ when that lands.

Assertions

- A. The system SHALL archive all timestamp proofs alongside clinical trial data for the required retention period.
- B. The system SHALL store timestamp proofs durably with their associated diary data.
- C. The system SHALL include all timestamp proofs in data exports.
- D. The system SHALL include all timestamp proofs in data backups.
- E. Timestamp proofs SHALL remain valid independent of system availability.
- F. The system SHALL ensure timestamp proofs are retrievable for regulatory review at any time.
- G. The system SHALL preserve timestamp proofs through system migrations without corruption.
- H. The system SHALL preserve timestamp proofs through system upgrades without corruption.
- I. The system SHALL support retrieval of timestamp proofs independently of application availability.
- J. The system SHALL document the timestamp proof format to enable long-term interpretation.

Rationale

Clinical trial data must be retained for 15-25 years per regulatory requirements. Timestamp proofs serve as cryptographic evidence of data integrity and must be preserved alongside the data they verify to maintain verifiability throughout the entire retention

period. This requirement ensures that timestamp proofs remain accessible and valid for regulatory review, even as systems evolve through migrations and upgrades. The proofs must be self-contained and interpretable independent of the original system to support long-term audit and compliance verification.

End Timestamp Proof Archival | **Hash:** 69a49395

DIARY-PRD-evidence-device-fingerprint: Device Fingerprinting

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-evidence-timestamp-attestation-B

Assertions

- A. The system SHALL record a device fingerprint with each data submission.
- B. The system SHALL derive device fingerprints from device hardware attributes as unique, non-reversible identifiers.
- C. The system SHALL use one-way hash functions to generate device fingerprints.
- D. The system SHALL include the device fingerprint in the timestamped evidence record for each submission.
- E. The system SHALL generate consistent fingerprints across multiple sessions on the same device.
- F. The system SHALL enable independent verification that data originated from a specific device.
- G. The system SHALL NOT store raw device identifiers.
- H. The system SHALL NOT transmit raw device identifiers.
- I. Auditors SHALL be able to verify fingerprint consistency across a patient's submissions.

Rationale

Device fingerprinting establishes how data was collected by binding each submission to a specific device. Combined with timestamp attestation (when) and patient authentication (who), this completes the chain of evidence required for ALCOA+ compliance. This requirement supports FDA 21 CFR Part 11 by providing attributable evidence of the data collection method and enables verification that submissions originated from authenticated devices throughout the trial period.

End Device Fingerprinting | **Hash:** 8e10b85a

DIARY-PRD-evidence-patient-authentication: Patient Authentication for Data Attribution

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-evidence-timestamp-attestation-B

Assertions

- A. The system SHALL authenticate patients before data entry to establish privileged access to the enrolled device.
- B. The system SHALL use the device's native lock screen as the primary authentication mechanism.
- C. When enabled by Sponsor configuration, the system SHALL require an in-app PIN as a fallback authentication mechanism when the device lock screen is not enabled.
- D. The system SHALL detect whether the device has a lock screen enabled.
- E. The system SHALL detect device lock screen status at patient enrollment.
- F. The system SHALL periodically detect device lock screen status after enrollment.
- G. When the in-app PIN fallback is enabled, the system SHALL prompt patients to set a PIN when the device lock screen is not enabled.
- H. When the in-app PIN fallback is enabled, the system SHALL allow Site Coordinators to send PIN reset notifications to patients.
- I. When the in-app PIN fallback is enabled, the system SHALL NOT allow Site Coordinators to view patient PINs.
- J. The system SHALL record authentication status with each data submission.
- K. The system SHALL log failed authentication attempts for audit purposes.
- L. The system SHALL use device-specific UUID binding as an identity assurance control, establishing a controlled association between the enrolled patient and registered application instances.
- M. The system SHALL treat the combination of mandatory device-level lock screen authentication and device UUID binding as equivalent to application-level login credentials for the purpose of patient identity assurance during data submission.

Rationale

This requirement establishes the authentication framework for attributing clinical trial data to the correct enrolled patient. In a bring-your-own-device (BYOD) clinical trial context, the patient's personal device with an active lock screen provides the strongest available authentication mechanism. By verifying privileged access to the enrolled device, the system creates reasonable assurance that data entries originate from the enrolled patient rather than an unauthorized user. This approach balances FDA 21

CFR Part 11 identity verification requirements with the practical constraints of mobile clinical trials. The fallback PIN mechanism ensures authentication is possible even when patients choose not to enable device-level security, while the PIN reset workflow maintains security without creating support burden for site staff. For the full regulatory rationale supporting the risk-based authentication design, see [docs/position-app-auth.md](#).

End Patient Authentication for Data Attribution | **Hash:** fc02cd59

DIARY-PRD-evidence-geolocation: Optional Geolocation Tagging

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-evidence-timestamp-attestation-B

Assertions

- A. The system SHALL support optional geolocation tagging of data submissions.
- B. Geolocation tagging SHALL be disabled by default for all trials.
- C. Geolocation tagging SHALL require explicit Sponsor enablement on a per-trial basis.
- D. The system SHALL only collect geolocation data when device location services are available.
- E. The system SHALL only collect geolocation data when the patient has granted location permissions to the app.
- F. The system SHALL record location coordinates with each data submission when geolocation is enabled and permitted.
- G. Geolocation data SHALL be included in the timestamped evidence record when collected.
- H. The app SHALL request location permission from the patient with a clear explanation when geolocation is enabled for a trial.
- I. The system SHALL clearly inform patients when geolocation is being collected.
- J. The app SHALL display the geolocation collection status in the settings interface.
- K. The system SHALL allow data entry to proceed successfully when location data is unavailable due to denied permissions.
- L. The system SHALL allow data entry to proceed successfully when location data is unavailable due to disabled location services.
- M. Geolocation collection settings SHALL be configurable at the trial level.
- N. Geolocation collection settings SHALL be configurable at the Sponsor level.

Rationale

Geolocation provides additional evidence of data collection context, strengthening provenance claims and supporting data integrity verification. Location data is considered potential PII under privacy regulations, requiring explicit consent and transparency. This requirement balances the evidentiary value of geolocation with privacy protection and regulatory compliance by implementing a multi-layered consent model (Sponsor enablement, device permissions, patient awareness).

End Optional Geolocation Tagging | **Hash:** f9a69607

DIARY-PRD-evidence-email-identity: Hashed Email Identity Verification

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-evidence-timestamp-attestation-B

Assertions

- A. The system SHALL record a hashed patient *Email Address* as an identity fingerprint with enrollment data.
- B. The system SHALL include the hashed email in the evidence record for each data submission.
- C. The system SHALL hash patient email using a standard, documented algorithm.
- D. The hashed email SHALL be recorded at enrollment and verifiable against submissions.
- E. The system SHALL enable Sponsors to retrieve the original email for auditor disclosure separate from the evidence record.
- F. The system SHALL allow auditors to independently hash a provided email and confirm it matches the stored hash value.
- G. The system SHALL support auditor contact with the patient via the verified *Email Address*.
- H. The hash algorithm SHALL be documented for long-term reproducibility.

Rationale

This requirement establishes a privacy-preserving identity verification mechanism for clinical trial data by using cryptographically hashed email addresses. The hash serves as a tamper-evident fingerprint that links data submissions to a specific patient without exposing personally identifiable information (PII) in the evidence record. This approach enables auditors to independently verify data provenance by contacting patients directly through their verified *Email Address*, supporting FDA 21 CFR Part 11 audit trail requirements while maintaining HIPAA compliance. The sponsor maintains

the original email separately for auditor disclosure when needed, allowing independent hash verification while keeping PII out of the main evidence chain.

End Hashed Email Identity Verification | **Hash:** 92c5e2b7

1.8 DIARY-PRD-help-resources: Help and Resources

Level: prd | **Status:** Legacy | **Implements:** -

Overview

The **Sponsor Portal** provides users with access to in-portal help materials including video tutorials, frequently asked questions, an interactive guided tour, and a downloadable user guide. The materials surfaced are role-aware so that each user is presented with content relevant to their responsibilities. The detailed inventory of materials, tour content, and FAQ entries is maintained in internal documentation; this requirement defines only the platform-level expectation that the help affordance exists and is available to all **Sponsor Portal** roles.

Definitions

Help and Resources: The collection of in-portal help materials available to **Sponsor Portal** users, comprising video tutorials, frequently asked questions, an Interactive Tour, and a downloadable user guide.

Interactive Tour: A guided walkthrough of the **Sponsor Portal** interface presented to a user on first login and replayable on demand from **Help and Resources**.

Assertions

Availability

A. The **System** SHALL make **Help and Resources** available to every **Administrator**, **Clinical Research Associate**, and **Study Coordinator** from any screen of the **Sponsor Portal**.

Content

B. The **System** SHALL present **Help and Resources** content scoped to the user's active **Role**.

C. **Help and Resources** SHALL include video tutorials, frequently asked questions, the **Interactive Tour**, and a downloadable user guide.

Interactive Tour

D. The **System** SHALL present the **Interactive Tour** to a user on their first successful login to the **Sponsor Portal**.

E. The **System** SHALL allow a user to replay the **Interactive Tour** on demand from **Help and Resources**.

Rationale

Clinical staff using the **Sponsor Portal** are not full-time users of the system — they engage with it intermittently during their participant-facing workday. In-portal, always-available help is essential to keep training overhead low and to support correct use without external documentation lookups. Role-aware content keeps each user's help surface focused on what they can actually do (a Study Coordinator should not have to filter through Administrator topics, and vice versa). The **Interactive Tour** on first login bootstraps new users into the portal's primary surfaces; replayability supports occasional refreshers without rerunning onboarding. This requirement deliberately defines only the affordance and the major content categories — the precise tour script, FAQ entries, and tutorial inventory are operational artifacts maintained in internal documentation, since they evolve continuously with the product.

End Help and Resources | **Hash:** 66c10c7b

1.9 Mobile Application Foundation

This file groups the foundational platform-level requirements that define the **Mobile Application** itself: the dual-mode (personal vs. linked) operating model, offline-first data entry, the **Diary Start Day** invariant, the sponsor-configurable **Clinical Trial Privacy Policy**, and the **Application Lock**. Sponsor-side configuration overlays (CAL-PRD-privacy-policy-configuration, CAL-PRD-application-lock-configuration) live in the hht_diary_callisto repo and refine the PRDs in this file.

DIARY-PRD-mobile-application: Diary Mobile Application

Level: prd | **Status:** Legacy | **Implements:** -

Overview

The mobile application serves dual purposes: personal health tracking for individual **Users** and compliant data capture for clinical trials. In personal use mode the application requires no account and stores data locally. In linked use mode the application synchronizes data with the sponsor portal for clinical trial participation.

Definitions

Mobile Application: The iOS and Android application provided by the System for participant-reported diary data capture and questionnaire completion. Operates in personal use mode without an account or in linked use mode connected to a **Sponsor Portal** deployment.

User: An individual using the **Mobile Application** in personal use mode, without an account and without a link to a clinical trial. A User who subsequently links to a study becomes a **Participant**.

Assertions

- A. The System SHALL provide a mobile application for iOS platforms, available via the iOS app store.
- B. The System SHALL provide a mobile application for Android platforms, available via the Android app store.
- C. The mobile application SHALL support full offline operation for core diary functions in both personal use mode and linked use mode.
- D. The mobile application SHALL NOT require account creation or login for personal use mode.

- E. The **User** SHALL retain control over locally-entered data, including the right to delete it from the device while in personal use mode.
- F. The mobile application SHALL obtain explicit **User** consent before synchronizing pre-existing local data to the **Sponsor Portal** upon linking.

Rationale

The dual-mode design serves two distinct populations from a single codebase: individuals tracking nosebleeds for personal health reasons (no clinical-trial context, no sponsor, no account) and clinical-trial participants whose data feeds the sponsor's regulatory submission. Personal mode is account-less by design — it removes onboarding friction for the first population, who would not benefit from an account they cannot use against any backend, and it keeps the device the single point of control for their data. Linked mode adds the **Sponsor Portal** synchronization path; the explicit-consent gate on first sync ensures the **User** transitioning to **Participant** affirmatively chooses to share their previously-private local entries with the sponsor, rather than that data being silently uploaded on link. iOS and Android coverage is required because participants in any plausible clinical-trial population will hold devices on both platforms; restricting to one would exclude participants without a fallback.

End Diary Mobile Application | **Hash:** f8e5cbb6

DIARY-PRD-mobile-offline-first: Offline-First Data Entry

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-mobile-application

Overview

Offline-first architecture ensures reliable data collection regardless of network conditions. All **Users** benefit from local storage for immediate, reliable data entry. For **Participants** linked to clinical trials, automatic synchronization provides cloud backup and enables remote monitoring while maintaining the offline-first experience.

Assertions

- A. The System SHALL allow **Users** to create diary entries without requiring internet connectivity.
- B. The System SHALL allow **Users** to edit diary entries without requiring internet connectivity.
- C. The System SHALL allow **Users** to view their complete entry history without requiring internet connectivity.
- D. For linked **Participants**, the System SHALL indicate which entries have not yet synchronized to the **Sponsor Portal**.

E. For linked **Participants**, the System SHALL automatically synchronize unsynchronized entries when network connectivity becomes available.

Rationale

Diary entries are time-sensitive: a nosebleed event must be captured as it occurs or shortly afterward to preserve recall accuracy and satisfy the ALCOA+ Contemporaneous principle. A network-required entry model would push participants toward either delayed entry (when connectivity is restored, by which time recall has degraded) or non-entry (when connectivity never recovers in the relevant window). Offline-first inverts that: every entry succeeds locally regardless of network state, and synchronization to the **Sponsor Portal** is a background concern handled by the platform whenever connectivity is available. The unsynchronized indicator preserves transparency for the participant — they can see whether their data has reached the sponsor — without making sync a gate on entry.

End Offline-First Data Entry | **Hash:** d9631250

DIARY-PRD-diary-start-day: Diary Start Day Definition

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Clinical trial **Participants** may need to record nosebleeds that occurred before their first Mobile Application usage. The **Diary Start Day** establishes the earliest date for which diary entries are valid, balancing the need for historical data capture with the requirement to maintain reliable data quality. The **Diary Start Day** is set automatically based on the **Participant**'s actual entries, when a **Participant** records data for a date earlier than any prior entry, the **Diary Start Day** moves backward to that date. The **Diary Start Day** never moves forward, even if the earliest entry is subsequently deleted, to preserve the historical scope of the diary and ensure that documented gaps remain visible.

Definitions

Diary Start Day: The earliest date for which diary entries are valid for a given **User**.

Assertions

- A. The System SHALL establish and maintain a **Diary Start Day** for each **User**.
- B. The System SHALL set the **Diary Start Day** to the date of the earliest entry the **Participant** has ever recorded.

- C. When a **Participant** records an entry for a date earlier than the current **Diary Start Day**, the System SHALL move the **Diary Start Day** backward to that date.
- D. The System SHALL NOT allow the **Diary Start Day** to be set earlier than 365 days before mobile application installation on the current device.
- E. The **Diary Start Day** SHALL only move backward, never forward, once set.
- F. The System SHALL NOT prompt **Users** to set the **Diary Start Day** during onboarding.
- G. Data entered for dates before the trial start date SHALL NOT synchronize to the **Sponsor Portal** or Rave EDC.
- H. The **System** SHALL NOT allow the **Participant** to record an entry for a date in the future.
- I. The **System** SHALL display each calendar day between the **Diary Start Day** and the current day, exclusive of any day for which a **Daily Status** has been recorded, as a missing day in the Calendar.

Rationale

The **Diary Start Day** is the single boundary the **Mobile Application** uses to distinguish “no entry expected” from “missing entry” — every day from the **Diary Start Day** to today is part of the diary period and therefore a candidate for the missing-day display in the calendar. Automatic backward expansion (set to the earliest entry, can move backward but never forward) reflects two operational realities: participants discover the need to backfill historical events incrementally rather than declaring a start date up front, and once a date is recognized as part of the diary period, the visible gaps on dates within it are evidence the audit and sponsor reporting rely on — collapsing those gaps by moving the start day forward when the earliest entry is deleted would silently erase that evidence. The 365-day floor below installation date caps the historical window at a clinically meaningful range while leaving room for retrospective capture early in a trial. Future-date rejection is a contemporaneous-data integrity floor. The non-sync rule for pre-trial dates keeps the **Sponsor Portal** dataset bounded to the trial period regardless of how far back the personal diary extends.

End Diary Start Day Definition | **Hash:** 6550e619

DIARY-PRD-privacy-policy: Clinical Trial Privacy Policy

Level: prd | **Status:** Legacy | **Implements:** -

Overview

The Mobile Application SHALL provide access to a **Clinical Trial Privacy Policy** governing sponsor-side data handling for a linked study, distinct from the **Application Privacy Policy**.

Definitions

Application Privacy Policy: The privacy policy governing the Mobile Application itself, covering platform-level data handling.

Clinical Trial Privacy Policy: The privacy policy governing a Participant's participation in a specific clinical trial, covering sponsor-side data handling. Sponsor-configurable per deployment.

Assertions

- A. The System SHALL support a sponsor-configurable **Clinical Trial Privacy Policy** per deployment.
- B. The System SHALL make the **Clinical Trial Privacy Policy** accessible to the Participant only when the Participant is linked to a study or is in the process of linking to a study.
- C. The System SHALL retain the **Clinical Trial Privacy Policy** version that was in effect at the time of consent against the Participant record.

Rationale

Two privacy policies coexist in the **Mobile Application** because two distinct controllers handle distinct data flows: the **Application Privacy Policy** covers platform-level data handling (the **Mobile Application** itself, the platform vendor's data-processor role), while the **Clinical Trial Privacy Policy** covers the sponsor's clinical-trial data handling for participants in a specific study. Surfacing the trial-specific policy only when the participant is linked or actively linking prevents the application from showing sponsor-specific terms to **Users** who are using the app in personal mode and have no relationship with the sponsor. Retaining the version in effect at time of consent against the participant record satisfies the regulatory expectation that the document the participant actually consented to (which may change over the life of the trial) is preserved as evidence of informed consent.

End Clinical Trial Privacy Policy | **Hash:** 8f74043e

DIARY-PRD-application-lock: Application Lock

Level: prd | **Status:** Legacy | **Implements:** -

Overview

The Mobile Application contains Participant-reported clinical data and identifying information that must be protected from unauthorized access on the Participant's device. An Application Lock requires the Participant to authenticate before each use, ensuring

that someone who picks up an unattended device cannot view or modify diary entries. The lock is configurable so each sponsor deployment can select the authentication method appropriate to its risk profile and Participant population. When the configured method is unavailable on a Participant's device, a fallback to Device Authentication ensures continuity of access without compromising the protective intent.

Definitions

Application Lock: The state in which the **Mobile Application** requires the **Participant** to authenticate before any **Participant**-facing screen, action, or data is accessible.

PIN: A numeric secret of configurable length set by the **Participant** and used to release the **Application Lock**.

Device Authentication: Authentication performed by the device operating system using the credential the **Participant** has configured at the device level, including but not limited to device passcode, fingerprint, or face recognition.

Authentication Method: The mechanism required to release the **Application Lock**. The configured **Authentication Method** is one of: **PIN**, **Device Authentication**, or none.

Idle Timeout: The configurable elapsed time the **Mobile Application** may remain in the background before the **System** re-applies the **Application Lock** on return to the foreground.

Assertions

Lock Application

- A. The **System** SHALL apply the **Application Lock** when the **Mobile Application** is launched from a fully closed state.
- B. The **System** SHALL apply the **Application Lock** when the **Mobile Application** returns to the foreground after spending more than the **Idle Timeout** in the background.
- C. While the **Application Lock** is applied, the **System** SHALL NOT display **Participant** data and SHALL NOT permit any **Participant** action other than authentication.

Authentication Method

- D. The **System** SHALL release the **Application Lock** only upon successful authentication using the configured **Authentication Method**.
- E. When the configured **Authentication Method** is **PIN**, the **System** SHALL require the **Participant** to enter their **PIN** to release the **Application Lock**.
- F. When the configured **Authentication Method** is **Device Authentication**, the **System** SHALL invoke the device operating system's authentication prompt to release the **Application Lock**.

G. When the configured **Authentication Method** is none, the **System** SHALL NOT apply the **Application Lock**.

PIN Setup and Reset

H. When the configured **Authentication Method** is **PIN** and the **Participant** has not yet set a **PIN**, the **System** SHALL require the **Participant** to set a **PIN** before any **Participant** data is accessible.

I. The **System** SHALL require the **Participant** to enter the **PIN** twice during setup and SHALL reject the setup when the two entries do not match.

J. The **System** SHALL allow the **Participant** to change their **PIN** from the **Mobile Application** settings, after first authenticating with their existing **PIN** or **Device Authentication**.

Failed Attempt Handling

K. The **System** SHALL track consecutive failed **PIN** entry attempts.

L. When the number of consecutive failed **PIN** entry attempts reaches the configured **Failed Attempt Threshold**, the **System** SHALL fall back to **Device Authentication**.

M. Upon successful **Device Authentication** following a fallback, the **System** SHALL require the **Participant** to reset their **PIN** before any **Participant** data is accessible.

N. The **System** SHALL reset the failed attempt counter upon successful authentication.

Configuration

O. The **System** SHALL support sponsor-configurable selection of the **Authentication Method** per deployment.

P. The **System** SHALL support sponsor-configurable **PIN** length per deployment.

Q. The **System** SHALL support sponsor-configurable **Idle Timeout** per deployment.

R. The **System** SHALL support sponsor-configurable **Failed Attempt Threshold** per deployment.

Rationale

The **Application Lock** is the per-device gate that protects participant clinical data when a device is left unattended, lost, or stolen. The lock applies at two boundaries — fresh launch and resume-from-background-after-idle — because both are realistic re-entry points after an interval in which the device could have changed hands. The “no data, no action other than authentication” rule while locked closes the leakage channels that would otherwise exist (e.g., notification preview, deep-link routing into a partial workflow). The sponsor-configurable **Authentication Method** lets each deployment choose between three security-posture options: **PIN** (cheapest, works on every device, controlled by the platform), **Device Authentication** (delegates to the OS-managed credential, which may include biometrics), or none (when the deployment determines a separate lock is not required). The failed-attempt fallback to **Device Authentication** rather than account lockout preserves participant access — a

participant who has forgotten their **PIN** can still recover by authenticating against the OS credential and resetting — while still raising the bar against a casual unauthorized attempt. Failed-attempt counter reset on success prevents accumulated rare typos from compounding into a fallback over normal use.

OPEN: Should the **Application Lock** be bypassed for **Push Notification** taps that lead to time-sensitive flows (e.g., Ongoing Epistaxis Event Reminder), or always enforced?

End Application Lock | **Hash:** cca93728

1.10 Participant Tasks and Notifications

This file groups every notification and reminder the **Mobile Application** delivers to the **Participant**, together with the on-screen Task List and Participation Status Badge that surface task-state information in the app itself. The seven push notifications defined here all conform to the cross-cutting DIARY-PRD-notification-behavior template (tap behavior, no in-notification actions, offline-deferred delivery). Sponsor-side configuration of the trigger offsets and notification text live in the `hht_diary_callisto` repo (CAL-PRD-notification-incomplete-record-lock-configuration, CAL-PRD-notification-CAL-PRD-notification-historical-gap-configuration).

DIARY-GUI-participant-task-list: Participant Task List

Level: GUI | **Status:** Legacy | **Implements:** -

Overview

The task list provides **Participants** with a prioritized set of actions requiring their attention. Displaying tasks prominently on the **Main Screen** and linking directly to the relevant flows reduces friction and encourages timely data entry. Tasks are automatically resolved as the **Participant** addresses them, keeping the list current.

Definitions

Task List: The prioritized set of actionable items displayed on the **Participant's Main Screen** requiring **Participant** attention.

Incomplete Records Task: A task indicating the **Participant** has one or more saved entries with missing required data.

Questionnaire Task: A task indicating a **Portal-Sent Questionnaire** has been sent to the **Participant** and has not yet been submitted.

Yesterday Reminder Task: A task prompting the **Participant** to record a **Daily Status** for the previous day.

Assertions

Task Types and Priority

A. The **Task List** SHALL support the following task types, displayed in the priority order listed:

Priority	Task Type	Trigger	Removal
1	Incomplete Records Task	Participant has one or more saved entries with missing required data	All incomplete entries are completed or deleted
2	Questionnaire Task	Study Coordinator sends a Portal-Sent Questionnaire	Sponsor finalizes the questionnaire
3	Yesterday Reminder Task	New day begins and the Participant has not recorded a Daily Status for the previous day	Participant records a Daily Status for the previous day

Incomplete Records Task

B. The **Incomplete Records Task** SHALL display the count of incomplete entries.

C. When the **Participant** selects the **Incomplete Records Task**, the interface SHALL navigate to a screen listing all incomplete entries.

D. From the incomplete entries screen, the **Participant** SHALL be able to complete or delete each entry.

E. The **Incomplete Records Task** SHALL persist regardless of the age of the incomplete entries.

Questionnaire Task

F. The interface SHALL display one **Questionnaire Task** per **Questionnaire Type**.

G. When the **Participant** selects a **Questionnaire Task**, the interface SHALL navigate to the questionnaire flow for that **Portal-Sent Questionnaire**.

H. After the Participant submits a **Portal-Sent Questionnaire**, the questionnaire SHALL be accessible as a record on the day it was submitted via the **Calendar**.

I. After the sponsor finalizes the **Portal-Sent Questionnaire**, the **Questionnaire Task** SHALL be removed from the **Task List**.

J. After **Submission**, the **Questionnaire Task** SHALL display a completed visual state indicating the questionnaire has been submitted and is awaiting sponsor review.

K. While the **Questionnaire Task** is in a submitted state, the Participant SHALL be able to select it to review and edit their answers.

Yesterday Reminder Task

L. The **Yesterday Reminder Task** SHALL present three response actions: Yes, No, and Don't Remember.

- M. When the **Participant** selects No, the interface SHALL record a **Daily Status** of No Nosebleed for the previous day and remove the task.
- N. When the **Participant** selects Don't Remember, the interface SHALL record a **Daily Status** of Don't Remember for the previous day and remove the task.
- O. When the **Participant** selects Yes, the interface SHALL navigate the **Participant** to the nosebleed recording flow with the date set to the previous day.
- P. The **Yesterday Reminder Task** SHALL NOT appear if the **Participant** has already recorded a **Daily Status** for the previous day.

General Behavior

- Q. Tasks SHALL be removed from the **Task List** when their removal condition is met.
- R. The **Task List** SHALL update as conditions change without requiring the **Participant** to refresh.

Rationale

The three task types correspond to the three categories of follow-up the platform needs to surface to the participant: data the participant already entered that is missing required fields (**Incomplete Records Task**), questionnaires the **Study Coordinator** has assigned that have not been submitted (**Questionnaire Task**), and the daily-status prompt for yesterday (**Yesterday Reminder Task**). The priority ordering reflects intervention urgency: incomplete records are at risk of becoming permanently locked (highest priority), questionnaires are due but not yet on a hard deadline (middle), and yesterday's status is a soft prompt that auto-resolves over time (lowest). The **Yesterday Reminder Task**'s three-action surface (Yes / No / Don't Remember) lets the participant resolve the task without leaving the Main Screen for the common case (Yes navigates to recording; No and Don't Remember commit the **Daily Status** in place). Removing tasks the moment their removal condition is met (and not requiring a refresh) keeps the list honest — a participant who completes their incomplete records sees the task disappear immediately, rather than being told to refresh or re-open the app. **Incomplete Records Task** persistence regardless of age (assertion E) is the operational counterpart of the **Lock Threshold**: even after the underlying record is locked and can no longer be completed, the task stays visible so the participant is aware of the now-permanently-incomplete state.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

End Participant Task List | **Hash:** a070f8d5

DIARY-PRD-notification-disconnection: Disconnection Notification

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Participants need immediate, clear notification when their mobile Application is disconnected from the sponsor portal. A persistent, non-dismissible notification ensures the **Participant** is always aware of their disconnected state and cannot accidentally or intentionally hide it.

Definitions

Disconnection Notification: A persistent, non-dismissible notification displayed to the **Participant** when their mobile Application is disconnected from the **Sponsor Portal**.

Assertions

- A. When a **Participant's** status is **Disconnected**, the System SHALL display a **Disconnection Notification** on the **Main Screen**.
- B. The **Disconnection Notification** SHALL persist until the **Participant** is reconnected to the **Sponsor Portal**.
- C. The **Disconnection Notification** SHALL NOT be dismissible by the **Participant**.
- D. The System SHALL support sponsor-configurable message content for the **Disconnection Notification**.
- E. When no sponsor-specific message content is configured, the **Disconnection Notification** SHALL display the following default message: "Your connection with the study has been interrupted. Please contact your study site for assistance."

Rationale

Disconnection is an operational state with clinical-data consequences: data the participant enters while disconnected does not reach the sponsor and may sit unsynced for an indeterminate period. The participant needs to know they are disconnected so they can either resolve it (contact their site for a new linking code) or at minimum understand why their data is not visible to the sponsor. Persistent + non-dismissible is the only display mode that survives the participant's natural impulse to clear unfamiliar notifications: a dismissible disconnection banner would be cleared within minutes by most participants, and the disconnection would then be invisible to them on every subsequent app open. The sponsor-configurable message + default text lets each deployment customize the language (e.g. site contact phone numbers, terminology

consistent with the sponsor's other communications) while keeping a working default for deployments that don't configure it.

Screen reference

See: Disconnection Notification

End Disconnection Notification | **Hash:** dd326d38

DIARY-GUI-participation-status-badge: Participation Status Badge

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-notification-disconnection

Overview

The **Participation Status Badge** gives **Participants** a persistent, at-a-glance view of their clinical trial involvement. The badge adapts its display based on the **Participant's** current status so they always know whether they are connected, disconnected, or no longer participating in a study.

Definitions

Participation Status Badge: The visual component displayed in the Clinical Trial section of the user profile screen showing the **Participant's** current study participation state and associated details.

Assertions

Placement

A. The interface SHALL display the **Participation Status Badge** in the Clinical Trial section of the user profile screen.

Linked State

B. When the **Participant's** status is **Linked - Awaiting Start** or **Trial Active**, the **Participation Status Badge** SHALL display the sponsor logo, the **Participant's** linking code, and the date and time the **Participant** joined.

C. The **Participation Status Badge** SHALL include a link to the **Clinical Trial Privacy Policy** from the moment the Participant links to the study, and the link SHALL remain available thereafter regardless of subsequent status changes.

Disconnected State

D. When the **Participant's** status is **Disconnected**, the **Participation Status Badge** SHALL display a warning indicator, the **Participant's** current linking code, and a message that the connection has been interrupted.

E. When the **Participant's** status is **Disconnected**, the **Participation Status Badge** SHALL present an Enter New Linking Code action that navigates the **Participant** to the linking code entry screen.

Not Participating State

F. When the **Participant's** status is **Not Participating**, the **Participation Status Badge** SHALL display in an inactive visual style with the end date of participation.

Automatic Update

G. The **Participation Status Badge** SHALL update automatically when the **Participant's** status changes.

H. The System SHALL support sponsor-configurable display of the sponsor logo on the **Participation Status Badge** when the **Participant's** status is **Not Participating**.

Rationale

The badge consolidates “where do I stand with this study” into a single visual the participant always finds in the same place (Clinical Trial section of the User Profile). Three state variants cover the three operational situations: actively-linked (sponsor logo, linking code, join date — full participation context), disconnected (warning indicator, linking code, Enter New Linking Code action — the recovery path is reachable from the badge itself), and not-participating (inactive style with end date — the badge persists as historical evidence of past participation without competing for attention against current state). The Clinical Trial Privacy Policy link persists across status changes because the policy version the participant consented to is part of their permanent record, and they have a continuing right to retrieve it. Automatic update on status change is the standard freshness guarantee; sponsor-configurable logo display in the Not Participating state lets the deployment decide whether the badge in its end-of-participation state still carries sponsor branding (an end-of-trial communication preference).

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

End Participation Status Badge | **Hash:** a6b22a5f

DIARY-PRD-notification-incomplete-record-lock: Incomplete Record Lock Warning Notification

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Once an **Incomplete Record** reaches the Lock Threshold defined in DIARY-PRD-entry-time-restrictions, the Participant cannot complete, edit, or delete it — the record is permanently retained in its incomplete state. To give Participants a final opportunity to act before this irreversible lock, the **System** sends a push notification at a configurable time before the lock fires. This requirement defines the platform mechanism; the trigger offset and notification text are sponsor-configurable.

Definitions

Lock Warning Offset: The configurable elapsed time before the Lock Threshold at which the **System** sends a push notification to the Participant about an **Incomplete Record** that has not been completed or deleted.

Assertions

Notification Behavior

- A. The **System** SHALL send a push notification to the Participant when an **Incomplete Record** reaches the Lock Warning Offset before the Lock Threshold defined in DIARY-PRD-entry-time-restrictions.
- B. The **System** SHALL send the notification only once per **Incomplete Record**.
- C. When a Participant completes or deletes an **Incomplete Record** before the Lock Warning Offset is reached, the **System** SHALL NOT send the notification for that record.

Configuration

- D. The **System** SHALL support a sponsor-configurable Lock Warning Offset per deployment.
- E. The **System** SHALL support sponsor-configurable notification text per deployment.
- F. When the Lock Warning Offset is not configured for a deployment, the **System** SHALL NOT send the notification.
- G. The Lock Warning Offset SHALL be less than the Lock Threshold defined in DIARY-PRD-entry-time-restrictions.

Rationale

The **Lock Threshold** produces an irreversible state — once exceeded, the **Incomplete Record** is permanently retained in its missing-field state and cannot be completed, edited, or deleted. The lock-warning notification exists because that irreversible outcome benefits from one final participant nudge before it lands; without the warning, a participant who simply forgot about an incomplete record discovers

the permanent lock only on next app open, long after they could have done anything about it. Once-per-record (assertion B) prevents notification spam from a single long-lingering record; suppression on completion or deletion before the warning offset (assertion C) avoids notifying about records the participant has already resolved. The opt-out semantics (not configured = no notification) and the Lock-Warning-less-than-Lock-Threshold invariant compose with the parent **Entry Time Restrictions** REQ — if a deployment hasn't configured the parent thresholds, the warning has no boundary to anchor to and is therefore not sent.

End Incomplete Record Lock Warning Notification | **Hash:** f3bfa7b3

DIARY-PRD-notification-portal-sent-questionnaire: Portal-Sent Questionnaire Notification

Level: prd | **Status:** Legacy | **Implements:** -

Overview

When a **Study Coordinator** sends a **Portal-Sent Questionnaire** from the Sponsor Portal, the **Participant** must be made aware that a new questionnaire is available for completion. A push notification serves as the awareness mechanism. The Questionnaire Task on the **Main Screen** provides the **Participant's** entry point into the questionnaire flow once the application is open.

Assertions

Trigger

A. When a **Portal-Sent Questionnaire** is successfully delivered to the **Mobile Application**, the System SHALL deliver a **Push Notification** to the **Participant**.

Offline Delivery

B. When the **Mobile Application** is offline at the time a **Portal-Sent Questionnaire** is sent, the System SHALL deliver the **Push Notification** when the **Mobile Application** next establishes connectivity.

Suppression

C. The System SHALL NOT deliver a **Push Notification** for a **Portal-Sent Questionnaire** that has already been submitted by the **Participant**.

D. The System SHALL NOT deliver a **Push Notification** for a **Portal-Sent Questionnaire** that has been called back by the **Study Coordinator**.

Rationale

The push notification is the participant's awareness signal that a new **Portal-Sent Questionnaire** is waiting; without it, the participant would discover the questionnaire only when they happen to open the app, which can extend the response window beyond what the **Study Coordinator** intended. Trigger-on-delivery (rather than trigger-on-send-from-portal) is the correct boundary because delivery is what actually puts the questionnaire on the participant's device; sending from the portal to an offline device is a sponsor-side action that has no participant-side effect yet. Offline-deferred delivery follows the cross-cutting notification template — the OS deferred-delivery mechanism is sufficient for the nudge purpose. Submission and call-back suppression closes the two cases where the notification would be misleading: a questionnaire already submitted no longer needs the participant's attention, and a called-back questionnaire is no longer assigned to them and a notification would invite wasted effort on an inactive questionnaire.

Screen reference

See: Portal-Sent Questionnaire Notification

End Portal-Sent Questionnaire Notification | **Hash:** 0346a456

DIARY-PRD-notification-yesterday-entry: Yesterday Entry Reminder Notification

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Clinical trial data completeness under ALCOA+ principles requires a **Daily Status** for each calendar day in the diary period. A daily reminder fired at a configured time of day prompts the **Participant** to review the previous day and respond via the Yesterday Reminder Task on the **Main Screen**.

Definitions

Reminder Time: The configurable time of day, in the **Participant's** local timezone, at which the Yesterday Entry Reminder Notification is delivered.

Assertions

Trigger

A. The System SHALL deliver a **Push Notification** at the **Reminder Time** when no **Daily Status** has been recorded for the previous calendar day.

Timezone

B. The System SHALL evaluate the **Reminder Time** against the **Participant's** device local timezone.

C. The System SHALL evaluate "the previous calendar day" against the **Participant's** device local timezone.

Suppression

D. The System SHALL NOT deliver the **Push Notification** when a **Daily Status** has been recorded for the previous calendar day.

E. The System SHALL deliver at most one Yesterday Entry Reminder Notification per calendar day.

Configuration

F. The System SHALL support sponsor-configurable **Reminder Time** per deployment.

Rationale

ALCOA+ Complete principle requires a **Daily Status** for every day in the diary period; the Yesterday Entry Reminder is the platform's mechanism for prompting participants to maintain completeness day-by-day rather than discovering large gaps weeks later. Local-timezone evaluation (both for the **Reminder Time** and for "previous calendar day") is essential because participants travel — a UTC-anchored reminder would fire at unpredictable local times for a participant who has moved across timezones, undermining the time-of-day intent. Sponsor-configurable **Reminder Time** lets each deployment select a delivery time appropriate to its participant population (e.g. morning for a working-age population). Once-per-day cap (assertion E) prevents pathological repeat-fires; suppression-on-status-recorded prevents the notification from arriving after the participant has already addressed the yesterday gap via the Task List or Calendar.

Screen reference

See: Yesterday Entry Reminder Notification

End Yesterday Entry Reminder Notification | **Hash:** 85edd14a

DIARY-PRD-notification-ongoing-epistaxis: Ongoing Epistaxis Event Reminder

Level: prd | **Status:** Legacy | **Implements:** -

Overview

A **Participant** who starts recording an **Epistaxis Event** but does not complete the record may forget to return and capture the end time, resulting in inaccurate duration data. A configured sequence of escalating reminders prompts the **Participant** to complete the record or confirm it is still ongoing. Once the sequence has fired, no further reminders are sent.

The **Participant** may configure their own reminder schedule for personal use. A sponsor-configured schedule overrides the personal setting for the duration of a clinical trial.

Definitions

Reminder Schedule: An ordered list of elapsed-time intervals defining the timing of successive reminder notifications for an Incomplete Record. Each interval is measured from the time of the previous notification, with the first interval measured from the Participant's most recent interaction with the record.

Assertions

Trigger and Sequence

- A. The **System** SHALL track the elapsed time since the **Participant's** most recent interaction with each **Incomplete Record**.
- B. The **System** SHALL deliver a **Push Notification** to the **Participant** at each interval in the **Reminder Schedule**.
- C. After the final interval in the **Reminder Schedule** has elapsed, the **System** SHALL NOT deliver further reminders for that **Incomplete Record**.

Reset on Interaction

- D. When the **Participant** interacts with an **Incomplete Record**, the **System** SHALL restart the **Reminder Schedule** from the first interval.

Termination

- E. When an **Incomplete Record** is completed, the **System** SHALL stop delivering reminders for that record.
- F. When an **Incomplete Record** is deleted, the **System** SHALL stop delivering reminders for that record.

Configuration

- G. The default **Reminder Schedule** for the **Mobile Application** SHALL be empty, resulting in no reminders being delivered.
- H. The **System** SHALL allow a **Participant** to configure their own **Reminder Schedule** in **Mobile Application** settings.

- I. The **System** SHALL support sponsor-configurable **Reminder Schedule** per deployment.
- J. When a sponsor-configured **Reminder Schedule** is in effect, the **System** SHALL apply the sponsor-configured schedule and SHALL NOT apply the **Participant's** personally configured schedule.

Rationale

A nosebleed event captured with an accurate start time but a missing or wildly-late end time produces unreliable duration data — one of the three primary outcome measures. The Ongoing Epistaxis Event Reminder addresses the natural failure mode where the participant starts the record at the moment the nosebleed begins, then forgets to return when it stops because their attention has moved on. An escalating schedule (intervals measured from previous notification) gives the participant a reasonable chance to respond without saturating their notification surface — once the final interval has fired and no response landed, further nudging is unlikely to help and the data will be flagged via the **Incomplete Records Task** anyway. Reset-on-interaction (assertion D) restarts the schedule when the participant touches the record, on the inference that they are now engaged again and a fresh sequence is the right cadence. Sponsor override of the participant's personal schedule (assertion J) reflects the clinical-trial priority: while participating in a study, the sponsor's clinical protocol drives the reminder cadence; outside a trial, the participant controls their own experience. Empty default (assertion G) is the right "no policy yet" state — a deployment that has not configured a schedule produces no reminders rather than guessing at intervals.

Screen reference

See: Ongoing Epistaxis Event Reminder

End Ongoing Epistaxis Event Reminder | **Hash:** 197fcd05

DIARY-PRD-notification-historical-gap: Historical Gap Reminder

Level: prd | **Status:** Legacy | **Implements:** -

Overview

The Historical Gap Reminder addresses missing **Daily Status** entries on days older than yesterday but still within the editable window.

Definitions

Historical Gap: A calendar day between the **Diary Start Day** and the current day, exclusive of the current day and the previous day, for which the **Participant** has not recorded a **Daily Status**.

Historical Gap Reminder: A daily **Push Notification** delivered to the **Participant** prompting them to address one or more **Historical Gaps**.

Assertions

Trigger

- A. The **System** SHALL evaluate, once per calendar day at the configured **Reminder Time**, whether the **Participant** has any **Historical Gap** within the editable window.
- B. When the **Participant** has at least one **Historical Gap** within the editable window, the **System** SHALL deliver a **Historical Gap Reminder** to the **Participant**.
- C. The **System** SHALL deliver at most one **Historical Gap Reminder** per calendar day.

Editable Window

- D. The **System** SHALL exclude from the **Historical Gap** evaluation any day for which the elapsed time has exceeded the **Lock Threshold**.
- E. In linked use mode, the **System** SHALL exclude from the **Historical Gap** evaluation any day before the trial start date.

Mode-Dependent Default

- F. In personal use mode, the **Historical Gap Reminder** SHALL be disabled by default.
- G. In personal use mode, the **System** SHALL allow the **User** to enable or disable the **Historical Gap Reminder** from the Mobile Application settings.
- H. In linked use mode, the **Historical Gap Reminder** SHALL be enabled by default.

Configuration

- I. The **System** SHALL support sponsor-configurable **Reminder Time** per deployment.
- J. The **System** SHALL support sponsor-configurable notification text per deployment.

Rationale

Historical gaps (days within the diary period that have no **Daily Status**) accumulate when a participant misses the Yesterday Entry Reminder window across multiple days; without a separate prompt, those gaps stay invisible to the participant unless they happen to open the Calendar. The Historical Gap Reminder is the platform's once-daily nudge to address them, evaluated at the same configured **Reminder Time** as the Yesterday reminder so the participant gets a coherent daily reminder cadence. The

editable-window exclusion (assertions D and E) prevents the reminder from prompting action on dates the participant can no longer modify — locked dates and pre-trial-start dates have no action available. The mode-dependent default (disabled in personal use, enabled in linked use) reflects the difference in motivation: a personal-use user has no external party expecting their data and may not want pressure to backfill; a linked **Participant** is held to a clinical-trial completeness standard. The participant retains the ability to override the default in personal mode (assertion G), while linked mode applies the sponsor-configured behavior.

Screen reference

See: Historical Gap Reminder

End Historical Gap Reminder | **Hash:** afad7d55

1.11 DIARY-PRD-notification-behavior: Notification Behavior

Level: prd | **Status:** Legacy | **Implements:** -

Overview

This requirement defines system-wide standards for **Push Notifications** delivered by the **Mobile Application** to the **Participant's** device. The standards defined herein apply to all notifications generated by the **Mobile Application** and shall not be restated within individual notification requirements.

Push Notifications delivered by the **Mobile Application** serve as nudges that prompt the **Participant** to open the application.

Definitions

Push Notification: A notification delivered by the **Mobile Application** to the **Participant's** device notification center via the operating system's notification service.

Assertions

Tap Behavior

- A. When the **Participant** taps a **Push Notification** delivered by the **Mobile Application**, the System SHALL open the **Mobile Application** to the **Main Screen**.
- B. The System SHALL NOT include response actions within the body of a **Push Notification**.

Offline Delivery

- C. When the **Mobile Application** is offline at the time a **Push Notification** is generated, the System SHALL deliver the **Push Notification** when the **Mobile Application** next establishes connectivity.

Rationale

Push notifications in the Diary Platform are nudges, not interaction surfaces: their job is to bring the participant back into the app, where the full clinical context, validation, and audit trail are available. Two consequences follow. First, tapping a notification always opens the **Main Screen** — never a deep link into a partial workflow — so the participant resumes from a known, audit-traceable starting point. Second, the platform never offers response actions inside the notification body (e.g. "Submit answer", "Acknowledge"), because actions taken without the in-app context bypass authentication, validation, and the data-attribution evidence chain. Offline delivery semantics

ensure that a notification generated while the device is disconnected is not dropped; the operating system's deferred-delivery mechanism is sufficient for the nudge purpose and avoids the complexity of an app-level re-notification scheduler.

End Notification Behavior | **Hash:** 8f7b4f54

1.12 Participant Workflows

This file groups the platform-side requirements that govern the **Participant** lifecycle in the sponsor portal: linking-code generation and use, participant registration, the link/disconnect/reconnect/mark-not-participating/reactivate workflows, and the **Participant Dashboard** GUI that the Study Coordinator uses to operate them. Sponsor-specific overlays — disconnection reason lists, mark-as-not-participating reason lists, linking-code expiry configuration, participant-registration source binding, and the participant dashboard configuration — live in the corresponding `hht_diary_callisto` file (`spec/prd-participant.md`).

DIARY-PRD-linking-code-lifecycle: Linking Code Lifecycle Management

Level: prd | **Status:** Legacy | **Implements:** -

Overview

The **Mobile Linking Code** mechanism provides a secure, time-limited channel through which clinical staff can link a **Participant**'s device to the **Sponsor Portal** without requiring the **Participant** to authenticate directly. Single-use enforcement and unpredictability ensure that codes cannot be reused or guessed, while configurable expiry balances security with the practical time **Participants** need to complete the linking process.

Definitions

Mobile Linking Code: A unique, time-limited, single-use code generated by the **System** and provided to a **Participant** by clinical staff. The **Participant** enters the code in the **Mobile Application** to establish a connection between their device and the **Sponsor Portal**.

Participant Linking Code: The **Mobile Linking Code** that was used to establish the connection between a **Participant**'s device and the **Sponsor Portal**. Displayed for reference and troubleshooting purposes. Cannot be used to establish a new connection.

Assertions

Generation

A. When a Study Coordinator initiates the participant linking workflow, the System SHALL generate a unique **Mobile Linking Code** for that participant.

B. The System SHALL generate each **Mobile Linking Code** such that it cannot be predicted or derived from any other known value.

C. The System SHALL generate **Mobile Linking Codes** in a format that is short and alphanumeric to support easy communication to the participant.

Expiry

D. The System SHALL expire a **Mobile Linking Code** after a configurable duration from the time of generation.

E. When a new **Mobile Linking Code** is generated for a participant, the System SHALL immediately invalidate any previously active **Mobile Linking Code** for that participant.

Single-Use

F. The System SHALL invalidate a **Mobile Linking Code** immediately upon successful use.

Rejection

G. The System SHALL reject any attempt to link using a **Mobile Linking Code** that is expired, already used, or does not exist.

Visibility

H. The System SHALL retain the **Participant Linking Code** against the Participant record after successful use for reference and troubleshooting purposes.

Rationale

The **Mobile Linking Code** is the bridge between two systems that otherwise do not share a credential surface: the **Sponsor Portal** (operated by clinical staff with managed accounts) and the **Mobile Application** (operated by a **Participant** with no account). Three properties are essential to that bridge. Unpredictability prevents a code from being guessed or derived in the gap between Study Coordinator generation and **Participant** entry. Single-use enforcement prevents a code that has been written down, photographed, or otherwise leaked from being replayed by a second device. Expiry caps the window during which a leaked or stale code can be used at all. Keeping the code short and alphanumeric is a usability requirement — clinical staff dictate codes to **Participants** over phone or in person, and codes that require strict capitalization or special characters fail in practice. Retaining the **Participant Linking Code** after use (under the **Participant Linking Code** definition rather than the active **Mobile Linking Code**) lets clinical staff confirm which code was used for troubleshooting and audit purposes, without that retention re-enabling use of the spent code.

End Linking Code Lifecycle Management | **Hash:** 15ed4111

DIARY-PRD-participant-registration: Participant Registration

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Clinical trial portals require a defined mechanism for making **Participant** records available to clinical staff. The platform exposes this as a sponsor-configurable hook; concrete bindings (e.g. **Rave EDC** synchronization) live in sponsor overlay requirements.

Definitions

Participant: An individual whose **Mobile Application** has been linked to the **Sponsor Portal** for the purpose of clinical trial data collection. A **Participant** has been registered in the **Sponsor Portal** as a clinical trial subject.

Assertions

- A. The System SHALL support a sponsor-configurable mechanism for adding **Participant** records to the **Sponsor Portal**.
- B. When a **Participant** record is added to the **Sponsor Portal**, the System SHALL assign an initial **Participant** status as defined by the sponsor configuration.

Rationale

Participant records originate outside the **Sponsor Portal** in the systems that own clinical trial enrollment (EDC, IRB-approved enrollment workflows). Modelling registration as a sponsor-configurable mechanism rather than as a platform-fixed hook lets each sponsor wire the portal to the system of record they actually use, while keeping the platform's downstream workflows — linking, disconnection, finalization — agnostic to the upstream source. The sponsor-configurable initial status accommodates protocols that begin participants in different states (paper-then-electronic, electronic-from-day-one) without hard-coding a single starting point.

End Participant Registration | **Hash:** e262998e

DIARY-PRD-participant-link-new: Link New Participant Workflow

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Linking connects a **Participant** record in the **Sponsor Portal** to the **Participant's Mobile Application**, enabling data synchronization and questionnaire distribution. The workflow is initiated by clinical staff and completed by the **Participant** entering the **Mobile Linking Code** in the **Mobile Application**.

Assertions

- A. The System SHALL allow a Study Coordinator to initiate the linking workflow for a **Participant** with **Not Connected** status.
- B. When a Study Coordinator initiates the linking workflow, the System SHALL generate a **Mobile Linking Code**.
- C. When a **Participant** successfully enters a valid **Mobile Linking Code** in the **Mobile Application** for the first time, the System SHALL link the **Participant**'s device to the **Sponsor Portal**.

Rationale

The first-link workflow has two halves separated by a human-physical handoff: the Study Coordinator generates a **Mobile Linking Code** in the portal, and the **Participant** enters that code in the **Mobile Application**. The platform's responsibilities are bounded by that handoff: the portal generates an unguessable, single-use code (per DIARY-PRD-linking-code-lifecycle) and accepts the corresponding entry on the mobile side. Gating the initiation on **Not Connected** status keeps the workflow's precondition explicit — a **Participant** record must exist (typically via the sponsor registration mechanism) and must not already be linked. The first-time qualifier on the **Mobile Application** entry distinguishes initial linking from subsequent reconnection, which uses the same mechanism but a different workflow (DIARY-PRD-participant-reconnection).

End Link New Participant Workflow | **Hash:** eb666103

DIARY-PRD-participant-disconnection: Participant Disconnection Workflow

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Disconnection temporarily severs the link between a **Participant**'s device and the **Sponsor Portal** without removing the **Participant** record or their historical data. Clinical staff may disconnect a **Participant** when there are issues with the device, technical problems, or other circumstances prevent normal trial participation.

Assertions

- A. The System SHALL allow a Study Coordinator to disconnect a **Participant** whose current status permits disconnection.
- B. The System SHALL require a reason before disconnection is applied.
- C. The System SHALL support sponsor-configurable reason format for disconnection, either as a free text field or a predefined list.

- D. The System SHALL preserve all **Participant** data and history upon disconnection.
- E. When a **Participant** is disconnected, the System SHALL stop synchronizing data from the **Participant's Mobile Application** to the **Sponsor Portal**.
- F. When a **Participant** is disconnected, sponsor-specific rules SHALL remain applied to the **Participant's Mobile Application**.

Rationale

Disconnection sits between linking (the **Participant** is in trial) and mark-as-not-participating (the **Participant** has left the trial). It encodes the case where the link is operationally broken — a device fault, a connectivity issue, a temporary withdrawal — but the **Participant** is still expected to resume. Preserving all data and history is the audit-trail requirement; the disconnection event must not erase the historical record. Continuing to apply sponsor-specific rules to the **Mobile Application** is the operational requirement; the **Participant** should still see locked records, validation rules, and questionnaire restrictions that apply to their trial role, because they may reconnect at any time and the **Mobile Application** must not silently revert to personal-use behavior in the interim. The reason format is sponsor-configurable because some sponsors require a controlled vocabulary for downstream analysis (CAL- overlay encodes one such list) while others want free-text capture.

End Participant Disconnection Workflow | **Hash:** c133a6be

DIARY-PRD-participant-reconnection: Participant Reconnection Workflow

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Reconnection restores a disconnected **Participant's** link to the **Sponsor Portal**, enabling them to resume trial participation. A new **Mobile Linking Code** is generated to ensure the previous invalidated code cannot be reused.

Assertions

- A. The System SHALL allow a Study Coordinator to reconnect a **Participant** with **Disconnected** status.
- B. The System SHALL require a reason before reconnection is applied.
- C. The reconnection reason SHALL be captured via a **Reason Dialog — Free Text** by default and is sponsor-configurable.
- D. When reconnection is confirmed, the System SHALL generate a new **Mobile Linking Code**.

E. When a **Participant** successfully enters the new **Mobile Linking Code**, the System SHALL restore the **Participant's** link to the **Sponsor Portal**.

F. When a **Participant's** link is restored, the System SHALL synchronize all data recorded on the **Participant's Mobile Application** during the disconnected period to the **Sponsor Portal**.

Rationale

Reconnection re-uses the link-code mechanism deliberately: the same unguessable, single-use, time-bounded code that establishes a first link is also the credential that re-establishes one. Generating a fresh code (rather than re-issuing the original) closes any window in which a leaked or remembered code from the prior link could be exploited; the code that was originally used has long since been invalidated by the single-use rule, and the new code carries its own independent expiry. Synchronizing the disconnected-period data on link restoration is the principal value of the disconnect/reconnect distinction over mark-as-not-participating: the **Participant** continued recording diary entries locally while disconnected, and reconnection promotes that buffered data to the portal so the trial's clinical record is complete. The free-text reason default with sponsor override matches the pattern used by disconnection.

End Participant Reconnection Workflow | **Hash:** b7eec931

DIARY-PRD-participant-mark-not-participating: Mark as Not Participating

Level: prd | **Status:** Legacy | **Implements:** -

Overview

A Study Coordinator may mark a **Participant** as not participating when they complete or withdraw from the trial. This is distinct from disconnection — sponsor-specific rules are no longer applied to the **Participant's Mobile Application** and the **Participant** is not expected to resume participation unless explicitly reactivated.

Assertions

A. The System SHALL allow a Study Coordinator to mark a **Participant** with **Disconnected** status as **Not Participating**.

B. The System SHALL require a reason before the action is applied.

C. The System SHALL support sponsor-configurable reason format for marking as not participating, either as a free text field or a predefined list.

D. When the action is confirmed, the System SHALL stop applying sponsor-specific rules to the **Participant's Mobile Application**.

Rationale

Mark-as-not-participating is the off-trial state: the **Participant** has either completed the protocol or withdrawn, and the platform no longer needs to enforce trial-specific rules against their device. Routing the action through **Disconnected** as a precondition keeps the lifecycle linear (Linked $\square$ Disconnected $\square$ Not Participating) and prevents the case where a still-actively-linked device suddenly loses its trial constraints without the Coordinator first severing the link. Removing sponsor-specific rules from the **Mobile Application** lets a former **Participant** retain personal use of the app post-trial without remaining subject to record-locking, validation thresholds, or reminder schedules that were specific to their trial role. Reason capture mirrors disconnection's pattern, with the controlled-vocabulary case being more common for end-of-trial reporting (CAL-overlay encodes the predefined list this deployment uses).

End Mark as Not Participating | **Hash:** fb1a6056

DIARY-PRD-participant-reactivate: Reactivate Participant

Level: prd | **Status:** Legacy | **Implements:** -

Overview

A Study Coordinator may reactivate a **Participant** who was incorrectly marked as not participating or who has re-enrolled in the trial. Reactivation restores sponsor-specific rules to the **Participant's Mobile Application** and makes the standard reconnection workflow available.

Assertions

- A. The System SHALL allow a Study Coordinator to reactivate a **Participant** with **Not Participating** status.
- B. The System SHALL require a free text reason before the action is applied.
- C. When the action is confirmed, the System SHALL re-apply sponsor-specific rules to the **Participant's Mobile Application**.

Rationale

Reactivation is the inverse of mark-as-not-participating: the **Participant** is returning to active trial status, and the platform must restore the trial-specific rules that were lifted at off-trial time. Routing reactivation back through the standard reconnection workflow (rather than skipping straight to **Trial Active**) means the device-side link must be re-established through the same link-code mechanism used initially; this is both an audit affordance and a defense against reactivating a **Participant** whose

device is no longer in their possession. The free-text reason captures the rationale for what is operationally an unusual event (Mark-as-Not-Participating is intended to be terminal); requiring the reason here gives the audit trail the context reviewers need to interpret the reactivation.

End Reactivate Participant | **Hash:** 50b09295

DIARY-GUI-participant-dashboard: Participant Dashboard

Level: GUI | **Status:** Legacy | **Implements:** -

Overview

The **Participant Dashboard** is the primary surface for clinical staff to manage participant linking, trial status, and questionnaires. It consists of a participant list organised by tabs, a search input, and a **Participant Actions Modal** accessible from each participant row.

Definitions

Participant Actions Modal: A modal dialog displayed when a Study Coordinator selects a participant row, presenting the **Participant** ID and the secondary actions available for that participant's current status.

Assertions

Tab Organization

- A. The System SHALL organise participants into tabs based on their current **Participant** status.
- B. Each tab SHALL display a count of participants matching its status grouping.

Search

- C. The System SHALL provide a search input that filters participants by **Participant** ID.
- D. The search SHALL apply across all tabs regardless of the currently selected tab.
- E. The search SHALL support partial matching.
- F. The search SHALL update results in real time as the user types.
- G. When the search is active, each tab count SHALL update to reflect matching participants within that tab's status grouping.
- H. When no participants match the search, the **System** SHALL display the message "No results found."

Participant List

- I. The interface SHALL display the **Participant** ID, **Site**, current status as a **Status Badge**, and a primary action in the Action column for each participant row.
- J. The interface SHALL display the currently active tab with a visual indicator.
- K. When a **Participant**'s status changes, the interface SHALL move the participant row to the appropriate tab immediately.

Participant Actions Modal

- L. The interface SHALL display the **Participant Actions Modal** when a Study Coordinator selects a participant row.
- M. The **Participant Actions Modal** SHALL display the **Participant** ID.
- N. The interface SHALL present the actions available in the **Participant Actions Modal** according to the action availability table.

Action Availability

- O. The interface SHALL display the primary action in the Action column and the secondary actions in the **Participant Actions Modal** according to the action availability table defined in CAL-GUI-participant-dashboard-configuration.

Rationale

The **Participant Dashboard** is the Study Coordinator's daily working surface, and its design choices favor speed and accuracy over visual variety. Tab-by-status organisation keeps the candidate set for any given action small and self-consistent — a Coordinator looking to disconnect a **Participant** finds them under the Active tab, never mixed with **Not Connected** rows. Cross-tab partial-match search supports the recurring "find this specific **Participant** ID" workflow without forcing the Coordinator to first guess which tab they live under. Tab counts that update with the search reflect the same scoping rule, so the Coordinator can see at a glance whether the search matched within each status grouping. Row selection opens the modal rather than navigating away because the dashboard context (other **Participants**, current tab, search state) must remain visible behind the modal for the common workflow of consecutive lookups.

End Participant Dashboard | **Hash:** e3a5efed

1.13 DIARY-PRD-platform-operations-monitoring: Platform Operations and Monitoring

Level: PRD | **Status:** Legacy | **Refines:** -

Rationale

Clinical trial platforms require high availability and rapid incident response to ensure uninterrupted access to critical study data and participant interfaces. Monitoring systems enable proactive detection of performance degradation, security threats, and system anomalies before they impact clinical operations. FDA 21 CFR Part 11 compliance mandates documented operational procedures, incident tracking, and audit log oversight to maintain the integrity and reliability of electronic records used in regulatory submissions. Service level agreements (SLAs) for uptime provide measurable commitments to sponsors and study teams, while incident management processes ensure timely resolution and appropriate escalation of issues that could compromise data integrity or participant safety.

TODO (URS-Phase-3 reconciliation): Original Refines target REQ-p00044 (Clinical Trial Compliant Diary Platform) is in URS-replaced prd-system.md. Re-parent to the URS-derived top-level platform REQ when that lands.

Assertions

- A. The platform SHALL provide real-time system health monitoring.
- B. The platform SHALL collect and track performance metrics.
- C. The platform SHALL provide automated alerting for system events.
- D. The platform SHALL detect security events.
- E. The platform SHALL provide incident management capabilities.
- F. The platform SHALL provide incident escalation capabilities.
- G. The platform SHALL monitor system uptime.
- H. The platform SHALL track uptime against defined SLAs.
- I. The platform SHALL monitor audit logs for compliance purposes.
- J. System health dashboards SHALL be accessible to the operations team.
- K. The platform SHALL generate automated alerts for performance degradation.
- L. The platform SHALL detect security incidents and escalate them within defined timeframes.
- M. Incident response procedures SHALL be documented.
- N. Incident response procedures SHALL be tested.

O. Uptime SLA metrics SHALL be tracked and reported.

P. The platform SHALL flag audit log anomalies for review.

End Platform Operations and Monitoring | **Hash:** af349286

1.14 Sponsor Portal Authentication

This file groups the platform-side authentication requirements for the **Sponsor Portal**: password composition and reuse rules, two-factor authentication, forgot-password recovery (PRD plus the workflow GUI), and session management. Sponsor-specific configuration values (the chosen second-factor method, code expiry, login rate-limit thresholds) live in the corresponding `hht_diary_callisto` file (`spec/prd-portal-auth.md`).

DIARY-PRD-password-requirements: Password Requirements

Level: prd | **Status:** Legacy | **Implements:** -

Definitions

Password: A secret string of characters used to authenticate a user's identity when accessing the **System**.

Password Reuse Limit: The configurable number of the user's most recent **Passwords** that a new **Password** must not match.

Assertions

- A. The **System** SHALL require a **Password** to be a minimum of 12 characters in length.
- B. The **System** SHALL require a **Password** to contain at least one uppercase letter, one lowercase letter, one numeric character, and one special character.
- C. The **System** SHALL reject a **Password** that is commonly used or easily guessable as per NIST SP 800-63B commonly used password list.
- D. The **System** SHALL require a **Password** to be changed after 90 days, unless a different interval has been configured for the deployment.
- E. The **System** SHALL prevent a user from accessing the **System** until their **Password** has been changed upon expiry.
- F. The **System** SHALL reject a new **Password** that matches any of the user's previous **Password Reuse Limit Passwords**, where **Password Reuse Limit** is configurable per deployment.

Rationale

The composition rules (length, character classes) and the common-password rejection are baseline defenses against credential-guessing attacks: each rule independently raises the cost of a successful brute-force or dictionary attempt, and together they

place the minimum acceptable password well above the threshold at which automated attacks succeed against unprotected accounts. The NIST SP 800-63B reference is the authoritative source for the common-password list and is named explicitly so the deployment can update the list as NIST updates its guidance. The 90-day expiry is a sponsor-overridable default; the override exists because some sponsor deployments operate under regulatory regimes that mandate a different interval. The reuse-limit rule prevents the common operator behavior of cycling between two passwords (defeats the purpose of expiry); making the limit configurable lets a deployment choose how aggressive its reuse defense should be.

End Password Requirements | **Hash:** 61dd6125

DIARY-PRD-two-factor-authentication: Two-Factor Authentication

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Two-Factor Authentication adds a second independent factor beyond the **Password** to prevent unauthorized access in the event of credential compromise. The platform supports configurable second factors so each sponsor deployment can select the method that best fits its operational and regulatory context.

Definitions

Second Factor: The independent authentication factor required in addition to the **Password** during login. The specific method is sponsor-configurable per deployment.

Verification Code: A single-use, time-limited code presented as the **Second Factor** during login.

Code Expiry: The configurable duration after which an unused **Verification Code** becomes invalid.

Assertions

Enforcement

A. The **System** SHALL require Two-Factor Authentication for every login to the **Sponsor Portal**.

B. The **System** SHALL NOT grant access to the **Sponsor Portal** until both the **Password** and a valid **Verification Code** have been successfully validated.

Verification Code Lifecycle

C. When a **User Account** owner submits a valid **Password**, the **System** SHALL generate a **Verification Code** and deliver it via the configured **Second Factor** method.

D. The **System** SHALL ensure each **Verification Code** is single-use and is invalidated immediately upon successful use.

E. When a **Verification Code** has not been used within the **Code Expiry** duration, the **System** SHALL invalidate the **Verification Code**.

F. When a **Verification Code** has been invalidated, the **System** SHALL require the **User Account** owner to restart the login process to receive a new **Verification Code**.

Configuration

G. The **System** SHALL support sponsor-configurable selection of the **Second Factor** method per deployment.

H. The **System** SHALL support sponsor-configurable **Code Expiry** per deployment.

Rationale

The single password is no longer a sufficient credential for clinical trial portal access: credential leakage from unrelated services, phishing, and shared-workstation compromise are all common, and any of them can hand an attacker a working password without the user noticing. The second factor breaks that single-credential failure mode by requiring possession of an independent channel (email, authenticator app, or SMS, depending on sponsor configuration) before access is granted. The **Verification Code** is single-use and time-limited because a code that survived either property would inherit the same replay vulnerability the second factor exists to prevent. Sponsor-configurability of the method and expiry duration recognises that the operational tradeoffs vary by deployment: a sponsor with strict email infrastructure may choose a longer expiry; a sponsor with authenticator-app adoption may choose a much shorter one.

End Two-Factor Authentication | **Hash:** 71195321

DIARY-PRD-password-forgot: Forgot Password

Level: prd | **Status:** Legacy | **Implements:** -

Overview

A **User Account** owner who has forgotten their **Password** must be able to reset it without Administrator intervention. The reset mechanism uses a time-limited, single-use **Verification Link** delivered to the registered **Email Address** to confirm the requester controls the account.

Assertions

Initiation

- A. The **System** SHALL allow a **User Account** owner to initiate a **Password** reset from the **Sponsor Portal** login interface by providing their **Email Address**.
- B. When a **Password** reset is initiated, the **System** SHALL generate a **Verification Link** and deliver it to the registered **Email Address**.
- C. The **System** SHALL display a confirmation that an email has been sent regardless of whether the **Email Address** matches an existing **User Account**.

Verification Link Behavior

- D. When a **Verification Link** issued for **Password** reset is used or 24 hours have passed from generation, whichever occurs first, the **System** SHALL invalidate the **Verification Link**.
- E. When a new **Password** reset **Verification Link** is issued for a **User Account**, the **System** SHALL invalidate any previously issued **Password** reset **Verification Link** for that **User Account**.

Reset Completion

- F. When a **User Account** owner accesses a valid **Verification Link**, the **System** SHALL allow them to set a new **Password**.
- G. Upon successful **Password** reset, the **System** SHALL terminate all active sessions associated with that **User Account**.
- H. The **System** SHALL require Two-Factor Authentication on the next login following a successful **Password** reset.

Rejection Behavior

- I. When a **User Account** owner attempts to access an invalidated **Verification Link**, the **System** SHALL reject the attempt and prompt the user to initiate a new **Password** reset.

Rationale

Self-service password reset is a usability requirement (users who lose their **Password** must not be blocked from the system pending an Administrator escalation) and a security requirement (the reset mechanism must not become an attack channel). The verification-link mechanism mirrors the activation workflow: a single-use, time-bounded URL delivered to the registered email address is the credential that proves control of the account. The display-confirmation-regardless rule prevents user enumeration via the reset form — an attacker submitting addresses learns nothing about which ones exist in the system. Terminating all active sessions on successful reset and requiring 2FA on the next login closes the window in which a stolen session or stolen 2FA token from the pre-reset interval could still operate against the now-changed account. The shorter expiry (24 hours, versus 14 days for activation) reflects the higher attack value of a password-reset link compared to an activation link.

End Forgot Password | **Hash:** 4ad0d3fa

DIARY-GUI-password-forgot-workflow: Forgot Password Workflow Interface

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-password-forgot

Overview

The Forgot Password workflow spans four screens: the request screen where the **User Account** owner enters their **Email Address**, the confirmation screen shown after submission, the reset screen reached through a valid **Verification Link**, and the invalid link screen reached through an invalidated **Verification Link**. Consistent screen behavior across the flow ensures the **User Account** owner can complete recovery or recognize when to start over.

Assertions

Entry Point

A. The **Sponsor Portal** login interface SHALL present a Forgot Password action that navigates to the Forgot Password Request screen.

Forgot Password Request Screen

B. The Forgot Password Request screen SHALL present a single **Email Address** input field and a Submit action.

C. The Forgot Password Request screen SHALL present a Back to Login action that returns to the login interface.

D. The interface SHALL not enable the Submit action until a value has been entered in the **Email Address** field.

E. The interface SHALL validate that the entered value conforms to a valid email address format before submission is accepted.

F. When the Submit action is invoked, the interface SHALL navigate to the Confirmation screen.

Confirmation Screen

G. The Confirmation screen SHALL display a message indicating that an email has been sent if the submitted **Email Address** is associated with an account.

H. The Confirmation screen SHALL display the duration after which the **Verification Link** expires.

I. The Confirmation screen SHALL display guidance to check the spam folder if the email is not received.

J. The Confirmation screen SHALL present a Back to Login action that returns to the login interface.

K. The Confirmation screen SHALL display the same content regardless of whether the

submitted **Email Address** matches an existing **User Account**.

Reset Screen

L. The Reset screen SHALL be reachable only by accessing a valid **Verification Link** delivered to the **User Account** owner's registered **Email Address**.

M. The Reset screen SHALL present a New **Password** field and a Confirm **Password** field.

N. Each **Password** field SHALL present a show/hide toggle that controls whether the entered value is displayed in plain text or masked.

O. The Reset screen SHALL present a Submit action.

P. The interface SHALL not enable the Submit action until both **Password** fields have been populated and contain matching values.

Q. When the Submit action is invoked and the **Password** is rejected for failing composition or reuse rules, the interface SHALL display an inline message identifying which rule was violated and SHALL not navigate away from the Reset screen.

R. When the Submit action is invoked and the **Password** is accepted, the interface SHALL navigate to the **Sponsor Portal** login interface and display a confirmation that the **Password** has been changed.

Invalid Link Screen

S. When a **User Account** owner accesses an invalidated **Verification Link**, the interface SHALL display the Invalid Link screen.

T. The Invalid Link screen SHALL display a message indicating that the **Verification Link** is no longer valid.

U. The Invalid Link screen SHALL present an action that navigates to the Forgot Password Request screen.

Rationale

The four-screen structure mirrors the four states the workflow can reach: requesting reset, awaiting email, completing reset, recovering from an invalidated link. Identical confirmation content regardless of whether the email matches is the GUI-level enforcement of the user-enumeration resistance established at the PRD level — divergent UI between matching and non-matching cases would leak the same information the PRD assertion is trying to hide. The inline composition-rule error on Submit is necessary because users who submitted an invalid **Password** must learn which rule failed without losing the typed values, otherwise the show/hide toggle gains a punitive UX where users repeatedly retype the same long password. The Invalid Link screen's link back to the request screen restarts the recovery loop in a single click, recognising that the most common reason for an invalidated link is the 24-hour expiry catching a user who opened the email belatedly.

End Forgot Password Workflow Interface | **Hash:** b845f181

DIARY-PRD-session-management: Session Management

Level: prd | **Status:** Legacy | **Implements:** -

Overview

The **Sponsor Portal** terminates inactive sessions to limit the window during which an unattended authenticated session could be exploited. Session limits also ensure that role and permission changes take effect within a bounded time.

Definitions

Session: An authenticated period during which a **User Account** owner can access the **Sponsor Portal** without re-entering credentials.

Session Idle Timeout: The configurable maximum duration of inactivity allowed during a **Session** before the **System** terminates it.

Assertions

Session Establishment

A. The **System** SHALL establish a **Session** when a **User Account** owner successfully completes Two-Factor Authentication.

Idle Timeout

B. The **System** SHALL track elapsed inactivity from the **User Account** owner's most recent interaction with the **Sponsor Portal**.

C. When the **Session Idle Timeout** is exceeded, the **System** SHALL terminate the **Session** and require re-authentication.

Termination

D. The **System** SHALL allow a **User Account** owner to explicitly terminate their **Session** by logging out.

E. When a **User Account** is deactivated, the **System** SHALL terminate all active **Sessions** associated with that **User Account** immediately.

F. When a **User Account's Role** or **Site** assignment is changed, the **System** SHALL terminate all active **Sessions** associated with that **User Account** immediately.

Configuration

G. The **System** SHALL support sponsor-configurable **Session Idle Timeout** per deployment, with a default of 10 minutes.

Rationale

A **Session** in the **Sponsor Portal** is a high-value authentication artifact — it represents a successful two-factor login and confers access to clinical data and User Account management capabilities for its duration. The idle timeout caps the window in which an unattended workstation could be exploited; tracking inactivity from the user's most recent interaction (rather than from session creation) is the standard pattern that balances security against operational disruption. The cascade rules (deactivation, role change, Site change immediately terminate sessions) ensure that authorization changes take effect synchronously rather than waiting for the next login: a Coordinator who has lost their role for cause cannot continue acting under the old role until their **Session** happens to time out. Sponsor-configurability of the timeout duration acknowledges that the right tradeoff between security and operator disruption varies by deployment; the 10-minute default reflects clinical-portal industry baseline.

End Session Management | **Hash:** 10331e8d

1.15 Questionnaire Management

This file groups the platform-side requirements that establish the **Questionnaire** system: the clinical questionnaire foundation, sponsor-specific configuration surfaces, the controlled submission gate that enforces approved structures, and the change-control discipline that keeps questionnaires stable within a trial. Sponsor-specific workflows (Start Trial, NOSE HHT and HHT-QoL administration, cycle tracking, session timing) live in the corresponding `hht_diary_callisto` file (`spec/prd-questionnaire-management.md`) and reference the foundation defined here.

DIARY-PRD-questionnaire-system: Clinical Questionnaire System

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Clinical trials use structured questionnaires to collect reliable data in a consistent and compliant way. The questionnaires are carefully built to ensure accuracy and a good participant experience. New questionnaires can be added as needed, and the content always comes from approved documents, not edited directly in the app. This requirement is the foundation that all other questionnaire-handling requirements refine.

Definitions

Questionnaire: A validated data collection instrument presented to a participant to capture self-reported clinical outcomes at protocol-defined points in a clinical trial.

Questionnaire Type: A named category of **Questionnaire**. Each **Questionnaire Type** is implemented as an individual coded component.

Trial Start: The event that formally begins a participant's active trial participation. The trigger and label for **Trial Start** are sponsor-configurable.

Completion Status: The current state of a **Questionnaire** instance for a given participant. Valid values are defined per **Questionnaire** workflow.

Diary Data Synchronization: The continuous transmission of participant diary entries from the participant's mobile application to the **Sponsor Portal** and **Rave EDC**.

Questionnaire Display Name: The participant-facing name of a **Questionnaire** as presented in the mobile application. Distinct from the instrument name used for internal identification and clinical reference.

Trial: A clinical study conducted under a Sponsor's protocol in which **Participants** are enrolled to capture self-reported clinical outcomes through the **Mobile Application**. The **Trial** begins at **Trial Start**.

Assertions

Questionnaire Registry

- A. The System SHALL allow implementation of multiple **Questionnaire Types**, each as an individual coded component.
- B. The System SHALL track **Completion Status** for each **Questionnaire** instance.

Data Synchronization

- C. The System SHALL activate **Diary Data Synchronization** upon **Trial Start**.
- D. The System SHALL deactivate **Diary Data Synchronization** when a participant is disconnected or is marked as Not Participating.

Configuration

- E. The System SHALL support sponsor-configurable configuration of the trigger and workflow that activates **Trial Start**.
- F. Each **Questionnaire** SHALL have a **Questionnaire Display Name**.

Rationale

The questionnaire subsystem is the principal source of trial-grade clinical data in the platform, and its design choices reflect that role. Implementing each **Questionnaire Type** as an individual coded component is a defense against ad hoc clinical content: every questionnaire that ships is a deliberate engineering artifact reviewed against an approved source document, not a configuration table that a sponsor could edit without change control. **Completion Status** tracking is the foundation for the workflow rules that govern how a questionnaire moves from sent to answered to finalized; without per-instance status the platform could not distinguish “answered but not finalized” from “answered and locked”, which is a clinical and regulatory distinction. **Trial Start** is the gate at which **Diary Data Synchronization** activates because pre-trial diary data is participant-private and must not be promoted to the sponsor portal or Rave EDC until the sponsor has acknowledged that the participant is in trial; making the trigger sponsor-configurable keeps the platform protocol-neutral.

End Clinical Questionnaire System | **Hash:** 306eeb1a

DIARY-PRD-questionnaire-sponsor-configuration: Sponsor Questionnaire Configuration

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-questionnaire-system

Overview

A configuration-driven approach allows the sponsor to tailor questionnaire behavior to their study protocol without requiring platform code changes, ensuring that mid-study modifications are managed deliberately and do not unintentionally affect data collection.

Assertions

- A. The System SHALL support sponsor-specific configuration of which **Questionnaires** are enabled per deployment.
- B. The System SHALL support sponsor-specific configuration of participant reminder behavior for each enabled **Questionnaire**.
- C. The System SHALL support sponsor-specific configuration of the processing steps that occur between participant submission and **Rave EDC** synchronization for each **Questionnaire**.
- D. The System SHALL support sponsor-specific configuration of whether a participant may edit their answers after submission and under what conditions.
- E. The System SHALL support sponsor-specific configuration of which languages are enabled per **Questionnaire**.

Rationale

The five configuration axes named here — enablement, reminder behavior, post-submission processing, post-submission editability, and language — are the dimensions along which clinical protocols typically vary between sponsors using the same underlying **Questionnaire Type**. Encoding them as sponsor configuration rather than platform code keeps the platform's questionnaire library stable across deployments while letting each sponsor express the protocol-specific decisions that their IRB and operations teams require. The configuration approach is bounded — sponsors choose from the offered axes, they do not edit questionnaire content — which preserves the change-control discipline established in the foundation requirement.

End Sponsor Questionnaire Configuration | **Hash:** 1c0e5f61

DIARY-PRD-questionnaire-submission-control: Clinical Data Submission Control

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-questionnaire-system

Overview

Clinical data integrity requires that the system only accepts data conforming to approved, predefined structures. A controlled registry of approved data collection instruments ensures that unrecognized or malformed data is rejected at the point of entry, supporting audit trail integrity and validation requirements under 21 CFR Part 11.

Assertions

- A. The System SHALL maintain a controlled list of approved **Questionnaire** definitions.
- B. The System SHALL reject any data submission that does not conform to an approved **Questionnaire** definition.

Rationale

A clinical trial's evidence chain begins at the data-ingest boundary: every submission that reaches the sponsor portal or **Rave EDC** must be traceable to an approved **Questionnaire** definition, because data that does not conform to a known instrument has no defensible interpretation and corrupts the trial's analyzability. The controlled registry is the platform's enforcement point — submissions are matched against it at the point of entry and rejected if they do not conform, so malformed or unrecognized data never enters the data store. This is a structural prerequisite for 21 CFR Part 11 audit trail integrity: every data point in the audit trail must reference a definition that existed at the time the data was submitted.

End Clinical Data Submission Control | **Hash:** 62ccbb18

DIARY-PRD-questionnaire-change-control: Questionnaire Change Control

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-questionnaire-system

Overview

Questionnaires must stay the same throughout an active trial so the data remain consistent and traceable. Any updates must follow a formal change process to protect the integrity of the system.

Assertions

- A. The System SHALL use predefined **Questionnaires** for all clinical data collection.
- B. The System SHALL NOT allow **Questionnaires** to be created, modified, or deleted during trial operation. If any new **Questionnaire** is required, a new **Questionnaire**

Type will be implemented. Modifications for an existing **Questionnaire** will follow the process outlined in the below two assertions.

C. Changes to **Questionnaires** SHALL be subject to formal change control procedures before deployment.

D. The System SHALL maintain backward compatibility to allow viewing of data collected with previous **Questionnaire** versions.

Rationale

Mid-trial **Questionnaire** changes would silently alter the meaning of the data being collected: a question reworded between Day 30 and Day 60 of a participant's enrollment produces two non-comparable answer streams under the same instrument name. The change-control discipline encoded here prevents that class of error by treating every **Questionnaire** modification as a deliberate engineering act subject to review and approval. The new-Questionnaire-Type rule (rather than in-place edit) keeps the data semantics for any given **Questionnaire Type** stable for the lifetime of any trial that uses it. Backward-compatible viewing is essential because trials regularly span multiple **Questionnaire** versions when long-running studies adopt updated instruments part-way through; reviewers must be able to inspect historical data captured under the previous version without that data being silently re-interpreted under the new version's rules.

End Questionnaire Change Control | **Hash:** e1e119e4

1.16 Questionnaires Overview

This file enumerates the three clinical questionnaires the platform implements: the **Daily Epistaxis Record** (the daily eDiary instrument), the **NOSE HHT** survey, and the **HHT-QoL** survey. Each requirement binds the instrument to its validated source document and to the computer-readable data file that is the single implementation reference. The participant-facing workflow over these instruments (Preamble, Question Screen, Review Screen, Submission, Finalization) is defined in `prd-questionnaire-participant-w`; the daily-eDiary recording flow is defined in `prd-epistaxis.md`; the scoring algorithms are defined in `prd-score-calculation.md`.

DIARY-PRD-questionnaire-daily-epistaxis: Daily Epistaxis Record Questionnaire

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Daily epistaxis tracking is the core data collection activity in HHT clinical trials because nosebleed frequency, duration, and severity are the primary outcome measures for evaluating treatment efficacy. participant self-reporting captures events as they occur rather than relying on retrospective recall during clinical visits, supporting the Contemporaneous principle of ALCOA+. "No nosebleeds" and "Don't remember" entries are valid daily summaries that distinguish between confirmed absence of events and missing data, which is critical for data integrity in regulatory submissions.

Definitions

Daily Epistaxis Record: The primary nosebleed tracking instrument used in the HHT Clinical Diary, derived from Clark et al. (2018) with modifications.

Epistaxis Event: A single nosebleed occurrence recorded by the participant with timing, duration, and intensity data.

Assertions

- A. The system SHALL implement the **Daily Epistaxis Record** questionnaire for capturing participant-reported nosebleed events.
- B. The source instrument SHALL be documented with citation: Clark et al. "Nosebleeds in hereditary hemorrhagic telangiectasia: Development of a participant-completed daily eDiary." *Laryngoscope Investig Otolaryngol*. 2018. doi:10.1002/lio2.211.
- C. All modifications to the source instrument SHALL be documented and traceable to their justification.

D. The instrument content, field definitions, and validation rules SHALL be implemented in a computer-readable data file that has been manually checked for accuracy.

Rationale

The **Daily Epistaxis Record** is the primary instrument the diary platform exists to capture; every other surface (calendar, day view, recording flow, notifications, score calculations) is in service of getting reliable instances of this record into the clinical dataset. Binding the requirement to the Clark et al. (2018) source instrument anchors the diary content to a validated reference so the platform's implementation can be audited against an external definition rather than re-derived locally. Any modification to the source is required to carry its own documented justification because clinical instruments derive their interpretive validity from the published version; undocumented deviations would break the chain back to the validated source and undermine the dataset's regulatory standing. The single computer-readable data file is the implementation reference — the same artifact the application loads and the specification references — which prevents drift between what the spec says and what the application runs.

End Daily Epistaxis Record Questionnaire | **Hash:** 153ee0cf

DIARY-PRD-questionnaire-nose-hht: NOSE HHT Questionnaire

Level: prd | **Status:** Legacy | **Implements:** -

Overview

The NOSE HHT (Nasal Outcome Score for Epistaxis in Hereditary Hemorrhagic Telangiectasia) is a validated 29-question participant-reported outcome measure assessing the physical, functional, and emotional impact of nosebleeds on HHT participants. The platform implements this instrument faithfully from the validated source. Questionnaire content is defined in the source document and transcribed once into a computer-readable data file — the data file is the single implementation reference, eliminating duplication and ensuring the application and the specification never diverge.

Definitions

NOSE HHT: The Nasal Outcome Score for Epistaxis in Hereditary Hemorrhagic Telangiectasia, a validated clinical instrument published in JAMA Otolaryngology — Head and Neck Surgery, 2020.

True Copy: A complete, unaltered reproduction of the source document obtained directly from the original publisher URL.

Assertions

A. The system SHALL implement the **NOSE HHT** questionnaire as defined in the source document referenced in assertion B.

B. A **True Copy** of the source document SHALL have been obtained from: <https://jamanetwork.com>.

C. The **True Copy** SHALL be referenced by the computer-readable data file that implements this questionnaire.

D. The source document SHALL be transcribed into a computer-readable data file that has been manually checked for accuracy. This transcribed version will serve as the official reference for all questions, answer choices, and scoring rules used in the system.

E. The System SHALL present the 29 **NOSE HHT** questions organized into the three categories defined in the source instrument: Physical, Functional, and Emotional.

F. The Questionnaire Display Name for the NOSE HHT Questionnaire SHALL be "NOSE HHT Survey".

Rationale

The **NOSE HHT** is a validated clinical instrument; its interpretive validity depends on faithful reproduction of the published questions, answer choices, and scoring rules. The True Copy obligation anchors the platform's implementation to the publisher's authoritative version — the same document any auditor can independently retrieve from the cited URL — and removes any ambiguity about which variant of the instrument the platform uses. Transcription into a computer-readable data file is the bridge between the human-readable source and the running application: the application loads the data file directly, so the spec's reference to the data file and the application's runtime behavior are guaranteed to match. The three-category presentation (Physical, Functional, Emotional) matches the source instrument's structure and is required for participant comprehension and downstream score interpretation.

End NOSE HHT Questionnaire | **Hash:** 3aaee714

DIARY-PRD-questionnaire-hht-qol: HHT-QoL Questionnaire

Level: prd | **Status:** Legacy | **Implements:** -

Overview

The HHT-QoL questionnaire is a validated 4-question participant-reported outcome measure assessing how nosebleeds and HHT-related problems affect participants' daily activities and social engagement. The platform implements this instrument faithfully from the validated source. Questionnaire content is defined in the source document and transcribed once into a computer-readable data file — the data file is the single

implementation reference, eliminating duplication and ensuring the application and the specification never diverge.

Definitions

HHT-QoL: A validated clinical instrument published in Blood Advances, 2022, measuring the impact of HHT symptoms on participants' daily and social activities.

Assertions

A. The system SHALL implement the **HHT-QoL** questionnaire as defined in the source document referenced in assertion B.

B. A **True Copy** of the source document SHALL have been obtained from: <https://ashpublications.org>

C. The **True Copy** SHALL be referenced by the computer-readable data file that implements this questionnaire.

D. The source document SHALL be transcribed into a computer-readable data file that has been manually checked for accuracy. This transcribed version will serve as the official reference for all questions, answer choices, and scoring rules used in the system.

E. The system SHALL emphasize the key phrases — interrupted, avoided, had to miss, and other than nosebleeds — in the **HHT-QoL** questions, matching the formatting of the original validated instrument.

F. The Questionnaire Display Name for the HHT-QoL Questionnaire SHALL be "HHT Quality of Life Survey".

Rationale

The **HHT-QoL** is a short (4-question) validated quality-of-life instrument complementing the NOSE HHT's symptom focus with an activities-and-engagement perspective. The True Copy and transcription requirements track the same faithfulness obligation that applies to the NOSE HHT (assertion-for-assertion): the published source defines the instrument, and the platform's data file is the single reference the application consumes. The key-phrases emphasis (italics on "interrupted", "avoided", "had to miss", "other than nosebleeds") is a substantive part of the published instrument's wording — these emphases are how the original instrument signals which words the participant should be asked to weigh most carefully — and dropping them would silently re-engineer the question stems away from the validated form. Preserving them keeps the platform's rendering aligned with the source.

End HHT-QoL Questionnaire | **Hash:** f9b343f7

1.17 Participant Questionnaire Workflow

This file defines the participant-facing workflow over **Portal-Sent Questionnaires** — the rules that govern how a participant moves through Preamble, questions, Review, and Submission, the corresponding interface behavior, and the Session Timeout / Session Expiry mechanism that bounds how long an in-progress questionnaire can be left idle. The instruments these workflows operate over are defined in `prd-questionnaire-overview.m`. The sponsor-side call-back behavior overlay (CAL-GUI-questionnaire-call-back-participant) lives in the `hht_diary_callisto` repo.

DIARY-PRD-questionnaire-portal-sent-rules: Portal-Sent Questionnaire Rules

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Portal-Sent Questionnaires are initiated by a **Study Coordinator** from the **Sponsor Portal** and delivered to the participant via push notification. The rules below define what the participant sees (Preamble), how they progress through the questionnaire, when their answers become a Submission, and the editing window between Submission and Finalization.

Definitions

Portal-Sent Questionnaire: A Questionnaire initiated by a Study Coordinator from the Sponsor Portal and delivered to the participant via push notification.

Preamble: The introductory content block presented to the participant before questionnaire questions begin informing the participant of the estimated completion time and that answers will not be saved until Submission.

Submission: The participant action that signals all questions are answered and the Questionnaire is ready for Study Coordinator review.

Finalization: The Study Coordinator action that permanently locks participant answers, triggers score calculation and pushes the data to Rave EDC.

Assertions

Preamble

A. The System SHALL present the **Preamble** to the participant each time the participant opens a **Portal-Sent Questionnaire**.

B. The **Preamble** SHALL inform the participant of the estimated time required to complete the questionnaire.

C. The **Preamble** SHALL inform the participant that their progress will be retained while completing the questionnaire and that answers are submitted only when they complete **Submission**.

D. The System SHALL require the participant to confirm readiness before proceeding from the **Preamble** to the questions.

E. When the participant indicates they are not ready, the System SHALL return the participant to the Main home screen.

Completion Rules

F. The System SHALL present one question at a time during **Portal-Sent Questionnaire** completion.

G. The System SHALL NOT permit the participant to skip any question in a **Portal-Sent Questionnaire**.

H. The System SHALL preserve in-progress answers locally while the participant is completing the questionnaire and SHALL NOT commit answers as a Submission until the participant completes **Submission**.

I. The System SHALL make completion progress available to the participant throughout the questionnaire.

J. The System SHALL allow the participant to navigate back and forth between the questions before **Submission**.

K. The System SHALL allow the participant to exit the questionnaire at any point before **Submission** without losing in-progress answers, subject to the **Session Timeout** defined in DIARY-PRD-questionnaire-session-timeout.

L. The System SHALL present a review of all answers to the participant before **Submission**, allowing the participant to modify any answer before proceeding.

Submission

M. The System SHALL require an explicit participant action - clicking the Submit button - to complete **Submission**.

Edit Rules

N. The System SHALL allow the participant to edit their answers with no time limit after **Submission** until **Finalization**.

O. The System SHALL NOT permit the participant to edit their answers after **Finalization**.

Rationale

The Preamble exists because a **Portal-Sent Questionnaire** is a non-trivial commitment of participant time (the NOSE HHT has 29 questions; an ad-hoc questionnaire can be longer); telling the participant up front how long it will take and that progress is preserved between sessions reduces the rate at which participants start and abandon mid-flow. One-question-at-a-time presentation matches the validated-instrument

format (the source documents present questions one at a time on paper) and prevents the participant from scanning ahead, which could bias later answers. Skipping is prohibited because the validated scoring algorithms require complete responses; partial questionnaires are not interpretable. The in-progress-preservation rule is the participant-side guarantee that “Exit” is safe — combined with the **Session Timeout** override (which can discard in-progress answers if the participant has been idle too long), it gives the participant flexible but bounded continuation. Editing is open between Submission and Finalization because Submission signals “participant is done”, but the **Study Coordinator** review may surface answer issues the participant should be able to correct without resubmitting from scratch; Finalization is the irreversible boundary because that is when the score is computed and the data ships to **Rave EDC**.

Screen reference

See: Questionnaire Preamble Screen

End Portal-Sent Questionnaire Rules | **Hash:** aff8012d

DIARY-GUI-questionnaire-portal-sent-workflow: Portal-Sent Questionnaire Workflow

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-questionnaire-portal-sent-rules

Overview

The interface for a **Portal-Sent Questionnaire** spans four screens: the **Preamble**, the per-question screen, the **Review Screen** (after the final question), and the post-Submission Acknowledgement Dialog. The behavior on each screen tracks the PRD-level rules above and adds the visual affordances (progress indicator, navigation controls, action labels) participants use to move through the flow.

Definitions

Review Screen: The screen presented to the participant after answering the final question, displaying all questions and selected answers before Submission.

Assertions

Preamble

A. The interface SHALL present the **Preamble** with two actions: an I’m Ready button and a Not Now button.

B. When the participant selects Not Now, the interface SHALL navigate the participant to the home screen.

C. When the participant selects I'm Ready, the interface SHALL navigate the participant to the first question.

Question Screen

D. The interface SHALL present a progress indicator showing the participant's current position within the questionnaire (Question #X out of Y).

E. The interface SHALL display the Questionnaire Display Name as the screen title on every question screen.

F. The interface SHALL present a back navigation control on every question screen.

G. The interface SHALL present a Next action on every question screen.

H. The Next action SHALL be disabled until the participant selects an answer.

I. When the participant selects an answer and selects Next, the interface SHALL advance to the next question.

J. When the participant navigates back to a previously answered question, the interface SHALL display the previously selected answer.

Review Screen

K. After the participant answers the final question, the interface SHALL present the **Review Screen**.

L. The **Review Screen** SHALL present all questions and the participant's selected answers.

M. The **Review Screen** SHALL allow the participant to navigate to any individual question to change their answer.

N. The **Review Screen** SHALL present a submit action.

Editing an Answer

O. When the participant navigates to an individual question from the **Review Screen** and changes an answer, the interface SHALL present a Save button.

P. When the participant selects Save, the interface SHALL return the participant to the **Review Screen**.

Submission

Q. When the participant confirms **Submission**, the interface SHALL display an Acknowledgement Dialog confirming the questionnaire has been submitted successfully.

Post-Submission Editing

R. When the participant re-opens a submitted questionnaire before **Finalization**, the interface SHALL present the **Review Screen**.

Finalization

S. When the participant opens a finalized questionnaire, the interface SHALL present

the questionnaire in a read-only state with no edit or submit actions available.

Rationale

The four-screen structure (Preamble, Question, Review, Acknowledgement) makes the questionnaire's logical structure visible to the participant: a clearly demarcated start (Preamble with I'm Ready / Not Now), per-question progress visibility (Question #X out of Y title), a final aggregated review before commit (Review Screen with per-question edit affordance), and an explicit confirmation that the submission landed (post-Submission Acknowledgement Dialog). The Next-disabled-until-answered rule combined with the prohibition on skipping is how the platform enforces the no-skipped-questions PRD assertion at the GUI layer. Returning to the **Review Screen** on re-open between Submission and Finalization keeps the participant's mental model aligned with the PRD-level edit window — they always re-enter through the aggregated review, not into a partial-flow ambiguity, so the next action is either "Submit again" or "I'm finished, close this". The read-only state after Finalization is the GUI-side counterpart of the PRD-level "no edit after Finalization" rule.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

Screen reference

See:

Questionnaire Review Screen

Questionnaire Preamble Screen

Post-Submission Acknowledgement Dialog

End Portal-Sent Questionnaire Workflow | **Hash:** 34a7c378

DIARY-PRD-questionnaire-session-timeout: Questionnaire Session Timeout

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-questionnaire-portal-sent-rules

Overview

A configurable session timeout allows each questionnaire to enforce appropriate completion constraints based on its length and clinical requirements.

Definitions

Session Timeout: The configurable maximum duration of inactivity allowed during questionnaire completion. When exceeded, the session expires and the answers selected so far are not retained.

Session Expiry: The state reached when a participant has not completed a questionnaire within the configured Session Timeout duration.

Timeout Warning Notification: Push notification delivered to the participant when the Session Timeout is approaching.

Session Expiry Notification: Push notification delivered to the participant when Session Expiry has occurred.

Assertions

Timeout Tracking

A. When a questionnaire is configured with a **Session Timeout**, the System SHALL track elapsed inactivity from the participant's most recent interaction with the questionnaire.

B. The System SHALL only advance the inactivity timer while the participant is not actively interacting with the questionnaire.

C. When the **Session Timeout** is exceeded before the participant submits the questionnaire, the System SHALL discard all answers selected so far.

Expiry Behavior

D. When a participant returns to a questionnaire in a state of **Session Expiry**, the System SHALL present the questionnaire from the beginning including the **Preamble**.

Notifications

E. When a questionnaire is configured with a **Session Timeout**, the System SHALL deliver a **Timeout Warning Notification** to the participant when the **Session Timeout** is approaching.

F. When a questionnaire is configured with a **Session Timeout**, the System SHALL deliver a **Session Expiry Notification** to the participant when **Session Expiry** has occurred.

State Preservation

G. When a questionnaire is not configured with a **Session Timeout**, the System SHALL preserve and restore the participant's in-progress answers on return with no timeout constraint.

H. When a questionnaire with a **Session Timeout** has not yet expired, the System SHALL return the participant to their in-progress questionnaire on return.

Configuration

- I. Each questionnaire definition SHALL support optional configuration of **Session Timeout** duration.
- J. Each questionnaire definition SHALL support configuration of the threshold before expiry at which the **Timeout Warning Notification** is delivered.

Rationale

Some clinical questionnaires (e.g. the NOSE HHT, the HHT-QoL) require contemporaneous answering — a participant's frame of reference shifts measurably if they pause for hours between questions, and the resulting answers no longer reflect a single moment of self-report. A configurable **Session Timeout** lets each questionnaire enforce a "complete this in one sitting" constraint without baking a single duration into the platform. Discarding in-progress answers on expiry rather than retaining them is what gives the timeout its clinical meaning: the next attempt starts fresh from the **Preamble**, with a participant frame of reference that the sponsor can interpret. The opt-out (no **Session Timeout** configured) covers the case where in-progress preservation is fine (e.g. an ad-hoc questionnaire with no contemporaneous requirement). The two notifications — Warning before expiry, Expiry on the event — give the participant a chance to return before answers are discarded (Warning) and clear feedback when answers have already been discarded (Expiry).

Screen reference

See:

Session Expiry Dialog

Timeout Warning Notification

Session Expiry Notification

End Questionnaire Session Timeout | **Hash:** 88870c17

DIARY-GUI-questionnaire-session-expiry: Questionnaire Session Expiry

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-questionnaire-session-timeout

Overview

The interface for **Session Expiry** consists of three surfaces: the **Timeout Warning Notification** (push notification before expiry), the **Session Expiry Dialog** (in-app dialog on return after expiry), and the **Session Expiry Notification** (push notification when expiry occurs). On return to an active (not-yet-expired) session, the interface restores the participant to where they left off.

Definitions

Session Expiry Dialog: The Acknowledgement Dialog displayed to the participant when they open a questionnaire that has reached Session Expiry.

Assertions

Timeout Warning

A. When a **Timeout Warning Notification** is delivered, the interface SHALL present it as a push notification indicating that the questionnaire session is approaching expiry.

Expired Session on Return

B. When the participant opens a questionnaire that has reached **Session Expiry**, the interface SHALL display a **Session Expiry Dialog** informing the participant that their session has expired and their previous answers were not saved.

C. The **Session Expiry Dialog** SHALL present a Start Again button and a Not Now button.

D. When the participant selects Start Again, the interface SHALL dismiss the **Session Expiry Dialog** and present the **Preamble**.

E. When the participant selects Not Now, the interface SHALL navigate the participant to the home screen.

Session Expiry Notification

F. When a **Session Expiry Notification** is delivered, the interface SHALL present it as a push notification indicating that the questionnaire session has expired.

Active Session on Return

G. When the participant returns to a questionnaire with a session that has not yet reached **Session Expiry**, the interface SHALL restore the participant to the question they were on when they left, with their in-progress answers intact.

Rationale

The **Session Expiry Dialog** is the in-app companion to the **Session Expiry Notification**: the notification tells the participant the session expired while they were away from the app; the dialog confirms it when they return and tells them their prior answers are gone. Start Again / Not Now are the two realistic next-actions — restart now from the Preamble, or defer until later — and both are made explicit so the participant does not face a silent reset on next open. Restoring the active-session participant to their last question (with in-progress answers intact) is the GUI-side guarantee that the PRD's in-progress-preservation rule actually surfaces to the participant; if the interface always re-entered through the Preamble, the preservation would be invisible and the participant would lose confidence that pausing is safe.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

Screen reference

See: Session Expiry Dialog

End Questionnaire Session Expiry | **Hash:** c596cbb7

1.18 Questionnaire Versioning, Localization, and Sponsor Eligibility

This file holds the clinical-domain obligations governing how the Diary Platform versions questionnaires (schema, content, GUI), localizes them across languages, and how each sponsor configures which questionnaires their study uses. These obligations were carved out of the legacy `prd-event-sourcing-system.md` during the URS-v1 migration on 2026-05-15: they are domain-specific to clinical questionnaires and did not belong alongside generic event-sourcing requirements (those are now owned by the `event_sourcing` repo as `EVS-PRD-*`).

The URS v1.0 §5.3 (Questionnaire Management) will refine and supersede portions of these obligations in a subsequent migration phase; until then this file is the active source.

DIARY-PRD-questionnaire-versioning: Questionnaire Versioning Model

Level: prd | **Status:** Legacy | **Implements:** -

Rationale

Clinical questionnaires evolve across multiple independent dimensions. Structural changes add or remove data fields requiring database schema updates. Content changes refine question wording to improve clarity or address translation issues without altering the underlying data structure. Presentation changes enhance user experience through visual redesign without modifying what questions are asked. Conflating these three evolution paths into a single version number creates unnecessary coupling: wording improvements would trigger schema migrations, UI redesigns would invalidate validated instrument versions, and the audit trail would obscure which dimension actually changed. By tracking schema, content, and GUI versions independently, clinical teams can refine instrument language without engineering involvement, UX teams can improve presentation without affecting clinical validation, and regulatory audits can reconstruct the exact patient experience across all three dimensions.

Once a version is deployed and used to capture patient data, it becomes an immutable artifact. Modifying a deployed version would silently alter the instrument for patients already using it, breaking the ability to reconstruct what the patient experienced (assertions M, N) and severing audit traceability (assertion S). Any change — whether bug fix or improvement — must produce a new version. The prior version remains frozen and available for sponsors that have not opted into the update. This immutability applies to all three dimensions independently: a GUI can be frozen at v1.0 for one sponsor while another adopts v1.1, without affecting the content or schema versions either sponsor uses.

Assertions

- A. The platform SHALL support independent versioning of questionnaire schema, content, and presentation.
- B. The system SHALL distinguish between schema version, content version, and GUI version as separate versioning dimensions.
- C. Schema version SHALL identify the data structure and field types stored in the database.
- D. Schema version SHALL change when fields are added, removed, or restructured.
- E. Schema version SHALL determine validation rules and migration requirements.
- F. Content version SHALL identify the source language question text, option labels, help text, and scoring rules.
- G. Content version SHALL change when wording is clarified or questions are refined, independent of schema changes.
- H. GUI version SHALL identify the presentation and rendering of the questionnaire in client applications.
- I. GUI version SHALL change when user interface is redesigned or user experience is improved, independent of content or schema.
- J. Each questionnaire response SHALL record the schema version identifier.
- K. Each questionnaire response SHALL record the content version identifier.
- L. Each questionnaire response SHALL record the GUI version identifier.
- M. The system SHALL enable complete reconstruction of what the patient saw using the recorded version identifiers.
- N. The system SHALL enable complete reconstruction of how the data was captured using the recorded version identifiers.
- O. Wording changes SHALL create a new content version without requiring schema migration.
- P. UI redesigns SHALL create a new GUI version without requiring content version changes.
- Q. UI redesigns SHALL create a new GUI version without requiring schema version changes.
- R. The system SHALL enable retrieval of historical responses with exact version context for all three version dimensions.
- S. The platform SHALL maintain complete audit traceability across all three versioning dimensions.
- T. Once a questionnaire version has been used to capture patient data, that version SHALL be immutable.

End Questionnaire Versioning Model | **Hash:** 6d08845d

DIARY-PRD-questionnaire-localization: Questionnaire Localization and Translation Tracking

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-questionnaire-versioning

Rationale

International clinical trials require validated translations of instruments, where each translation has its own validation status and version lifecycle independent of the source content. For ALCOA+ compliance, the audit trail must show exactly what question text the patient saw in their language. For analysis purposes, responses must be normalized to a common language. Storing both the original patient response and canonical normalized response preserves the complete audit trail while enabling consistent cross-site analysis.

Assertions

- A. The platform SHALL support localized questionnaires with independent translation versioning.
- B. The system SHALL store the language identifier showing the specific language and locale presented to the patient (e.g., es-MX for Spanish-Mexico).
- C. The system SHALL store the translation version for each language, independent of the source content version.
- D. The system SHALL store the source content reference indicating which source language content version each translation is based upon.
- E. The system SHALL capture the original response as the exact value the patient entered or selected in their language.
- F. The system SHALL capture the canonical response as the normalized value used for study analysis.
- G. The system SHALL store the translation method for free-text translations, indicating whether the canonical value was auto-translated, manually translated, or verified by a human translator.
- H. The system SHALL record patient language preference at enrollment.
- I. The system SHALL present questionnaires in the patient's configured language.
- J. The system SHALL track translation version per language per questionnaire.
- K. The system SHALL enable reconstruction of the audit trail showing the exact localized content shown to each patient.
- L. The system SHALL support management of translation versions independently of source content versions.

End Questionnaire Localization and Translation Tracking | **Hash:** 4218237c

DIARY-PRD-questionnaire-sponsor-eligibility: Sponsor Questionnaire Eligibility Configuration

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-questionnaire-versioning

Rationale

Multi-sponsor deployments require sponsor-specific questionnaire portfolios to accommodate varying study designs. One sponsor may use only epistaxis tracking while another includes quality-of-life assessments. Version constraints enable patients in ongoing studies to continue using validated instrument versions while new enrollments can use updated versions, ensuring data consistency within study cohorts. Language enablement ensures only properly validated translations are offered to participants. This configuration-driven approach allows questionnaire changes to be managed deliberately, preventing unintended mid-study modifications that could impact response patterns and data quality. The platform enforces these constraints at data capture time to maintain study protocol compliance and data integrity across the multi-sponsor environment.

Assertions

- A. The system SHALL allow each sponsor to configure which questionnaire types are enabled for their clinical trial.
- B. The system SHALL allow each sponsor to configure which questionnaire versions are enabled for their clinical trial.
- C. The system SHALL allow each sponsor to configure which questionnaire languages are enabled for their clinical trial.
- D. Sponsor questionnaire configuration SHALL specify the current version for new entries.
- E. Sponsor questionnaire configuration SHALL specify the minimum accepted version for historical data.
- F. Sponsor questionnaire configuration SHALL designate the source language for each enabled questionnaire.
- G. The system SHALL present only sponsor-enabled questionnaires in client applications.
- H. The system SHALL use the configured current version when capturing new questionnaire data.

- I. The system SHALL accept historical questionnaire data from any version between the minimum version and the current version inclusive.
- J. The system SHALL restrict language options to sponsor-enabled translations during data capture.
- K. The system SHALL validate questionnaire responses against the rules defined in the appropriate questionnaire version.
- L. The system SHALL enforce sponsor eligibility constraints during all data capture operations.
- M. Configuration changes SHALL NOT invalidate existing historical questionnaire data.
- N. The system SHALL support addition of new questionnaire types whose content conforms to an existing renderer class through catalog entry creation and sponsor configuration update, without requiring renderer code changes.

End Sponsor Questionnaire Eligibility Configuration | **Hash:** ac2f2aac

1.19 Role-Based Access Control

This file groups the platform-side role-based access control (RBAC) requirements: the customizable RBAC foundation, the master Action Inventory, the generic Role Definitions, and the interface behavior for switching between assigned roles. The sponsor-specific permissions table (binding roles to actions) and the per-role site-view scope live in the `hht_diary_callisto` repo (CAL-PRD-permissions-table, CAL-PRD-site-view-scope) and Refines the requirements established here.

DIARY-PRD-rbac-customizable: Customizable Role-Based Access Control

Level: prd | **Status:** Legacy | **Implements:** -

Overview

The system supports multi-role assignment and seamless role switching without disrupting active sessions. Self-privilege-escalation is prevented structurally by disallowing users to modify role assignments of their own account.

Assertions

- A. The System SHALL allow a **User Account** to be assigned one or more roles.
- B. The System SHALL allow an active role to be switched without terminating the user's session.
- C. The System SHALL NOT permit a user to modify the role assignments of their own **User Account**.

Rationale

Clinical trial sponsor portals are routinely staffed by individuals who hold more than one operational role across studies or sites (e.g. a Study Coordinator who is also an Administrator on a sister study). Forcing those users to maintain separate accounts duplicates audit trails, fragments responsibility, and increases the operational burden of account lifecycle. Allowing multi-role assignment with seamless switching keeps a single accountable identity per individual while supporting the legitimate need to perform actions in different capacities. The structural prohibition on self-modification of role assignments is a defense-in-depth control against privilege escalation that does not depend on any single permission check elsewhere in the system.

End Customizable Role-Based Access Control | **Hash:** 099e2eb7

DIARY-PRD-action-inventory: Action Inventory**Level:** prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-rbac-customizable**Overview**

The Action Inventory enumerates every operation subject to role-based access control. It is the authoritative source for the sponsor-level permissions table — every permission granted or denied in a sponsor deployment must reference an action from this inventory. Actions not listed here cannot be granted or denied. Sponsor deployments may extend the inventory with deployment-specific actions.

Definitions

Action: A discrete operation that a user can perform within the System, subject to role-based access control. An Action either changes system state or retrieves protected data.

Action Inventory: The complete enumerated set of Actions the System recognizes, comprising the platform-level inventory in this requirement and any sponsor-level extensions.

Sponsor-Level Action Extension: An additional Action defined by a specific sponsor deployment beyond the platform Action Inventory.

Assertions

- A. The System SHALL enforce role-based access control for every **Action** in the **Action Inventory**.
- B. The System SHALL reject any attempt to perform an **Action** by a user whose active role does not have permission for that **Action**.
- C. The System SHALL permit an **Action** to proceed when the user's active role has permission for that **Action** and the user is within the permitted scope.
- D. The System SHALL support **Sponsor-Level Action Extensions** to the **Action Inventory**.

Action Inventory Table

Category	Action ID	Action
Participant Management	ACT-PAT-001	Link participant
	ACT-PAT-002	Start Trial
	ACT-PAT-003	Disconnect participant

Category	Action ID	Action
Questionnaire	ACT-PAT-004	Reconnect participant
	ACT-PAT-005	Mark participant as not participating
	ACT-PAT-006	Reactivate participant
	ACT-PAT-007	View participant info
	ACT-QST-001	Send questionnaire
	ACT-QST-002	Call back questionnaire
	ACT-QST-003	Finalize questionnaire
User Account	ACT-USR-001	Create User Account
	ACT-USR-002	Edit User Account
	ACT-USR-003	Deactivate User Account
	ACT-USR-004	Reactivate User Account
	ACT-USR-005	Unlock User Account
	ACT-USR-006	Resend activation email
	ACT-USR-007	Assign role to User Account
	ACT-USR-008	Assign Site to User Account
Site	ACT-SIT-001	View Sites
Audit Log	ACT-AUD-001	View audit log
Administrator Settings	ACT-ADM-001	View Administrator Settings

Rationale

The Action Inventory is the single source of truth for what operations the platform recognizes as access-controlled. Sponsor deployments author their permissions table by referencing these Action IDs; deployments may extend the inventory with sponsor-specific actions but cannot redefine or remove platform actions. Centralizing the inventory ensures that every audit log entry, permission check, and role binding refers to a stable, named operation rather than an ad hoc string, which is essential for FDA 21 CFR Part 11 auditability and for cross-sponsor consistency in the platform's compliance posture.

End Action Inventory | **Hash:** f17b676a

DIARY-PRD-role-definitions: Role Definitions

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-rbac-customizable

Overview

This requirement defines the generic role templates the platform provides and maps the sponsor portal's named roles to those templates. Sponsor deployments bind permissions to roles via the deployment's permissions table; the platform-side definitions establish the shared vocabulary.

Definitions

Study Coordinator: A portal user responsible for managing day-to-day participant interactions at one or more assigned Sites.

Clinical Research Associate (CRA): A portal user responsible for monitoring Study Coordinator activities and site compliance at one or more assigned Sites.

Administrator: A portal user responsible for managing User Accounts, role assignments, and Site assignments across the study.

Assertions

A. The System SHALL recognize the following roles in the sponsor portal: **Study Coordinator**, **Clinical Research Associate**, and **Administrator**.

B. The System SHALL map the **Study Coordinator** role to the **Investigator** generic role template.

C. The System SHALL map the **Clinical Research Associate** role to the **Auditor** generic role template.

D. The System SHALL map the **Administrator** role to the **Administrator** generic role template.

Rationale

The sponsor portal's named roles are deployment-facing labels chosen for clarity to clinical staff; the underlying generic role templates (Investigator, Auditor, Administrator) carry the platform-level semantics and are the units of inheritance for cross-sponsor permission patterns. Establishing the mapping at the platform level lets sponsor configurations alter labels and add scope rules without diverging from the platform's shared model, and lets cross-cutting compliance behavior key off the generic template rather than every sponsor's chosen name.

End Role Definitions | **Hash:** 1f0ab3a3

DIARY-GUI-role-switching: Role Switching — Interface Behavior

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-rbac-customizable

Overview

This requirement specifies how the sponsor portal interface exposes role switching to users assigned more than one role.

Definitions

Role Selector: UI element displayed in the header that shows the user's currently active role and allows users with multiple assigned roles to switch their active role.

Assertions

Display

- A. The interface SHALL display the **Role Selector** in the header for any user assigned two or more roles.
- B. The interface SHALL NOT display the **Role Selector** for a user assigned exactly one role.
- C. The **Role Selector** SHALL display the user's currently active role.
- D. When opened, the **Role Selector** SHALL display the complete list of the user's assigned roles and SHALL visually indicate which role is currently active.

Interaction

- E. When the user selects a role from the **Role Selector**, the interface SHALL set that role as the active role.
- F. When the active role changes, the interface SHALL load the default landing view for the selected role.
- G. The **Role Selector** SHALL NOT present a confirmation step before switching roles.

Rationale

Users assigned multiple roles need an unobtrusive, always-visible affordance to confirm and change which role is currently active, since the active role determines visible data and available actions throughout the portal. Hiding the selector from single-role users keeps the header uncluttered for the common case. Omitting a confirmation step keeps role switching fast for users who switch many times per session; the underlying audit log already records role context for every action so the cost of an accidental switch is bounded.

End Role Switching — Interface Behavior | **Hash:** 8d6caff0

1.20 Requirements Management

Obligations governing how the Diary Platform maintains formal requirements, traceability, and architectural decision records throughout the system's regulated lifetime. These obligations support FDA 21 CFR Part 11 validation by ensuring every capability is specified, traceable, and auditable, and that significant design decisions are documented for future maintainers and regulatory review.

DIARY-PRD-system-validation-traceability: System Validation and Traceability

Level: PRD | **Status:** Legacy | **Refines:** -

TODO (URS-Phase-3 reconciliation): Original Refines target REQ-p01085 (Compliance Systems) is in URS-replaced prd-clinical-trials.md. Re-parent to the URS-derived compliance REQ when that lands.

Assertions

- A. The system development process SHALL maintain formal requirements with complete traceability from product requirements through operational procedures to implementation code.
- B. Every system capability SHALL be defined in a written requirement.
- C. Every requirement SHALL be traceable to a business need or regulatory mandate.
- D. Every implementation file SHALL be linked to the requirements it implements.
- E. The system SHALL maintain a complete audit trail of requirement changes via version control.
- F. The system SHALL provide traceability matrices demonstrating requirement relationships.
- G. Requirements SHALL follow top-down cascade from PRD to OPS to DEV.
- H. All requirements SHALL follow a standardized format with unique IDs.
- I. All implementation files SHALL include header comments referencing requirements.
- J. Traceability matrices SHALL demonstrate complete requirement relationships.
- K. All requirement changes SHALL be captured in version control history.
- L. Requirements SHALL validate successfully before commits are accepted.
- M. Validation tooling SHALL prevent commits with invalid requirements.
- N. Validation tooling SHALL prevent commits with orphaned requirements.
- O. Top-down cascade SHALL be enforced such that PRD requirements drive OPS and DEV requirements.

P. DEV requirements SHALL NOT drive PRD requirements.

Q. OPS requirements SHALL NOT drive PRD requirements.

Rationale

FDA 21 CFR Part 11 requires validated systems for electronic records in clinical trials. System validation requires formal requirements with complete traceability demonstrating the system was built to specification. Without traceable requirements, the system cannot be validated, and cannot be used for regulatory submissions. Formal requirements also enable change impact analysis, support quality audits, and ensure institutional knowledge is documented rather than tribal knowledge.

End System Validation and Traceability | **Hash:** 7d81caf7

DIARY-PRD-architecture-decision-docs: Architecture Decision Documentation

Level: PRD | **Status:** Legacy | **Refines:** DIARY-PRD-platform-operations-monitoring-M

Assertions

- A. The system SHALL formally document significant architectural and design decisions.
- B. Decision documentation SHALL include context and driving factors for each decision.
- C. Decision documentation SHALL include alternatives that were considered.
- D. Decision documentation SHALL include explicit trade-offs and consequences.
- E. Decision documentation SHALL include the rationale for the decision.
- F. Decision documentation SHALL be reviewable by future maintainers and auditors.
- G. Decision documentation SHALL support decisions being revisable when context changes.
- H. The system SHALL document decisions with long-term impact.
- I. The system SHALL make decision rationale available for audits.
- J. Architectural decisions SHALL be documented in Architecture Decision Records (ADRs).
- K. ADRs SHALL capture context, decision, consequences, and alternatives.
- L. ADRs SHALL be linked to implementing requirements when applicable.
- M. ADRs SHALL follow a defined lifecycle of Proposed, Accepted, Deprecated, or Superseded.
- N. ADRs SHALL be maintained in version control with system evolution.
- O. The system SHALL maintain an ADR index showing all decisions and their status.

Rationale

Clinical trial systems have long operational lifetimes (25+ years per FDA retention requirements). Architectural decisions made today will be maintained by different teams over decades. Formal decision documentation ensures future maintainers understand why the system was built the way it was, enables informed evolution of the system, supports regulatory audits by explaining technical choices, and prevents repeating past mistakes by documenting alternatives that were rejected and why.

End Architecture Decision Documentation | **Hash:** 76c82ce6

1.21 Score Calculation

This file defines the platform's score-calculation obligations: the general rule that scores are computed per a validated algorithm and stored with the questionnaire record, plus the specific algorithm bindings for the **HHT-QoL** and **NOSE HHT** instruments. The corresponding questionnaire definitions live in `prd-questionnaire-overview.md` (assertions B in each REQ cite the validated source documents the algorithms come from).

Authoring note: the URS source uses defective section numbering inside §6.5 (the body repeats 6.4.1, 6.4.2, 6.4.3 from §6.4). This file treats them as §6.5.1, §6.5.2, §6.5.3 respectively, matching the §6.5 chapter heading.

DIARY-PRD-questionnaire-score-calculation: Questionnaire Score Calculation

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Validated clinical questionnaires require scoring according to published, protocol-defined algorithms to ensure data integrity and comparability across participants and studies. Storing scores with survey records ensures traceability and supports downstream Rave EDC synchronization and clinical analysis.

Assertions

- A. The System SHALL calculate **Questionnaire** scores according to the validated algorithm defined for each **Questionnaire**.
- B. The System SHALL store the calculated score with the associated questionnaire record.
- C. The scoring algorithm SHALL be traceable to its source definition for each **Questionnaire Type**.

Rationale

Scores are the analytical artifact downstream clinical research derives outcomes from — frequency, severity, quality-of-life impact. Computing scores per the validated published algorithm rather than per a platform-local re-derivation is what gives the resulting numbers their interpretive validity: a NOSE HHT score from this platform is comparable to a NOSE HHT score from any other study only because both implement the published JAMA Otolaryngology algorithm faithfully. Storing the score with the

questionnaire record (rather than recomputing on demand) preserves the value the participant's answers actually produced at the time of submission — even if the algorithm reference is later updated, the historical record reflects what was computed and shipped to **Rave EDC** for that participant. Traceability to source for each **Questionnaire Type** is the audit-trail counterpart of the True Copy obligation on the questionnaire definitions: any auditor must be able to follow the chain from a stored score back to the published algorithm that produced it.

End Questionnaire Score Calculation | **Hash:** 85a0d5dd

DIARY-PRD-score-hht-qol: HHT-QoL Score Calculation

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-questionnaire-score-calculation

Assertions

A. The System SHALL calculate the **HHT-QoL** questionnaire score according to the algorithm defined in: Development and performance of a hereditary hemorrhagic telangiectasia-specific quality-of-life instrument, Blood Advances, 26 July 2022, Volume 6, Number 14, DOI 10.1182/bloodadvances.2022007748.

Rationale

The **HHT-QoL** scoring algorithm is fully specified by the cited Blood Advances 2022 publication; the platform's role is to implement that algorithm faithfully against the questionnaire's four answer fields. Anchoring the requirement to the DOI rather than restating the algorithm here preserves the property that the published reference is the single source of truth — any platform-side restatement would create a second authority that could drift from the original. The DOI is stable across editorial revisions, so an auditor can retrieve exactly the version the platform's implementation was checked against.

End HHT-QoL Score Calculation | **Hash:** 175ac62e

DIARY-PRD-score-nose-hht: NOSE HHT Score Calculation

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-questionnaire-score-calculation

Assertions

A. The System SHALL calculate the **NOSE HHT** questionnaire score according to the algorithm defined in: Development and Validation of the Nasal Outcome Score for Epis-

taxis in Hereditary Hemorrhagic Telangiectasia (NOSE HHT), JAMA Otolaryngology – Head & Neck Surgery, November 1, 2020, Volume 146, Number 11, DOI 10.1001/jamaoto.2020.3040.

Rationale

The **NOSE HHT** scoring algorithm is fully specified by the cited JAMA Otolaryngology 2020 publication, which is the same publication the questionnaire definition itself cites for the question and answer choices. Anchoring the score requirement to the same DOI keeps the questionnaire and its score on a single source-of-truth chain: the published instrument defines what to ask and how to score the answers, and the platform's two requirements (one for the questionnaire content, one for the score) both reference it. As with **HHT-QoL**, the DOI is the stable retrieval handle an auditor uses to confirm the platform's implementation matches the validated reference.

End NOSE HHT Score Calculation | **Hash:** 7610ed55

1.22 Service Level Agreement

Service-level commitments the Diary Platform makes to sponsors regarding availability, incident handling, disaster recovery, and regulatory event support. These obligations are sponsor-visible (PRD-level audience) and underpinned by operational obligations in ops-platform-operations.md and related OPS-level specs.

DIARY-PRD-sla-service-availability: Service Availability Commitment

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-platform-operations-monitoring-G+H

Rationale

99.9% uptime (~43 minutes monthly downtime) is industry standard for healthcare SaaS platforms requiring 24/7 availability. This commitment balances reliability with operational feasibility for a clinical trial platform. The requirement establishes clear service level expectations for sponsors and participants, ensuring the platform maintains sufficient availability for clinical trial operations while allowing for necessary maintenance windows and circumstances beyond operational control.

Assertions

- A. The platform SHALL provide 99.9% monthly uptime for all customer-facing production services.
- B. The availability commitment SHALL include portal web application accessibility.
- C. The availability commitment SHALL include mobile app synchronization API availability.
- D. The availability commitment SHALL include authentication service availability.
- E. The availability commitment SHALL include database connectivity for read/write operations.
- F. Uptime percentage SHALL be calculated as $(\text{Total Minutes in Month} - \text{Downtime Minutes}) / \text{Total Minutes in Month} \times 100$.
- G. Scheduled maintenance with 48-hour advance notice SHALL be excluded from downtime calculation.
- H. Force majeure events, including natural disasters and widespread internet outages, SHALL be excluded from downtime calculation.
- I. Customer-caused outages or failures SHALL be excluded from downtime calculation.
- J. Beta or trial services SHALL be excluded from downtime calculation.
- K. The system SHALL measure uptime monthly per sponsor.

- L. The system SHALL report uptime monthly per sponsor.
- M. The system SHALL perform automated uptime monitoring with 60-second check intervals.
- N. Uptime reports SHALL be available to sponsors on request.
- O. Historical uptime data SHALL be retained for 2 years.

End Service Availability Commitment | **Hash:** d39861a9

DIARY-PRD-sla-incident-severity: Incident Severity Classification

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-platform-operations-monitoring-E+F

Rationale

Standardized severity classification ensures consistent prioritization, appropriate resource allocation, and clear customer expectations for incident handling. This requirement establishes a four-tier severity model that aligns response timelines, escalation procedures, and customer communications with the actual business and patient safety impact of service incidents. The classification system enables operational teams to make rapid triage decisions while providing stakeholders with predictable expectations for incident resolution.

Assertions

- A. The system SHALL classify all service incidents by severity level.
- B. The system SHALL support a Critical (P0) severity level for complete service loss or security events with active exploitation, data breach, or risk to patient safety where no workaround is available.
- C. The system SHALL support a High (P1) severity level for significant service degradation or security events with high exploitation likelihood where partial workarounds may exist.
- D. The system SHALL support a Medium (P2) severity level for moderate impact to non-critical functions or security events with limited risk where workarounds are available.
- E. The system SHALL support a Low (P3) severity level for minor impact, cosmetic issues, or negligible security risk with no immediate operational threat.
- F. Critical (P0) severity SHALL include incidents such as database down, authentication failure, data corruption, or active security breach.
- G. High (P1) severity SHALL include incidents such as slow response times exceeding 10 seconds, sync failures for subset of users, or elevated error rates.

- H. Medium (P2) severity SHALL include incidents such as minor feature unavailable, cosmetic issues, or non-critical integrations down.
- I. Low (P3) severity SHALL include incidents such as UI inconsistencies, minor performance degradation, or documentation errors.
- J. The system SHALL assign a severity level to all incidents at the time of detection.
- K. The incident severity level SHALL determine the response timeline for the incident.
- L. The incident severity level SHALL determine the escalation path for the incident.
- M. The system SHALL notify customers of the severity level for incidents impacting them.
- N. The system SHALL allow severity levels to be upgraded with documented justification.
- O. The system SHALL allow severity levels to be downgraded with documented justification.

End Incident Severity Classification | **Hash:** d1734735

DIARY-PRD-sla-incident-response-times: Incident Response Times

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-sla-incident-severity-A+K

Rationale

Defined response times set clear expectations for customers and ensure appropriate urgency in incident handling. Resolution timelines account for complexity while maintaining accountability. This requirement establishes service level commitments for incident management, ensuring that incidents are acknowledged and resolved within timeframes appropriate to their severity. The tiered approach balances customer expectations with practical considerations of incident complexity, while tracking and reporting mechanisms ensure organizational accountability.

Assertions

- A. The platform SHALL respond to Critical (P0) incidents within 1 hour of detection.
- B. The platform SHALL respond to High (P1) incidents within 4 hours of detection.
- C. The platform SHALL respond to Medium (P2) incidents within 1 business day of detection.
- D. The platform SHALL respond to Low (P3) incidents within 2 business days of detection.
- E. The platform SHALL resolve Critical (P0) incidents within 7 days of detection.

- F. The platform SHALL resolve High (P1) incidents within 14 days of detection.
- G. The platform SHALL resolve Medium (P2) incidents within 30 days of detection.
- H. The platform SHALL resolve Low (P3) incidents within 90 days of detection.
- I. Initial response SHALL include confirmation that the incident is being investigated.
- J. Initial response SHALL include the assigned severity level.
- K. Initial response SHALL include a preliminary impact assessment.
- L. Initial response SHALL include the timeline for next communication.
- M. Resolution SHALL include restoration of service to normal operation.
- N. Resolution SHALL include identification of root cause for Critical (P0) incidents.
- O. Resolution SHALL include identification of root cause for High (P1) incidents.
- P. Resolution SHALL include preventive measures that are either implemented or scheduled.
- Q. The system SHALL track response time from incident detection to first customer communication.
- R. The system SHALL track resolution time from incident detection to service restoration.
- S. The platform SHALL report SLA compliance monthly.
- T. The system SHALL trigger escalation when approaching response or resolution deadlines.

End Incident Response Times | **Hash:** 40fb577e

DIARY-PRD-sla-disaster-recovery: Disaster Recovery Objectives

Level: prd | **Status:** Legacy | **Implements:** -

Rationale

This requirement ensures business continuity for clinical trial operations through defined disaster recovery capabilities. The 4-hour RPO (Recovery Point Objective) aligns with backup frequency to minimize data loss in FDA-regulated environments, while the 24-hour RTO (Recovery Time Objective) allows for thorough recovery verification required for regulated systems. Regular drills and documented procedures ensure preparedness for actual disaster scenarios, and timely sponsor communication maintains transparency during incidents.

TODO (URS-Phase-3 reconciliation): Original Refines target REQ-p00012 (Clinical Data Retention Requirements) is in URS-replaced prd-clinical-trials.md. Re-parent to the URS-derived clinical-trials REQ when that lands.

Assertions

- A. The platform SHALL maintain a Recovery Time Objective (RTO) of 24 hours for full production service restoration.
- B. The platform SHALL maintain a Recovery Point Objective (RPO) of 4 hours representing the maximum acceptable data loss window.
- C. The platform SHALL support point-in-time database recovery within the RPO window.
- D. The platform SHALL support service restoration to alternate infrastructure when needed.
- E. The platform SHALL verify data integrity after recovery completion.
- F. The platform SHALL communicate recovery status to affected sponsors during recovery operations.
- G. Automated backups SHALL run at intervals meeting the RPO objective of every 4 hours minimum.
- H. Disaster recovery drills SHALL be conducted quarterly.
- I. Disaster recovery drills SHALL be documented.
- J. Recovery procedures SHALL be validated.
- K. Recovery procedures SHALL be documented.
- L. The platform SHALL notify sponsors within 1 hour of disaster declaration.

End Disaster Recovery Objectives | **Hash:** 6b076a50

DIARY-PRD-sla-customer-notification: Customer Incident Notification

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-sla-incident-severity-M

Rationale

Proactive customer communication maintains trust and allows sponsors to inform their study teams about system incidents. Transparent incident notification is essential for FDA-regulated clinical trial environments where sponsors have regulatory oversight responsibilities and must maintain awareness of any issues affecting data collection or system availability. Timely and structured notifications enable sponsors to take appropriate actions, such as informing study coordinators or documenting potential impacts on trial conduct.

Assertions

- A. The system SHALL notify customers of Critical (P0) incidents within 15 minutes of incident declaration.
- B. The system SHALL notify customers of High (P1) incidents within the initial response time defined for that severity level.
- C. The system SHALL notify customers of Medium (P2) incidents within the initial response time defined for that severity level.
- D. The system SHALL notify customers of Low (P3) incidents when the incident is resolved.
- E. The system SHALL deliver Critical (P0) incident notifications via email, status page, and direct contact channels.
- F. The system SHALL deliver High (P1) incident notifications via email and status page channels.
- G. The system SHALL deliver Medium (P2) incident notifications via email channel.
- H. The system SHALL deliver Low (P3) incident notifications via status page channel.
- I. Incident notifications SHALL include a brief description of the issue.
- J. Incident notifications SHALL include the impact scope identifying affected functionality.
- K. Incident notifications SHALL include the current status as one of: investigating, identified, monitoring, or resolved.
- L. Incident notifications SHALL include the estimated time to resolution when such estimate is known.
- M. Incident notifications SHALL include the timeline for the next update.
- N. The system SHALL automatically update the status page for all P0 incidents.
- O. The system SHALL automatically update the status page for all P1 incidents.
- P. The system SHALL maintain pre-approved customer notification templates.
- Q. The system SHALL track notification timing for all customer notifications.
- R. The system SHALL report on notification timing metrics.
- S. The system SHALL provide subscriber management functionality for status update notifications.

End Customer Incident Notification | **Hash:** 3ca3df0f

DIARY-PRD-sla-root-cause-analysis: Root Cause Analysis

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-sla-incident-response-times-N+O

Rationale

Root cause analysis prevents recurring incidents and demonstrates due diligence for regulatory compliance. The systematic investigation of significant incidents ensures that underlying causes are identified and addressed, reducing the likelihood of recurrence. This documentation supports sponsor audit requirements, demonstrates continuous improvement efforts, and maintains compliance with FDA 21 CFR Part 11 record retention requirements.

Assertions

- A. The system SHALL require documented root cause analysis (RCA) for all Critical (P0) incidents within 5 business days.
- B. The system SHALL require documented root cause analysis (RCA) for all High (P1) incidents within 10 business days.
- C. The system SHALL support summary RCA documentation for Medium (P2) incidents upon request.
- D. The system SHALL NOT require RCA documentation for Low (P3) incidents.
- E. RCA documentation SHALL include an incident timeline from detection to resolution.
- F. RCA documentation SHALL include root cause identification.
- G. RCA documentation SHALL include contributing factors.
- H. RCA documentation SHALL include impact assessment.
- I. RCA documentation SHALL include corrective actions taken.
- J. RCA documentation SHALL include preventive measures implemented.
- K. The system SHALL retain all RCA documentation for FDA inspection per 21 CFR Part 11 requirements.
- L. The system SHALL use a standardized RCA template.
- M. RCAs SHALL be reviewed by a technical lead before delivery.
- N. The system SHALL track RCA delivery against defined timelines.
- O. The system SHALL archive RCAs for 7 years.

End Root Cause Analysis | **Hash:** 0778e3bb

DIARY-PRD-sla-corrective-preventive: Corrective and Preventive Action

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-sla-root-cause-analysis-I+J

Rationale

CAPA (Corrective and Preventive Action) processes are a cornerstone of FDA quality management systems for regulated software. By establishing formal processes to respond to incidents affecting data integrity or regulatory compliance, the system demonstrates a commitment to continuous improvement and proactive risk management. This requirement aligns with FDA expectations for quality systems under 21 CFR Part 820 (Quality System Regulation) and ICH Q9 (Quality Risk Management). CAPA processes provide sponsors with confidence that issues are systematically addressed through root cause analysis, implementation of corrective actions, and verification of effectiveness. The documentation and tracking requirements ensure transparency and accountability throughout the remediation lifecycle.

Assertions

- A. The system SHALL trigger a Corrective and Preventive Action (CAPA) process for all incidents affecting data integrity or regulatory compliance.
- B. The system SHALL initiate CAPA within 72 hours of incident confirmation.
- C. CAPA documentation SHALL be made available to the sponsor for audit purposes.
- D. The system SHALL document effectiveness verification within 30 days of CAPA implementation.
- E. The system SHALL track all CAPAs to completion.
- F. CAPA documentation SHALL include a problem description and scope.
- G. CAPA documentation SHALL include a root cause analysis reference.
- H. CAPA documentation SHALL include corrective actions (immediate fixes).
- I. CAPA documentation SHALL include preventive actions (systemic improvements).
- J. CAPA documentation SHALL include effectiveness criteria.
- K. CAPA documentation SHALL include verification results.
- L. The system SHALL initiate CAPA for all P0 incidents.
- M. The system SHALL initiate CAPA for all P1 incidents affecting data integrity.
- N. The system SHALL maintain a CAPA tracking system.
- O. CAPA status SHALL be visible to the sponsor upon request.
- P. The system SHALL perform an annual CAPA effectiveness review.

End Corrective and Preventive Action | **Hash:** f2c78d76

DIARY-PRD-sla-data-recovery: Data Recovery Guarantee

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-sla-disaster-recovery-A+B+C

Rationale

Data recovery guarantees are critical for maintaining clinical trial integrity and regulatory compliance when disaster scenarios occur. This requirement ensures that sponsors can fulfill their obligations to regulatory authorities (FDA, EMA) to maintain complete and accurate trial records, even after data loss or corruption events. The no-cost guarantee protects sponsors from financial exposure during recovery operations. Clear communication protocols support sponsor responsibilities under 21 CFR Part 11 and ICH-GCP to document and report data integrity incidents. Recovery procedures must be tested and documented to demonstrate preparedness during regulatory audits.

Assertions

- A. The platform SHALL provide data recovery services at no additional cost in the event of data loss or corruption.
- B. The platform SHALL restore data to the most recent Recovery Point Objective (RPO) checkpoint during recovery operations.
- C. The platform SHALL provide a detailed impact assessment if full data recovery is not possible.
- D. The platform SHALL reconstruct affected audit trails where technically feasible during recovery operations.
- E. The platform SHALL document all recovery actions for regulatory reporting purposes.
- F. Recovery communication SHALL include the scope of data affected.
- G. Recovery communication SHALL include the recovery actions taken.
- H. Recovery communication SHALL include data verification results.
- I. Recovery communication SHALL include recommendations for sponsor regulatory notification.
- J. Recovery procedures SHALL be documented.
- K. Recovery procedures SHALL be tested.
- L. The platform SHALL use a recovery verification checklist post-recovery.
- M. Recovery documentation SHALL be suitable for regulatory submission.

End Data Recovery Guarantee | **Hash:** 5909fb2b

DIARY-PRD-sla-chronic-failure-escalation: Chronic Failure Escalation

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-sla-service-availability-A

Rationale

Chronic failure provisions protect sponsors from persistent service degradation and provide clear accountability for sustained reliability issues. This requirement establishes contractual safeguards when service reliability consistently falls below acceptable thresholds, defining specific uptime criteria that trigger escalation processes, mandatory remediation timelines, and sponsor exit rights. The provisions ensure that sponsors have recourse when platform reliability problems persist over extended periods, preventing situations where individual incidents meet SLA requirements but cumulative degradation creates unacceptable risk to clinical trial operations.

Assertions

- A. The system SHALL trigger escalation and remediation processes when service reliability falls below acceptable thresholds for sustained periods.
- B. A chronic failure condition SHALL be defined as uptime below 99.0% for three (3) consecutive months.
- C. The platform SHALL enable sponsors to request a mandatory executive escalation meeting within 10 business days when chronic failure occurs.
- D. The platform SHALL provide a written remediation plan with milestones within 15 business days of chronic failure escalation.
- E. The platform SHALL grant sponsors the right to terminate without penalty upon 60 days written notice if remediation is unsuccessful.
- F. The system SHALL track monthly uptime for each sponsor.
- G. The system SHALL report monthly uptime metrics to each sponsor.
- H. The system SHALL automatically notify sponsors when approaching the chronic failure threshold.
- I. The platform SHALL maintain a remediation plan template for chronic failure scenarios.
- J. The system SHALL document escalation contacts per sponsor.
- K. The platform SHALL provide service remedies when uptime commitments are not met, as defined in operational specifications.

End Chronic Failure Escalation | **Hash:** 63205737

DIARY-PRD-sla-regulatory-event-support: Regulatory Event Support

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-sla-root-cause-analysis-A+B

Rationale

SLA failures in FDA-regulated clinical trial systems may trigger sponsor obligations for regulatory notification, inspection responses, or audit findings under FDA 21 CFR Part 11 and ICH-GCP guidelines. Provider support ensures sponsors can meet their regulatory responsibilities by providing timely documentation, technical expertise, and corrective action implementation without imposing additional financial burden during regulatory events.

Assertions

- A. The system SHALL provide support services when an SLA failure results in a regulatory inquiry or inspection finding.
- B. The system SHALL provide all relevant documentation within 5 business days of a regulatory request.
- C. The system SHALL participate in regulatory responses at no additional cost to the sponsor.
- D. The system SHALL implement required corrective actions within agreed timelines.
- E. The system SHALL support the sponsor in preparing regulatory submissions.
- F. Documentation packages SHALL include incident timeline and root cause analysis.
- G. Documentation packages SHALL include system validation documentation.
- H. Documentation packages SHALL include audit trail exports.
- I. Documentation packages SHALL include CAPA documentation.
- J. Documentation packages SHALL be prepared within 5 business days of a regulatory event.
- K. Technical staff SHALL be available for regulatory calls.
- L. Corrective action timelines SHALL be negotiated in good faith.
- M. The system SHALL NOT impose additional charges for regulatory support services.

End Regulatory Event Support | **Hash:** f62f5e4d

1.23 User Account Management

This file groups all platform-side user-account lifecycle requirements (administration in §5.1 of the URS) and their corresponding sponsor portal interface surfaces (user interface in §5.2). The PRD-level requirements define what the platform supports for **User Account** creation, activation, site assignment, email-resend, edit, deactivation, and reactivation; the GUI requirements specify how the sponsor portal exposes these capabilities through tabs, modals, and the Administrator Dashboard. The shared **Reason Field Constraints** template governs every free-text reason input across this surface and is consumed by the Reason Dialog appendix entries.

DIARY-PRD-user-account-create: Create User Account

Level: prd | **Status:** Legacy | **Implements:** -

Overview

This requirement defines the minimum identity and authorization data the **System** captures when an **Administrator** creates a new **User Account**. Subsequent requirements in this file refine activation, site assignment, edits, and lifecycle transitions on the account established here.

Definitions

User Account: The complete set of identity, credential, and authorization data that the **Sponsor Portal** maintains for a single human user.

Full Name: A common name, which is **PII**, of the person associated with a **User Account** that identifies a specific individual in the context of the **Sponsor Portal**.

Email Address: A unique technical identifier used as a destination for system notifications and as a username for authentication to access the system.

Assertions

- A. The **System** SHALL require a **Full Name**, **Email Address**, at least one **Role**, and — for non-**Administrator** accounts — at least one **Site** to be selected before a **User Account** is created.
- B. The **System** SHALL validate the format of the email address provided (e.g., presence of an "@" symbol).
- C. The **System** SHALL allow assignment of the **Administrator**, **CRA** and **Study Coordinator** role during User Account creation.

Rationale

A **User Account** is the unit of authorization and audit attribution in the **Sponsor Portal**: every action recorded in the audit trail is bound to the account that performed it, and every permission check resolves against the account's assigned roles and Sites. Requiring **Full Name**, **Email Address**, role, and (for non-Administrators) at least one Site at creation time ensures the account is fully addressable, fully scoped, and fully attributable from the moment it exists; partial accounts that lack one of these fields would either be unable to receive activation email, unable to participate in audit reporting, or unable to be authorized for any operation. Validating the email format at the creation boundary catches obvious typos before the activation flow consumes an invalid address.

End Create User Account | **Hash**: e92dfa41

DIARY-PRD-user-account-activation-workflow: Account Activation Workflow

Level: prd | **Status**: Legacy | **Implements**: - **Refines**: DIARY-PRD-user-account-create

Overview

The account activation process starts when a **User Account** is created, reactivated, or when an activation email is sent again. Activation links are time-limited and can only be used once, which helps protect account security. A user cannot access the system until their account has been successfully activated.

Definitions

Verification Link: A unique, time-limited, single-use secured URL delivered to an email address to confirm that the recipient controls that address.

Activation Webpage: A **Verification Link** associated with a **User Account** that provides a mechanism for the **Account Owner** to activate their account and configure their password and 2FA.

Assertions

Account States

A. When the status of the **User Account** is **Pending Activation**, the user SHALL NOT be able to access the system.

Activation Webpage Lifecycle

B. When a **User Account** is set to **Pending Activation**, the **System** SHALL create a unique **Activation Webpage** for that **User Account**.

C. When an **Activation Webpage** is created, the **System** SHALL deliver it to the associated **User Account** owner's registered **Email Address**. The username of the user is the **Email Address** that has been entered during user creation.

D. When a **User Account** owner has activated their account and configured a valid password and 2FA, the **System** SHALL set its status to **Active**.

Email Address Verification Lifecycle

E. When an **Administrator** updates the email address of a user, a new **Verification Link** is generated and sent to that updated email address. When the **Verification Link** is used, the **System** SHALL replace the previous **Email Address** with the new **Email Address** as the **User Account**'s username.

Verification Link Behavior

F. When a **Verification Link** is used or 14 days have passed from generation, whichever occurs first, the **System** SHALL invalidate the **Verification Link**.

G. When a new **Verification Link** is issued for a **User Account**, the **System** SHALL invalidate any previously issued **Verification Link** for that **User Account**.

Rejection Behavior

H. When a user attempts to access an invalidated **Verification Link**, the **System** SHALL reject the attempt and display the following message: "This link is no longer valid. Please contact your Administrator to request a new activation email."

Rationale

Account activation is the boundary at which a candidate **User Account** (created or restored by an Administrator) becomes a credentialed account that can authenticate. Two security properties are essential at this boundary: control of the registered email address (verified by delivering a single-use link to that address), and a bounded window in which the verification holds (14 days). Single-use enforcement prevents the link from being replayed if it leaks; expiry caps the window during which a stolen or stale link could be exploited; invalidating prior links when a new one is issued ensures that resend operations do not leave multiple valid paths open simultaneously. The mid-lifecycle email-update path uses the same verification mechanism so that an Administrator-initiated address change cannot redirect login to an address the user does not control. The rejection message is deliberately generic — it directs the user to the Administrator without disclosing whether the underlying account exists, which preserves user-enumeration resistance.

End Account Activation Workflow | **Hash:** d66d0ab7

DIARY-PRD-user-account-site-assignment: Site Assignment

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-user-account-create

Overview

Sites are managed in **Rave EDC**. The portal displays **Sites** for assignment purposes only and does not allow modification, to maintain data integrity. Near real-time synchronization ensures that the site list reflects the data within **Rave EDC** at all times.

Definitions

Site: A clinical research location authorized by the Sponsor to enroll **Participants** and conduct trial activities. Each **Site** is uniquely identified and synchronized from **Rave EDC**.

Assertions

- A. The system SHALL include only **Sites** that are created in **Rave EDC**.
- B. The system SHALL refresh the list of valid **Sites** each time an **Administrator** logs in.
- C. An **Administrator** SHALL have access to all **Sites**.
- D. The system SHALL allow an **Administrator** to assign any **Site** to a **User Account**.

Rationale

Sites are clinical trial entities owned by the EDC system of record, not by the portal; replicating Site authorship in the portal would create a divergent inventory and break the EDC-to-portal data-integrity contract. The portal therefore reads the Site list and never edits it, refreshing on Administrator login so that newly-created Sites are available promptly for assignment without requiring an explicit sync action. Administrators are granted study-wide Site visibility because their accountability for **User Account** lifecycle requires they be able to assign any Site to any account; restricting Administrators to a Site subset would block legitimate account configuration and create operational deadlocks during staff onboarding.

End Site Assignment | **Hash**: 3aa95698

DIARY-PRD-user-account-activation-resend: Resend Activation Email

Level: prd | **Status**: Legacy | **Implements**: - **Refines**: DIARY-PRD-user-account-activation-workflow

Overview

A user may not receive or may lose their activation email before completing setup. Resending generates a new **Activation Webpage** and invalidates the previous one, ensuring only one valid link exists at any time.

Assertions

- A. While the status of the **User Account** is **Pending Activation**, the system SHALL allow the **Administrator** to resend a new activation email.
- B. The system SHALL NOT impose a limit on the number of resend attempts for a **Pending Activation User Account**.
- C. When an activation email is resent, the system SHALL create a new **Activation Webpage** for that **User Account** and invalidate the previous one.

Rationale

The activation email is the only delivery channel for the **Activation Webpage**, and legitimate failure modes (spam filtering, address typos surfaced after creation, transient email infrastructure issues) are common. Allowing unlimited resends keeps the recovery path open for the Administrator without requiring an escalation channel; the resend operation is itself audited and rate-limited only by the Administrator's manual cadence, so abuse potential is low. Invalidating the previous link on each resend keeps the single-valid-link invariant from the parent activation workflow intact: at most one **Verification Link** exists per **User Account** at any moment, regardless of how many times the activation email has been resent.

End Resend Activation Email | **Hash:** fb61a75a

DIARY-PRD-user-account-edit: Edit User Account

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-user-account-create

Overview

This functionality allows user access to be updated as roles or responsibilities change. If access is removed or changed, the user is immediately logged out by the system to prevent access to information they should no longer see. When an email address is updated, the new one must be verified by the user again before using their user account.

Assertions

- A. The **System** SHALL allow an **Administrator** to edit any **User Account** other than their own.
- B. The **System** SHALL require at least one **Role** for non-**Administrator** users.
- C. The **System** SHALL require at least one **Site** for non-**Administrator** users.
- D. The **System** SHALL validate **Email Address** uniqueness across all existing **User Accounts**.
- E. When an **Administrator** changes a **User Account's Role** or **Site**, the **System** SHALL enforce the change immediately.
- F. When an **Administrator** changes a **User Account's Email Address**, the **System** SHALL deliver a **Verification Link** to the new **Email Address**.
- G. When an **Administrator** initiates an **Email Address** change, the **System** SHALL notify the **User Account** owner at the original **Email Address** that a change has been initiated.

Rationale

User edits encode three risks the platform contains structurally. First, an Administrator editing their own account could escalate privilege or remove their own deactivation safeguard; prohibiting self-edit closes that channel. Second, role or site changes that take effect on the next login leave a window in which a user retains stale authorization; enforcing the change immediately collapses that window. Third, an email change is effectively a credential change (the email is the username and the activation channel), so the new address must be verified by the same mechanism used at account creation, and the prior address must be notified so that an unauthorized change initiated against an unattended Administrator session is visible to the legitimate account owner.

End Edit User Account | **Hash:** c4d1c981

DIARY-PRD-user-account-deactivate: Deactivate User Account

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-user-account-create

Overview

Deactivation is the action of revoking a **User Account's** ability to access the system without permanently removing the account or its associated data. A deactivated account retains all historical data and audit trail but cannot be used to log in or maintain active sessions.

Definitions

Deactivation: The action of revoking a **User Account**'s ability to access the system without permanently removing the account or its associated data. A deactivated account retains all historical data and **Audit Trail** but cannot be used to log in or maintain active sessions.

Assertions

- A. The System SHALL allow the Administrator to deactivate any **User Account** except their own.
- B. The System SHALL terminate all active sessions associated with a **User Account** immediately upon **deactivation**.
- C. The System SHALL prevent login to a deactivated **User Account**.
- D. The System SHALL preserve all data and **Audit Trail** associated with a deactivated **User Account**.
- E. The System SHALL set the status of a deactivated **User Account** to **Deactivated**.
- F. When an **Administrator** deactivates a **User Account**, the **System** SHALL require a free text reason before applying the change. The reason SHALL be captured in the audit trail.

Rationale

Deactivation is the standard off-boarding mechanism for users who should no longer access the system — completion of a study assignment, role change off the study, departure from the sponsor or site organization. Retaining the account record and its full **Audit Trail** is required by FDA 21 CFR Part 11: every historical action attributed to that account must remain auditable indefinitely, even after the account is no longer usable. Terminating active sessions immediately is necessary because any session in flight at deactivation time was authenticated under the about-to-be-revoked credentials and must be invalidated to honor the access decision. Prohibiting self-deactivation closes a denial-of-service vector against the Administrator workforce (an Administrator deactivating themselves and leaving no other active Administrator could lock out the deployment). The free-text reason is captured to support audit reviews of why each account was deactivated.

End Deactivate User Account | **Hash:** 6f0247b3

DIARY-PRD-user-account-reactivate: Reactivate User Account

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-user-account-deactivate

Assertions

- A. The System SHALL support reactivation of any deactivated **User Account**.
- B. When a **User Account** is reactivated, the system SHALL restore all previously assigned roles and **Site** assignments.
- C. Upon reactivation, the status of the **User Account** SHALL be set to **Pending Activation** state.
- D. When an **Administrator** reactivates a **User Account**, the **System** SHALL require a free text reason before applying the change. The reason SHALL be captured in the audit trail.

Rationale

Reactivation is the inverse of deactivation and serves the case where a previously off-boarded user resumes a role on the study — a returning Study Coordinator, an Administrator restored after a leave of absence. Restoring previously assigned roles and **Site** assignments is the operational expectation: the reactivation event is logically “this same person, same responsibilities, again”, not a fresh account creation. Routing the reactivated account through **Pending Activation** rather than directly to **Active** is a security floor: the user must complete a fresh activation flow, which proves they still control the registered email address and produces a current credential (password, 2FA) rather than reviving stale credentials that may have been compromised during the deactivated interval. The free-text reason is captured for the same audit purpose as deactivation.

End Reactivate User Account | **Hash:** c36b760f

DIARY-PRD-reason-field-constraints: Reason Field Constraints

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Where a reason is required before an action proceeds, consistent input constraints ensure reasons are meaningful and auditable. Empty or whitespace-only submissions are rejected to ensure every high-impact action is accompanied by a documented reason for compliance purposes. This requirement is the platform-wide template for free-text reason inputs and is referenced by every Reason Dialog (Free Text) defined elsewhere in the specification.

Assertions

- A. Where the **System** requires a reason, the **System** SHALL reject a submission where the reason is empty or contains only whitespace.
- B. Where the **System** requires a reason, the **System** SHALL enforce a maximum input length of 100 characters.

Rationale

Free-text reason inputs accompany high-impact, irreversible actions (deactivation, disconnection, mark-as-not-participating, record deletion) and are the principal narrative artifact the audit trail carries forward for those actions. Two failure modes degrade their audit value: empty or whitespace-only submissions (which let users bypass the reason requirement while passing the input control) and unbounded length (which lets users paste pages of irrelevant text that obscure the actual rationale). The whitespace rejection rule eliminates the former; the 100-character cap addresses the latter while still leaving room for a usefully descriptive sentence. Both constraints are platform-wide so reviewers see consistent reason-field behavior across every action that requires one, and so the audit log's free-text field has predictable bounds.

End Reason Field Constraints | **Hash:** 602a83b1

DIARY-GUI-user-management-tabs: User Management Tabs

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-user-account-create, DIARY-PRD-user-account-edit

Overview

The user management interface separates active and inactive accounts into distinct tabs to reduce cognitive load and prevent accidental actions on the wrong account. Real-time search and preserved search state allow Administrators to work efficiently across tabs without losing context.

Definitions

User Information Modal: A modal dialog displaying the details and available actions for a selected **User Account**.

Assertions

Tab Display

- A. The interface SHALL display two tabs: Active Users and Inactive Users.
- B. The Active Users tab SHALL display all User Accounts with a status of Active or Pending Activation.
- C. The Inactive Users tab SHALL display all User Accounts with a status of Deactivated.
- D. The interface SHALL display the Active Users tab by default.
- E. The interface SHALL highlight the currently active tab.
- F. Each tab SHALL display the total count of User Accounts it contains inline with the tab label.
- G. When a User Account status changes, the interface SHALL move the account to the appropriate tab immediately.

Search Behavior

- H. The interface SHALL provide a single search input that filters User Accounts by Full Name and Email Address.
- I. The interface SHALL update search results in real time as the user types.
- J. The interface SHALL preserve the search state when the user switches between tabs.

Empty State

- K. When a tab contains no User Accounts matching the current search, the interface SHALL display a message indicating no results were found.

Row Display

- L. The interface SHALL display each **User Account** as a row showing **Full Name**, **Email Address**, **Role(s)**, **Assigned Sites**, and **Status**.
- M. Each **User Account** row SHALL be selectable and open the **User Information Modal**.
- N. The interface SHALL display a **Create User** action.

Rationale

Active and inactive accounts are operationally distinct surfaces: actions available on one are not available on the other, and the most common Administrator error in unified lists is acting on an inactive account believing it was active. Splitting into two tabs makes the active/inactive distinction structural rather than visual, and immediate row-movement on status change keeps each tab self-consistent without manual refresh. Real-time search across **Full Name** and **Email Address** matches the two identifiers Administrators use in practice (a Full Name supplied by someone reporting an issue, or an Email Address supplied by the user themselves). Preserving search state across tabs supports the recurring workflow of "search for a user, check Active, then check Inactive" without forcing the Administrator to retype the query. Row selection opens the **User Information Modal** rather than navigating away, so the table context remains visible behind the modal and consecutive lookups are quick.

End User Management Tabs | **Hash:** c1ab4c55

DIARY-GUI-user-account-deactivate: Deactivate User Account

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-user-account-deactivate

Overview

The deactivation action is available from the Active Users tab only, since deactivated accounts are no longer present there after the action completes.

Assertions

Availability

A. The interface SHALL make the deactivation action available for every **User Account** displayed in the **Active Users** tab, with the exception of the current user's own account.

Interaction Flow

B. When the deactivation action is initiated, the interface SHALL display the confirmation dialog and require a free text reason before proceeding.

C. The interface SHALL not allow the deactivation action to proceed until a reason has been provided.

D. If the user cancels the confirmation, the **User Account** will remain unchanged.

E. When deactivation is confirmed, the interface SHALL move the **User Account** to the **Inactive Users** tab immediately.

Rationale

The deactivation surface is anchored to the **Active Users** tab because that is where the Administrator finds candidates for deactivation; offering the same action from the Inactive Users tab would be either a no-op (the account is already deactivated) or a confusing alternative entry point. Excluding the Administrator's own account from the action list closes the self-deactivation channel from the GUI side, complementing the PRD-level prohibition and making the unavailable state visible rather than producing a back-end rejection after the Administrator has invested in the workflow. Requiring the reason at the confirmation step rather than after the action commits gives the Administrator a final opportunity to back out and ensures the reason is captured before any state change is applied.

End Deactivate User Account | **Hash:** 864d7776

DIARY-GUI-user-account-reactivate: Reactivate User Account

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-user-account-reactivate

Overview

Reactivation is initiated from the Inactive Users tab and immediately returns the account to the Active Users tab with **Pending Activation** status, requiring the user to complete the activation workflow before regaining access.

Assertions

Availability

A. The interface SHALL make the reactivation action available for every **User Account** displayed in the **Inactive Users** tab.

Interaction Flow

B. When the reactivation action is initiated, the interface SHALL display the confirmation dialog and require a free text reason before proceeding.

C. The interface SHALL NOT allow the reactivation action to proceed until a reason has been provided.

D. If the user cancels the confirmation, the **User Account** will remain unchanged.

E. When reactivation is confirmed, the interface SHALL move the **User Account** to the **Active Users** tab immediately with a status of **Pending Activation**.

Rationale

Reactivation is anchored to the **Inactive Users** tab to mirror the deactivation pattern: the candidate set lives in the inactive tab, and the action there is the only one available. Moving the account to the **Active Users** tab with **Pending Activation** status on confirmation makes the next-step responsibility (the user must complete the activation flow before logging in) immediately visible — the account is back in the active surface and its status badge communicates that login is gated on activation. The free-text reason is gathered at the same confirmation step as deactivation for symmetry and to keep the audit trail's reason fields populated for every reactivation event.

End Reactivate User Account | **Hash:** f9ae8a4f

DIARY-GUI-user-information-modal: User Information Modal

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-user-account-create, DIARY-PRD-user-account-edit

Overview

The **User Information Modal** provides a summary view of a **User Account** and the primary actions available for that account. All account management actions are initiated from this modal.

Assertions

Display

- A. The **User Information Modal** SHALL display the **Full Name**, **Email Address**, **Status**, assigned **Role(s)**, and **Site** assignments of the selected **User Account**.
- B. The **User Information Modal** SHALL display every **Role** assigned to the **User Account**.
- C. When the **User Account** is an **Administrator**, the interface SHALL indicate that the **User Account** has access to all **Sites**.
- D. When the **User Account** holds one or more non-**Administrator Roles**, the interface SHALL allow the viewer to select a single **Role** to scope the **Site** list.
- E. When a **Role** is selected, the interface SHALL display only the **Site(s)** assigned to the **User Account** under that **Role**.
- F. The interface SHALL display the total count of **Sites** assigned to the selected **Role**.
- G. The interface SHALL display each **Site** entry using the **Site** identifier and **Site** name.
- H. The interface SHALL indicate which **Role** is currently selected.

Actions

- I. The **User Information Modal** SHALL present an **Edit User** action, a **Deactivate User** action, and a **Close** action for accounts displayed in the **Active Users** tab.
- J. The **User Information Modal** SHALL present a **Reactivate User** action and a **Close** action for accounts displayed in the **Inactive Users** tab.
- K. The **User Information Modal** SHALL NOT present a **Deactivate User** action for the currently authenticated user's own account.
- L. When the user selects **Close**, the interface SHALL dismiss the modal without making any changes.
- M. When the user selects **Edit User**, the interface SHALL open the edit workflow for that **User Account**.

Rationale

The **User Information Modal** is the per-account hub: every lifecycle action (edit, deactivate, reactivate) launches from here, and every account-related question (who, what role, which Sites) is answered here. Scoping the Site list by Role addresses the

multi-role case directly — a user holding both Study Coordinator and CRA roles on overlapping but distinct Site sets is shown the correct subset for whichever role context the Administrator is reasoning about. Surfacing **Sites** by Role rather than as a flat union prevents the inverse error in which an Administrator believes a user has Study Coordinator access at a Site that they actually have only as a CRA. Suppressing **Deactivate User** on the current user's own account mirrors the PRD-level self-deactivation prohibition and the GUI-level Active Users tab rule, so the unavailable state is consistent across every surface a self-deactivation attempt could originate from.

End User Information Modal | **Hash:** 3a130783

DIARY-GUI-administrator-dashboard: Administrator Dashboard

Level: GUI | **Status:** Legacy | **Implements:** -

Overview

The **Administrator Dashboard** is the primary surface for an **Administrator** to manage **User Accounts** and review audit log activity. Organising the surface as a container with two top-level tabs separates **User Account** management from audit log review while keeping both areas reachable from a single entry point. The dashboard scaffolding defined here establishes the header, the top-level tab navigation, and the default landing tab; the contents of each tab are governed by the requirements referenced from those tabs.

Definitions

Administrator Dashboard: The default surface presented to an **Administrator** upon successful authentication to the **Sponsor Portal**, containing top-level navigation to **User Account** management and audit log review.

Assertions

Header

- A. The interface SHALL display a header at the top of the **Administrator Dashboard** containing the **Sponsor Portal** name, the **Full Name** of the authenticated user, the user's active **Role**, a **Settings** action, and a **Logout** action.
- B. The header SHALL remain visible on every screen of the **Administrator Dashboard**.
- C. When the **Administrator** selects **Settings**, the interface SHALL navigate to the **Administrator Settings** surface defined in DIARY-GUI-administrator-settings.

Top-Level Tabs

D. The **Administrator Dashboard** SHALL display two top-level tabs: **Users** and **Audit Logs**.

E. The **Users** tab SHALL display the User Management interface defined in DIARY-GUI-user-management-tabs.

F. The **Audit Logs** tab SHALL display the Administrator Audit Log View defined in DIARY-GUI-audit-log-administrator.

G. The interface SHALL display the **Users** tab by default upon login.

H. The interface SHALL display a visual indicator identifying the currently active top-level tab.

Logout

I. When the **Administrator** selects **Logout**, the **System** SHALL terminate the **Session** and return the **Administrator** to the **Sponsor Portal** login interface.

Rationale

The **Administrator Dashboard** consolidates the two responsibilities of the **Administrator** role — **User Account** management and audit review — into adjacent tabs of a single surface so the Administrator can switch between investigating an account and reviewing the audit log without navigating away from a shared context. A persistent header keeps identity (who is logged in, in what role), system context (which Sponsor Portal), settings access, and logout reachable from any screen of the dashboard, which is necessary because an Administrator may need to log out from any state without first navigating back to a landing page. Defaulting to the **Users** tab on login matches the most common Administrator entry point (open the dashboard to perform account actions); the Audit Logs tab is one click away when investigation is the goal.

End Administrator Dashboard | **Hash:** 8f0fa6a0

1.24 DIARY-PRD-data-backup-and-archival: Data Backup and Archival

Level: PRD | **Status:** Legacy | **Refines:** DIARY-PRD-platform-operations-monitoring-G

Rationale

Clinical trial data must be protected and retained for extended periods per FDA regulations. FDA 21 CFR Part 11 requires electronic records to remain accessible throughout their retention period, which typically extends 7+ years for clinical trials. Backup systems ensure data survivability and business continuity in disaster scenarios, while archival systems enable long-term regulatory compliance and support potential future regulatory audits. Geographic redundancy provides resilience against site-level failures, though geographic placement must align with data residency requirements (such as GDPR for EU-based trials). Sponsor isolation in backup storage ensures multi-tenant data segregation principles extend to disaster recovery systems.

Assertions

- A. The system SHALL perform automated database backups at defined frequencies without requiring manual intervention.
- B. The system SHALL store backups in geographically separate locations to enable disaster recovery.
- C. The system SHALL provide point-in-time recovery capability for database restoration.
- D. The system SHALL retain archived data for a minimum of 7 years to meet regulatory compliance requirements.
- E. The system SHALL verify backup integrity using cryptographic checksums or equivalent mechanisms.
- F. The system SHALL isolate each sponsor's backup storage from other sponsors' backups.
- G. The system SHALL maintain archived data in an accessible format for regulatory audits throughout the retention period.
- H. The system SHALL document and test recovery procedures on a quarterly basis.

End Data Backup and Archival | **Hash:** 4e9501e4

1.25 DIARY-PRD-ancillary-platform-services: Ancillary Platform Services

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-platform-operations-monitoring-A+C

Rationale

Core platform functionality requires supporting services for communication, reporting, and integration. These services enable patient engagement through notifications, support staff workflows through reports, and provide sponsor data access through exports and APIs while maintaining security and compliance. The ancillary services complement the core clinical trial diary functionality by handling cross-cutting concerns such as device communications, email delivery, document generation, data interchange, and third-party system integration.

Assertions

- A. The platform SHALL provide push notification services to mobile devices.
- B. Push notifications SHALL be delivered reliably to mobile devices.
- C. The platform SHALL provide email notification services for staff and patients.
- D. Email delivery SHALL include an audit trail.
- E. The platform SHALL provide report generation capabilities.
- F. Reports SHALL be generated on demand.
- G. Reports SHALL be generated on scheduled basis.
- H. The platform SHALL provide report distribution capabilities.
- I. The platform SHALL provide data export capabilities for analysis.
- J. Data exports SHALL be provided in standard formats.
- K. Data exports SHALL include access controls.
- L. The platform SHALL provide integration APIs for sponsor systems.
- M. Integration APIs SHALL be secured.
- N. Integration APIs SHALL be documented.

End Ancillary Platform Services | **Hash:** ff326529

Operational Requirements

Development Requirements

Topic Index

account: [DIARY-GUI-administrator-dashboard](#), [DIARY-GUI-user-account-deactivate](#), [DIARY-GUI-user-account-reactivate](#), [DIARY-GUI-user-information-modal](#), [DIARY-GUI-user-management-tabs](#), [DIARY-PRD-reason-field-constraints](#), [DIARY-PRD-user-account-activation-resend](#), [DIARY-PRD-user-account-activation-workflow](#), [DIARY-PRD-user-account-create](#), [DIARY-PRD-user-account-deactivate](#), [DIARY-PRD-user-account-edit](#), [DIARY-PRD-user-account-reactivate](#), [DIARY-PRD-user-account-site-assignment](#)

administrator: [CAL-PRD-administrator-settings-inventory](#), [DIARY-GUI-administrator-settings](#), [DIARY-PRD-administrator-settings](#)

app: [CAL-PRD-application-lock-configuration](#), [CAL-PRD-privacy-policy-configuration](#), [DIARY-PRD-application-lock](#), [DIARY-PRD-diary-start-day](#), [DIARY-PRD-mobile-application](#), [DIARY-PRD-mobile-offline-first](#), [DIARY-PRD-privacy-policy](#)

audit: [CAL-PRD-audit-log-view-scope](#), [DIARY-GUI-audit-log-administrator](#), [DIARY-GUI-audit-log-common](#), [DIARY-GUI-audit-log-cra](#), [DIARY-GUI-audit-log-study-coordinator](#)

auth: [CAL-PRD-login-rate-limiting-configuration](#), [CAL-PRD-two-factor-authentication-configuration](#), [DIARY-GUI-password-forgot-workflow](#), [DIARY-PRD-password-forgot](#), [DIARY-PRD-password-requirements](#), [DIARY-PRD-session-management](#), [DIARY-PRD-two-factor-authentication](#)

backup: [DIARY-PRD-data-backup-and-archival](#)

behavior: [DIARY-PRD-notification-behavior](#)

calculation: [DIARY-PRD-questionnaire-score-calculation](#), [DIARY-PRD-score-hht-qol](#), [DIARY-PRD-score-nose-hht](#)

ci: [CAL-OPS-ci-change-appropriate-validation](#), [CAL-OPS-ci-code-quality-static-analysis](#), [CAL-OPS-ci-secret-vulnerability-scanning](#), [CAL-OPS-ci-traceability-pipeline](#)

configuration: [DIARY-PRD-configuration-precedence](#)

data: [DIARY-PRD-data-backup-and-archival](#)

database: [CAL-DEV-database-minimize-deleted-records](#), [CAL-DEV-database-no-implicit-field-changes](#), [CAL-DEV-database-proxy-purpose](#), [CAL-DEV-database-survey-response-storage](#)

deployment: [CAL-OPS-deploy-audit-trail](#), [CAL-OPS-deploy-cloud-run-verification](#), [CAL-OPS-deploy-concurrency-locks](#), [CAL-OPS-deploy-cross-repo-image-trigger](#), [CAL-OPS-deploy-notification](#), [CAL-OPS-deploy-notification-dev](#), [CAL-OPS-deploy-notification-prod](#), [CAL-OPS-deploy-notification-qa](#), [CAL-OPS-deploy-notification-uat](#), [CAL-OPS-deploy-permission](#), [CAL-OPS-deploy-permission-dev](#), [CAL-OPS-deploy-permission-prod](#), [CAL-OPS-deploy-permission-qa](#), [CAL-OPS-deploy-permission-uat](#), [CAL-OPS-deploy-provenance-traceability](#), [CAL-OPS-deploy-rollback-canary](#), [CAL-OPS-deploy-runbooks-failure-modes](#), [CAL-OPS-secrets-doppler-cloud-run](#), [CAL-OPS-secrets-github-actions-routing](#), [CAL-OPS-secrets-sponsor-agnostic-automation](#)

device: CAL-PRD-linking-code-entry-errors-configuration, DIARY-GUI-join-study-screen, DIARY-GUI-linking-confirmation, DIARY-PRD-linking-code-entry-errors

diary: CAL-PRD-entry-duration-check-configuration, CAL-PRD-entry-time-restrictions-configuration, DIARY-GUI-entry-overlap-resolution, DIARY-PRD-entry-duration-check, DIARY-PRD-entry-overlap-resolution, DIARY-PRD-entry-time-restrictions

display: CAL-PRD-questionnaire-score-display-prohibition

edc: CAL-DEV-edc-audit-context-structure, CAL-DEV-edc-audit-location-format, CAL-DEV-edc-audit-reason-values, CAL-DEV-edc-audit-timestamp-format, CAL-DEV-edc-audit-user-values, CAL-DEV-edc-callisto-transforms, CAL-DEV-edc-change-history-via-eventstore, CAL-DEV-edc-data-source-trace, CAL-DEV-edc-date-field-format, CAL-DEV-edc-duration-calculation, CAL-DEV-edc-incomplete-entry-flag, CAL-DEV-edc-no-nosebleeds-flag, CAL-DEV-edc-notes-field-handling, CAL-DEV-edc-patient-id-format, CAL-DEV-edc-requester-id, CAL-DEV-edc-requester-name, CAL-DEV-edc-severity-field-format, CAL-DEV-edc-site-id-format, CAL-DEV-edc-survey-name, CAL-DEV-edc-survey-response-storage, CAL-DEV-edc-survey-version, CAL-DEV-edc-time-field-format, CAL-DEV-edc-unknown-event-flag, CAL-DEV-edc-uuid-field-format, CAL-DEV-edc-uuid-table-prefix

entry: CAL-PRD-entry-duration-check-configuration, CAL-PRD-entry-time-restrictions-configuration, DIARY-GUI-entry-overlap-resolution, DIARY-PRD-entry-duration-check, DIARY-PRD-entry-overlap-resolution, DIARY-PRD-entry-time-restrictions

epistaxis: DIARY-GUI-epistaxis-delete, DIARY-GUI-epistaxis-record, DIARY-PRD-epistaxis-capture-standard

evidence: DIARY-PRD-evidence-bitcoin-timestamp, DIARY-PRD-evidence-device-fingerprint, DIARY-PRD-evidence-email-identity, DIARY-PRD-evidence-geolocation, DIARY-PRD-evidence-patient-authentication, DIARY-PRD-evidence-timestamp-archival, DIARY-PRD-evidence-timestamp-attestation, DIARY-PRD-evidence-timestamp-verification

help: DIARY-PRD-help-resources

linking: CAL-PRD-linking-code-entry-errors-configuration, DIARY-GUI-join-study-screen, DIARY-GUI-linking-confirmation, DIARY-PRD-linking-code-entry-errors

log: CAL-PRD-audit-log-view-scope, DIARY-GUI-audit-log-administrator, DIARY-GUI-audit-log-common, DIARY-GUI-audit-log-cra, DIARY-GUI-audit-log-study-coordinator

management: CAL-GUI-manage-questionnaires-modal, CAL-GUI-questionnaire-finalization-workflow, CAL-GUI-trial-start-workflow, CAL-PRD-nose-hht-qol-workflow, CAL-PRD-questionnaire-cycle-tracking, CAL-PRD-questionnaire-session-resume-timeout, CAL-PRD-trial-start-workflow, DIARY-PRD-architecture-decision-docs, DIARY-PRD-questionnaire-change-control, DIARY-PRD-questionnaire-sponsor-configuration, DIARY-PRD-questionnaire-submission-control, DIARY-PRD-questionnaire-system, DIARY-PRD-system-validation-traceability

mobile: CAL-PRD-application-lock-configuration, CAL-PRD-notification-historical-gap-configuration, CAL-PRD-notification-incomplete-record-lock-configuration, CAL-PRD-notification-reminder-configuration, CAL-PRD-privacy-policy-configuration, DIARY-GUI-calendar-day-view, DIARY-GUI-main-screen-layout, DIARY-GUI-mobile-navigation, DIARY-GUI-participant-task-list, DIARY-GUI-participation-status-badge, DIARY-PRD-application-lock, DIARY-PRD-diary-start-day, DIARY-PRD-mobile-application, DIARY-PRD-mobile-offline-first, DIARY-PRD-notification-disconnection, DIARY-PRD-notification-historical-gap, DIARY-

[PRD-notification-incomplete-record-lock](#), [DIARY-PRD-notification-ongoing-epistaxis](#), [DIARY-PRD-notification-portal-sent-questionnaire](#), [DIARY-PRD-notification-yesterday-entry](#), [DIARY-PRD-privacy-policy](#)

navigation: [DIARY-GUI-calendar-day-view](#), [DIARY-GUI-main-screen-layout](#), [DIARY-GUI-mobile-navigation](#)

notification: [DIARY-PRD-notification-behavior](#)

notifications: [CAL-DEV-push-notifications-fcm-delivery](#), [CAL-PRD-notification-historical-gap-configuration](#), [CAL-PRD-notification-incomplete-record-lock-configuration](#), [CAL-PRD-notification-reminder-configuration](#), [DIARY-GUI-participant-task-list](#), [DIARY-GUI-participation-status-badge](#), [DIARY-PRD-notification-disconnection](#), [DIARY-PRD-notification-historical-gap](#), [DIARY-PRD-notification-incomplete-record-lock](#), [DIARY-PRD-notification-ongoing-epistaxis](#), [DIARY-PRD-notification-portal-sent-questionnaire](#), [DIARY-PRD-notification-yesterday-entry](#)

operations: [DIARY-PRD-platform-operations-monitoring](#)

overview: [DIARY-PRD-questionnaire-daily-epistaxis](#), [DIARY-PRD-questionnaire-hht-qol](#), [DIARY-PRD-questionnaire-nose-hht](#)

participant: [CAL-GUI-participant-dashboard-configuration](#), [CAL-GUI-questionnaire-call-back-participant](#), [CAL-PRD-linking-code-lifecycle-configuration](#), [CAL-PRD-participant-disconnection-reason-options](#), [CAL-PRD-participant-not-participating-reason-options](#), [CAL-PRD-participant-registration-configuration](#), [DIARY-GUI-participant-dashboard](#), [DIARY-GUI-questionnaire-portal-sent-workflow](#), [DIARY-GUI-questionnaire-session-expiry](#), [DIARY-PRD-linking-code-lifecycle](#), [DIARY-PRD-participant-disconnection](#), [DIARY-PRD-participant-link-new](#), [DIARY-PRD-participant-mark-not-participating](#), [DIARY-PRD-participant-reactivate](#), [DIARY-PRD-participant-reconnection](#), [DIARY-PRD-participant-registration](#), [DIARY-PRD-questionnaire-portal-sent-rules](#), [DIARY-PRD-questionnaire-session-timeout](#)

platform: [DIARY-PRD-ancillary-platform-services](#), [DIARY-PRD-platform-operations-monitoring](#)

portal: [CAL-PRD-login-rate-limiting-configuration](#), [CAL-PRD-two-factor-authentication-configuration](#), [DIARY-GUI-password-forgot-workflow](#), [DIARY-PRD-password-forgot](#), [DIARY-PRD-password-requirements](#), [DIARY-PRD-session-management](#), [DIARY-PRD-two-factor-authentication](#)

push: [CAL-DEV-push-notifications-fcm-delivery](#)

questionnaire: [CAL-GUI-manage-questionnaires-modal](#), [CAL-GUI-questionnaire-call-back-participant](#), [CAL-GUI-questionnaire-finalization-workflow](#), [CAL-GUI-trial-start-workflow](#), [CAL-PRD-nose-hht-qol-workflow](#), [CAL-PRD-questionnaire-cycle-tracking](#), [CAL-PRD-questionnaire-score-display-prohibition](#), [CAL-PRD-questionnaire-session-resume-timeout](#), [CAL-PRD-trial-start-workflow](#), [DIARY-GUI-questionnaire-portal-sent-workflow](#), [DIARY-GUI-questionnaire-session-expiry](#), [DIARY-PRD-questionnaire-change-control](#), [DIARY-PRD-questionnaire-daily-epistaxis](#), [DIARY-PRD-questionnaire-hht-qol](#), [DIARY-PRD-questionnaire-localization](#), [DIARY-PRD-questionnaire-nose-hht](#), [DIARY-PRD-questionnaire-portal-sent-rules](#), [DIARY-PRD-questionnaire-session-timeout](#), [DIARY-PRD-questionnaire-sponsor-configuration](#), [DIARY-PRD-questionnaire-sponsor-eligibility](#), [DIARY-PRD-questionnaire-submission-control](#), [DIARY-PRD-questionnaire-system](#), [DIARY-PRD-questionnaire-versioning](#)

rbac: CAL-PRD-permissions-table, CAL-PRD-site-view-scope, DIARY-GUI-role-switching, DIARY-PRD-action-inventory, DIARY-PRD-rbac-customizable, DIARY-PRD-role-definitions

records: DIARY-PRD-evidence-bitcoin-timestamp, DIARY-PRD-evidence-device-fingerprint, DIARY-PRD-evidence-email-identity, DIARY-PRD-evidence-geolocation, DIARY-PRD-evidence-patient-authentication, DIARY-PRD-evidence-timestamp-archival, DIARY-PRD-evidence-timestamp-attestation, DIARY-PRD-evidence-timestamp-verification

requirements: DIARY-PRD-architecture-decision-docs, DIARY-PRD-system-validation-traceability

resources: DIARY-PRD-help-resources

rules: CAL-PRD-entry-duration-check-configuration, CAL-PRD-entry-time-restrictions-configuration, DIARY-GUI-entry-overlap-resolution, DIARY-PRD-entry-duration-check, DIARY-PRD-entry-overlap-resolution, DIARY-PRD-entry-time-restrictions

schema: CAL-DEV-edc-audit-context-structure, CAL-DEV-edc-audit-location-format, CAL-DEV-edc-audit-reason-values, CAL-DEV-edc-audit-timestamp-format, CAL-DEV-edc-audit-user-values, CAL-DEV-edc-callisto-transforms, CAL-DEV-edc-change-history-via-eventstore, CAL-DEV-edc-data-source-trace, CAL-DEV-edc-date-field-format, CAL-DEV-edc-duration-calculation, CAL-DEV-edc-incomplete-entry-flag, CAL-DEV-edc-no-nosebleeds-flag, CAL-DEV-edc-notes-field-handling, CAL-DEV-edc-patient-id-format, CAL-DEV-edc-requester-id, CAL-DEV-edc-requester-name, CAL-DEV-edc-severity-field-format, CAL-DEV-edc-site-id-format, CAL-DEV-edc-survey-name, CAL-DEV-edc-survey-response-storage, CAL-DEV-edc-survey-version, CAL-DEV-edc-time-field-format, CAL-DEV-edc-unknown-event-flag, CAL-DEV-edc-uuid-field-format, CAL-DEV-edc-uuid-table-prefix

score: CAL-PRD-questionnaire-score-display-prohibition, DIARY-PRD-questionnaire-score-calculation, DIARY-PRD-score-hht-qol, DIARY-PRD-score-nose-hht

secrets: CAL-OPS-secrets-doppler-cloud-run, CAL-OPS-secrets-github-actions-routing, CAL-OPS-secrets-sponsor-agnostic-automation

services: DIARY-PRD-ancillary-platform-services

settings: CAL-PRD-administrator-settings-inventory, DIARY-GUI-administrator-settings, DIARY-PRD-administrator-settings

sla: DIARY-PRD-sla-chronic-failure-escalation, DIARY-PRD-sla-corrective-preventive, DIARY-PRD-sla-customer-notification, DIARY-PRD-sla-data-recovery, DIARY-PRD-sla-disaster-recovery, DIARY-PRD-sla-incident-response-times, DIARY-PRD-sla-incident-severity, DIARY-PRD-sla-regulatory-event-support, DIARY-PRD-sla-root-cause-analysis, DIARY-PRD-sla-service-availability

user: DIARY-GUI-administrator-dashboard, DIARY-GUI-user-account-deactivate, DIARY-GUI-user-account-reactivate, DIARY-GUI-user-information-modal, DIARY-GUI-user-management-tabs, DIARY-PRD-reason-field-constraints, DIARY-PRD-user-account-activation-resend, DIARY-PRD-user-account-activation-workflow, DIARY-PRD-user-account-create, DIARY-PRD-user-account-deactivate, DIARY-PRD-user-account-edit, DIARY-PRD-user-account-reactivate, DIARY-PRD-user-account-site-assignment

validation: CAL-OPS-ci-change-appropriate-validation, CAL-OPS-ci-code-quality-static-analysis, CAL-OPS-ci-secret-vulnerability-scanning, CAL-OPS-ci-traceability-pipeline

versioning: [DIARY-PRD-questionnaire-localization](#), [DIARY-PRD-questionnaire-sponsor-eligibility](#), [DIARY-PRD-questionnaire-versioning](#)

workflow: [CAL-GUI-questionnaire-call-back-participant](#), [DIARY-GUI-questionnaire-portal-sent-workflow](#), [DIARY-GUI-questionnaire-session-expiry](#), [DIARY-PRD-questionnaire-portal-sent-rules](#), [DIARY-PRD-questionnaire-session-timeout](#)

eCOA URS — Callisto Sponsor Overlays

May 16, 2026

Contents

1	Product Requirements	2
1.1	Callisto Administrator Settings Overlay	3
1.2	CAL-PRD-audit-log-view-scope: Audit Log View Scope	6
1.3	Callisto Device Linking Overlays	8
1.4	Callisto Diary Entry Rules Overlays	10
1.5	Callisto Mobile Application Foundation Overlays	13
1.6	Callisto Participant Tasks and Notifications Overlays	15
1.7	Callisto Participant Workflow Overlays	18
1.8	Callisto Portal Authentication Overlays	24
1.9	Callisto Questionnaire Management Overlays	26
1.10	Callisto Participant Questionnaire Workflow Overlays	37
1.11	CAL-PRD-questionnaire-score-display-prohibition: Questionnaire Score Display Prohibition	39
1.12	Callisto RBAC Overlays	40
2	Operational Requirements	43
2.1	Callisto CI Validation Placeholders	44
2.2	Callisto Deployment Secrets	46
2.3	Callisto Deployment Operations	49
3	Development Requirements	63
3.1	Callisto Database Conventions	64
3.2	Callisto EDC Schema — Field Formats and Audit Conventions	68
3.3	CAL-DEV-push-notifications-fcm-delivery: Push Notification Delivery via FCM	85
4	Topic Index	87

Product Requirements

1.1 Callisto Administrator Settings Overlay

This file holds the Callisto sponsor overlay for the **Administrator Settings** surface: the inventory of **Configuration Parameters** surfaced for this deployment, the **Configuration Categories** that group them, and the source requirement that establishes each value. The platform-side **Administrator Settings** surface and interface (DIARY-PRD-administrator-settings, DIARY-GUI-administrator-settings) live in `hht_diary/spec/prd-administrator-settings.md`.

CAL-PRD-administrator-settings-inventory: Administrator Settings Configuration Inventory

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-administrator-settings, DIARY-PRD-configuration-precedence

Overview

This requirement defines the **Configuration Parameters** surfaced in **Administrator Settings** for the Callisto **Sponsor Portal** deployment, the **Configuration Categories** under which they are grouped, and the source requirement establishing each value.

Assertions

- A. The **Administrator Settings** surface SHALL display the following **Configuration Categories** in the order listed: **Authentication and Sessions**, **Mobile Application Security**, **Diary Entry Rules**, **Questionnaire Sessions**, **Notifications and Reminders**, and **Rate Limiting**.
- B. The **Administrator Settings** surface SHALL display the following **Configuration Parameters** under their respective **Configuration Categories** with the current values shown:

Configuration Category	Parameter	Source Requirement
Authentication and Sessions	Sponsor Portal Session Idle Timeout	DIARY-PRD-session-management
Authentication and Sessions	Password Expiry Interval	DIARY-PRD-password-requirements
Authentication and Sessions	Two-Factor Authentication Code Expiry	CAL-PRD-two-factor-authentication-configuration
Mobile Application Security	Mobile Linking Code Expiry	CAL-PRD-linking-code-lifecycle-configuration

Configuration Category	Parameter	Source Requirement
Mobile Application Security	Application Lock Idle Timeout	CAL-PRD-application-lock-configuration
Mobile Application Security	Application Lock PIN Length	CAL-PRD-application-lock-configuration
Mobile Application Security	Application Lock Failed Attempt Threshold	CAL-PRD-application-lock-configuration
Diary Entry Rules	Justification Threshold	CAL-PRD-entry-time-restrictions-configuration
Diary Entry Rules	Lock Threshold	CAL-PRD-entry-time-restrictions-configuration
Diary Entry Rules	Short Duration Threshold	CAL-PRD-entry-duration-check-configuration
Diary Entry Rules	Long Duration Threshold	CAL-PRD-entry-duration-check-configuration
Questionnaire Sessions	NOSE HHT and HHT-QoL Session Timeout	CAL-PRD-questionnaire-session-resume-timeout
Questionnaire Sessions	NOSE HHT and HHT-QoL Timeout Warning Threshold	CAL-PRD-questionnaire-session-resume-timeout
Notifications and Reminders	Incomplete Record Lock Warning Offset	CAL-PRD-notification-incomplete-record-lock-configuration
Notifications and Reminders	Yesterday Entry Reminder Time	CAL-PRD-notification-reminder-configuration
Notifications and Reminders	Ongoing Epistaxis Event Reminder Interval	CAL-PRD-notification-reminder-configuration
Rate Limiting	Login Rate Limit Threshold	CAL-PRD-login-rate-limiting-configuration
Rate Limiting	Login Rate Limit Cooldown	CAL-PRD-login-rate-limiting-configuration
Rate Limiting	Linking Code Rate Limit Threshold	CAL-PRD-entry-time-restrictions-configuration
Rate Limiting	Linking Code Rate Limit Cooldown	CAL-PRD-entry-time-restrictions-configuration

Rationale

The configuration inventory is the per-deployment realisation of the platform-level **Administrator Settings** surface: each row binds a user-visible parameter name to the source requirement that establishes its current value, so the Administrator surface can render the current value by reading the named requirement rather than maintaining a parallel value store. Six categories were chosen to reflect the operational domains an Administrator reasons about — authentication, mobile security, entry validation, questionnaire timing, notifications, and rate limits — and the order from authentica-

tion outward to rate limiting follows the rough operational frequency with which an Administrator might consult each category. Cross-repo refines targets are used for the platform-side parents (session management, password rules, administrator settings, configuration precedence) and for the Callisto-side overlays in this and sibling files; binding the surface to the source requirements (rather than to a free-text value table) keeps the surface in sync with whatever values future change-control rounds adopt.

End Administrator Settings Configuration Inventory | **Hash:** bf0ea5e3

1.2 CAL-PRD-audit-log-view-scope: Audit Log View Scope

Level: prd | **Status:** Legacy | **Implements:** -

Overview

The System maintains an audit trail of all Actions performed by users. Each role requires a different scope of visibility into that audit trail to support its responsibilities while preserving separation of duties. **Study Coordinators** need visibility into their own Actions for self-verification and to support documentation requests during monitoring. CRAs need visibility into **Study Coordinator** Actions at their assigned Sites to perform monitoring and compliance verification. **Administrators** need visibility into **User Account** management Actions to support account lifecycle verification and troubleshooting.

Definitions

Audit Log Entry: A single immutable record capturing one **Action** performed in the System, including the **Action** performed, the user who performed it, the date and time, and any associated context.

Audit Log View: The interface presented to a user that displays a scoped set of Audit Log Entries the user is authorized to see.

Assertions

General

A. The System SHALL create an **Audit Log Entry** for every **Action** in the **Action Inventory** (DIARY-PRD-action-inventory for reference) that is performed in the System.

B. The Actions visible in any **Audit Log View** are drawn from the **Action Inventory**.

Study Coordinator Scope

C. The System SHALL provide each **Study Coordinator** with read-only access to the **Audit Log View**.

D. The System SHALL include in a **Study Coordinator's Audit Log View** only Audit Log Entries for Actions performed by that **Study Coordinator**.

CRA Scope

E. The System SHALL provide each **CRA** with an **Audit Log View** scoped to a single **Site** selected by the **CRA** from their assigned Sites.

F. The System SHALL include in a **CRA's Audit Log View** only Audit Log Entries for Actions performed by Study Coordinators at the selected **Site**.

G. The System SHALL exclude from a **CRA's Audit Log View** all Audit Log Entries for Sites not assigned to the **CRA**.

H. The System SHALL provide CRAs with read-only access to the **Audit Log View**.

Administrator Scope

I. The System SHALL provide each **Administrator** with read-only access to the **Audit Log View**.

J. The System SHALL include in an **Administrator's Audit Log View** all Audit Log Entries for **User Account** management Actions performed by any **Administrator**.

Rationale

Each portal role exercises a distinct slice of authority — Study Coordinators run participant workflows, CRAs monitor site compliance, Administrators manage User Accounts — and the audit log scope assigned to each role is shaped to its accountability surface. Study Coordinators see their own actions for self-verification and to support documentation requests during monitoring; expanding that scope to peer actions would erode separation of duties without serving any operational need. CRAs see Study Coordinator actions at a CRA-selected Site so that monitoring is targeted (one Site at a time) and bounded (the CRA cannot reach into Sites they are not assigned to); per-Site selection rather than per-CRA aggregation reflects how monitoring visits are actually scheduled. Administrators see all Administrator-on-User-Account actions because account lifecycle is the Administrator's accountability surface, and lateral visibility across Administrators is necessary to verify that role and Site assignments stay consistent across the team. Every role is read-only against the audit log because the log's evidentiary value depends on its immutability — modification surfaces do not exist for any role.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

End Audit Log View Scope | **Hash:** e43fc08c

1.3 Callisto Device Linking Overlays

This file holds the Callisto sponsor overlay for the device linking flow: the linking-code rate-limit values and error message text. The platform-side foundation (DIARY-PRD-linking-code-
DIARY-GUI-join-study-screen, DIARY-GUI-linking-confirmation) lives in `hht_diary/spec/prd` and is referenced cross-repo via Refines.

CAL-PRD-linking-code-entry-errors-configuration: Linking Code Entry Error Handling Configuration

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-linking-code-entry-errors

Overview

These values configure the platform linking code error handling mechanism defined in DIARY-PRD-linking-code-entry-errors.

Assertions

- A. The **Rate Limit Threshold** SHALL be configured to 10 attempts.
- B. The **Rate Limit Cooldown** SHALL be configured to 1 hour.
- C. The **Linking Code Validation Failure** error message SHALL display the text: "Invalid linking code."
- D. The **Rate Limit** error message SHALL display the text: "Too many attempts. Please wait an hour."

Rationale

The 10-attempt **Rate Limit Threshold** is calibrated for the linking-code surface — a longer typeable code than a password, where a participant making honest typos may go through several attempts on a single onboarding session before getting the code right. Setting the threshold below 10 (e.g. the Sponsor Portal's 5) would too often lock out legitimate participants on first-time linking; setting it much higher would give enumeration attempts useful headroom. The 1-hour **Rate Limit Cooldown** is long enough to make brute-force enumeration impractical at the platform's code-space (an attacker hitting the limit gets at most ~240 attempts/day against a single device, against a code space measured in millions) while keeping a realistic recovery window for a legitimate participant who has hit the limit through honest typos — they can return after lunch or after contacting their site for a fresh code. The generic error message "Invalid linking code." satisfies the parent REQ's no-information-leakage rule (the same text fires regardless of whether the code is unknown, expired, or used), and

the Rate Limit message names the specific delay so the participant knows when they may try again rather than retrying indefinitely.

End Linking Code Entry Error Handling Configuration | **Hash:** 859c5872

1.4 Callisto Diary Entry Rules Overlays

This file holds the Callisto sponsor overlays for the diary entry rules: the time-based entry restriction thresholds (Justification Threshold, Lock Threshold, Entry Justification list, prompt text) and the duration reasonableness check thresholds (Short Duration Threshold, Long Duration Threshold, prompt text). The platform-side foundation REQs (DIARY-PRD-entry-time-restrictions, DIARY-PRD-entry-duration-check, DIARY-PRD-entry-overlap-resolution) live in `hht_diary/spec/prd-diary-entry-rules.md` and are referenced cross-repo via Refines.

CAL-PRD-entry-time-restrictions-configuration: Time-Based Entry Restrictions Configuration

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-entry-time-restrictions

Overview

The Callisto clinical protocol requires that nosebleed event entries older than one calendar day include documented justification for the timing discrepancy, and that entries older than 48 hours are locked entirely to maintain data reliability. These values configure the platform mechanism defined in DIARY-PRD-entry-time-restrictions.

Assertions

- A. The **Justification Threshold** SHALL be configured to 24 hours.
- B. The **Lock Threshold** SHALL be configured to 48 hours.
- C. The **Entry Justification** predefined list SHALL contain exactly four options: "Entered from paper records," "Remembered specific event," "Estimated event," and "Other."
- D. The **Entry Justification** prompt SHALL display the text: "This is an event more than one day old. Please explain why you are adding/changing it now."

Rationale

The 24-hour **Justification Threshold** binds the soft "explain why this is late" boundary to one calendar day, matching the Yesterday Entry Reminder's daily-cadence prompt — a participant who has not entered yesterday's data within 24 hours has crossed into territory where the protocol requires a documented reason for the delay. The 48-hour **Lock Threshold** doubles the justification window, recognizing that some justifiable-late entries take an additional day to surface (a participant who was traveling or unwell at the 24-hour mark gets a further day to backfill with justification) while still capping the editable window at two days for reliability. The four-option predefined

list covers the realistic operational distribution of late-entry causes: transferring from paper records during onboarding, retrospective recall of a specific event, retrospective estimation when specifics are not recalled, and the residual “Other” for cases the three categories do not cover. The prompt text frames the obligation in plain language so the participant understands what they are being asked to explain.

End Time-Based Entry Restrictions Configuration | **Hash:** 58dc3e04

CAL-PRD-entry-duration-check-configuration: Duration Reasonableness Check Configuration

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-entry-duration-check

Overview

The Callisto clinical protocol defines specific duration boundaries for nosebleed events to flag potential data entry errors. These values configure the platform mechanism defined in DIARY-PRD-entry-duration-check.

Assertions

- A. The **Short Duration Threshold** SHALL be configured to 1 minute.
- B. The **Long Duration Threshold** SHALL be configured to 60 minutes.
- C. Both the short duration check and the long duration check SHALL be enabled.
- D. The short duration confirmation prompt SHALL display the text: “Duration is under 1 minute, is that correct?”
- E. The long duration confirmation prompt SHALL display the text: “Duration is over 1 hour, is that correct?”

Rationale

The 1-minute **Short Duration Threshold** catches the most likely short-side data-entry error — a participant who tapped Set End Time immediately after Set Start Time, producing a near-zero duration that almost certainly reflects a misclick rather than a real event. The 60-minute **Long Duration Threshold** flags the long-side counterpart — a duration over an hour is clinically possible for some HHT participants but unusual enough that a confirmation prompt is worthwhile against the much more common case of a wrong end-date or wrong-timezone entry producing a multi-hour or multi-day duration. Both checks are enabled for the Callisto deployment because the cost of an undetected outlier (a corrupted primary outcome measure) substantially exceeds the friction cost of a one-tap confirmation when a real outlier event occurs. The prompt text is phrased as a question (“is that correct?”) rather than a warning,

matching the soft-confirmation intent — the participant is invited to confirm or correct, not blocked.

End Duration Reasonableness Check Configuration | **Hash:** 9a298a25

1.5 Callisto Mobile Application Foundation Overlays

This file holds the Callisto sponsor overlays for the **Mobile Application** foundation: the **Clinical Trial Privacy Policy** URL binding and the **Application Lock** parameter values (Authentication Method, PIN length, Idle Timeout, Failed Attempt Threshold). The platform-side foundation REQs (DIARY-PRD-mobile-application, DIARY-PRD-mobile-offline, DIARY-PRD-diary-start-day, DIARY-PRD-privacy-policy, DIARY-PRD-application-lock) live in `hht_diary/spec/prd-mobile-app.md` and are referenced cross-repo via Refines.

CAL-PRD-privacy-policy-configuration: Clinical Trial Privacy Policy Configuration

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-privacy-policy

Assertions

A. The Clinical Trial Privacy Policy SHALL be made available at [URL placeholder].

Rationale

The Callisto deployment binds the platform-configurable **Clinical Trial Privacy Policy** to a single hosted URL. The URL placeholder will be replaced with the production policy location before deployment goes live; the obligation is that the policy is hosted at a stable, retrievable URL the **Mobile Application** can link to from the **Linking Consent** text on the **Join the Study** screen and from the **Participation Status Badge**. A platform-configurable URL (rather than embedded content) lets the legal text be updated independently of mobile-app release cycles, while the version-retention rule in the parent REQ ensures every participant's consent is anchored to the specific version they read at the time of consent.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

End Clinical Trial Privacy Policy Configuration | **Hash:** afe92245

CAL-PRD-application-lock-configuration: Application Lock Configuration

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-application-lock

Overview

These values configure the platform mechanism defined in DIARY-PRD-application-lock for the Callisto deployment.

Assertions

- A. The **Authentication Method** SHALL be configured to **PIN**.
- B. The **PIN** length SHALL be configured to 4 digits.
- C. The **Idle Timeout** SHALL be configured to 15 minutes.
- D. The **Failed Attempt Threshold** SHALL be configured to 5 consecutive failed attempts.

Rationale

The Callisto deployment selects **PIN** as the **Authentication Method** because it works uniformly across every iOS and Android device the participant population may carry, requires no device-level biometric or passcode configuration the participant might not have set up, and keeps the unlock UX consistent regardless of device capabilities. A 4-digit **PIN** is the everyday-credential length participants encounter elsewhere (bank cards, transit, app-level locks) and balances memorability against the brute-force surface, which is further bounded by the **Failed Attempt Threshold** fallback to **Device Authentication** after 5 attempts. The 15-minute **Idle Timeout** is long enough to cover routine interruptions (a participant putting the device down to answer a door, take a call, attend to a child) without forcing repeated re-authentication during an active diary session; it is short enough that an unattended device left in a public space cannot be picked up much later and still access participant data without authentication. The 5-attempt **Failed Attempt Threshold** matches the **Login Rate Limit Threshold** used on the Sponsor Portal, keeping the failed-attempt boundary consistent across the two authentication surfaces.

End Application Lock Configuration | **Hash:** 7c6cbaea

1.6 Callisto Participant Tasks and Notifications Overlays

This file holds the Callisto sponsor overlays for the participant notifications: the Lock Warning Offset and text for the Incomplete Record Lock Warning Notification, the Reminder Time and Reminder Schedule for the Yesterday Entry Reminder and Ongoing Epistaxis Event Reminder, and the Reminder Time and text for the Historical Gap Reminder. The platform-side foundation REQs (DIARY-PRD-notification-disconnection, DIARY-PRD-notification-incomplete-record-lock, DIARY-PRD-notification-portal-sent-q, DIARY-PRD-notification-yesterday-entry, DIARY-PRD-notification-ongoing-epistaxis, DIARY-PRD-notification-historical-gap) live in `hht_diary/spec/prd-mobile-notifications` and are referenced cross-repo via Refines.

Authoring note: the URS source assigns REQ-CAL-p00093 to **two distinct** §6.8 requirements (§6.8.9 *Notification & Reminder Configuration* and §6.8.11 *Historical Gap Reminder Configuration*). This is a defect in the URS source. The Phase-1.2 mapping splits them by scope into two distinct kebab REQ ids — CAL-PRD-notification-reminder-configuration (§6.8.9, the Yesterday + Ongoing Epistaxis configurations) and CAL-PRD-notification-historical-gap-configuration (§6.8.11, the Historical Gap configuration) — and that split is reflected here.

CAL-PRD-notification-incomplete-record-lock-configuration: Incomplete Record Lock Warning Notification Configuration

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-notification-incomplete-record-lock

Overview

The clinical protocol defines a 48-hour lock on diary entries (CAL-PRD-entry-time-restrictions-configuration). To give Participants a final opportunity to complete or delete an **Incomplete Record** before the lock fires, a warning notification is 24 hours prior. These values configure the platform mechanism defined in DIARY-PRD-notification-incomplete-record-lock.

Assertions

- A. The Lock Warning Offset SHALL be configured to 24 hours.
- B. The notification text SHALL be: "You have an incomplete nosebleed record from [date]. Complete or delete it within the next 24 hours, otherwise it will be permanently locked and you will not be able to change or remove it."

Rationale

The 24-hour Lock Warning Offset places the warning notification at the midpoint of the 48-hour Lock Threshold window (CAL-PRD-entry-time-restrictions-configuration) — early enough that the participant has a full day to act, late enough that the record has been in incomplete state long enough for the warning to be useful rather than premature. The notification text names the dated record explicitly so the participant knows which entry needs action, states the 24-hour deadline so the participant understands the urgency, and spells out the consequence (permanent lock, no further changes) so the action's importance is clear without medicalized hedging.

End Incomplete Record Lock Warning Notification Configuration | **Hash:** 7f7d231e

CAL-PRD-notification-reminder-configuration: Notification & Reminder Configuration

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-notification-ongoing-epistaxis, DIARY-PRD-notification-yesterday-entry

Assertions

A. The **Reminder Time** for the Yesterday Entry Reminder Notification SHALL be configured to 09:00 (24H format) in the **Participant's** local timezone.

B. The **Reminder Schedule** for the Ongoing Epistaxis Event Reminder SHALL be configured to deliver notifications at the following intervals: 5 minutes, 10 minutes, 15 minutes, 30 minutes.

Rationale

The 09:00 **Reminder Time** for the Yesterday Entry Reminder targets a morning-routine delivery window: the participant is likely awake, has finished the previous day, and is at a natural reflect-and-record point before the day's activities begin. A morning fire is more reliable than evening for daily-status capture because the participant has full memory of yesterday and the day is not yet competing for their attention. Evaluating the time against the participant's device local timezone (per the parent REQ) keeps the 09:00 delivery consistent regardless of participant travel. The Ongoing Epistaxis Event Reminder schedule (5/10/15/30 minutes) is a deliberate escalation: the first three reminders fire in the window where a typical nosebleed is still resolving (5-15 minutes) — most events end within this window, and the reminders help the participant remember to capture the end time as it happens. The 30-minute interval is the final attempt: if the participant has not responded by then, they are likely engaged with something else or the event has ended without active capture, and further reminders would saturate without informing. The four-interval cap respects the parent REQ's terminating-sequence rule.

End Notification & Reminder Configuration | **Hash:** dff44029

CAL-PRD-notification-historical-gap-configuration: Historical Gap Reminder Configuration

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-notification-historical-gap

Assertions

A. The **Reminder Time** for the Historical Gap Reminder SHALL be configured to 09:00 (24H format) in the **Participant's** local timezone.

B. The **Historical Gap Reminder** notification text SHALL be: "You have one day without a recorded entry. Tap to review and complete your diary."

Rationale

The 09:00 **Reminder Time** matches the Yesterday Entry Reminder time so the participant receives a single coherent daily-reminder cadence at the same hour rather than two notifications at unrelated times. The platform's once-per-day cap on each reminder type ensures the participant sees at most one Historical Gap Reminder per day even if multiple historical gaps exist; the notification text is therefore phrased to direct the participant to the diary (where they can address whichever gaps they choose) rather than naming any specific date. The "Tap to review and complete your diary" call-to-action sends the participant to the Calendar where they can see every gap visually and choose how to backfill.

End Historical Gap Reminder Configuration | **Hash:** 56b66860

1.7 Callisto Participant Workflow Overlays

This file holds the Callisto sponsor overlays for the participant workflow surfaces: the linking-code expiry value, the participant-registration source binding (**Rave EDC**), the disconnection reason list, the mark-as-not-participating reason list, and the participant dashboard configuration. The platform-side participant workflow foundation (linking-code lifecycle, registration, link/disconnect/reconnect/mark-not-participating/reactivate workflows, the **Participant Dashboard** GUI) lives in `hht_diary/spec/prd-participant.md` and is referenced cross-repo via Refines.

CAL-PRD-linking-code-lifecycle-configuration: Linking Code Lifecycle Configuration

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-linking-code-lifecycle

Assertions

A. The **Mobile Linking Code** expiry duration SHALL be configured to 72 hours from the time of generation.

Rationale

The 72-hour expiry window balances two operational realities: a **Participant** may receive their **Mobile Linking Code** from clinical staff at a site visit and then have several days before they next have the opportunity to install the **Mobile Application** and complete the linking process; and the longer a code remains valid the larger the window in which a leaked or photographed code could be misused. Three days is sufficient time for the common participant onboarding cadence (clinic visit on Monday, home setup by Thursday) without leaving the code valid for a full week, which would exceed any plausible legitimate use window.

End Linking Code Lifecycle Configuration | **Hash:** 522755f6

CAL-PRD-participant-registration-configuration: Participant Registration Configuration

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-participant-registration

Overview

The sponsor portal uses **Rave EDC** as the participant registration source. Near real-time synchronization ensures that participant records are available in the **Sponsor Portal** as soon as they are created in **Rave EDC**, without requiring manual intervention from clinical staff.

Definitions

Rave EDC: The electronic data capture system used by the portal as the authoritative source of participant and site records.

Participant ID: The unique identifier assigned to a **Participant** in **Rave EDC** and synchronized to the **Sponsor Portal**. The **Participant ID** is the primary reference used to identify a **Participant** across the System.

Assertions

- A. The System SHALL synchronize **Participant** records from **Rave EDC** to the **Sponsor Portal** in near real-time.
- B. The System SHALL synchronize the **Participant ID** and assigned **Site** for each **Participant** record.
- C. When a **Participant** record is synchronized from **Rave EDC** for the first time, the System SHALL assign an initial **Participant** status of **Not Connected**.
- D. The System SHALL NOT permit **Participant** records to be created, edited, or deleted manually in the **Sponsor Portal**.

Rationale

Rave EDC is the clinical system of record for the Callisto deployment, and binding the **Sponsor Portal**'s participant registration to it preserves the single-source-of-truth property: every **Participant** in the portal corresponds to a Rave EDC enrollment, and the **Participant ID** carries the same value across both systems. Near real-time synchronization removes the operational delay that would otherwise force clinical staff to wait for a manual sync after enrollment. Initial **Not Connected** status reflects the lifecycle reality that EDC enrollment precedes device linking by hours or days; the portal-side **Not Connected** state is what makes the participant a candidate for the **Link New Participant Workflow**. Prohibiting manual create/edit/delete in the portal closes the divergence risk: if portal-side authorship were allowed, the portal could hold records that **Rave EDC** does not, breaking the cross-system identity match.

End Participant Registration Configuration | **Hash**: 1abf676e

CAL-PRD-participant-disconnection-reason-options: Disconnection Reason Options

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-participant-disconnection

Assertions

- A. The disconnection reason SHALL be selected from a predefined list.
- B. The predefined disconnection reason options SHALL be: **Device Issues**, **Technical Issues**, and **Other**.

Rationale

The Callisto deployment elects the predefined-list option for disconnection reasons (over the free-text alternative the platform supports) because operational reporting at sponsor and site level benefits from a controlled vocabulary: trend analysis of disconnection causes requires categories that aggregate consistently across many disconnect events. The three categories cover the realistic distribution of disconnect causes — hardware faults (**Device Issues**), connectivity or software faults (**Technical Issues**), and the unavoidable residual (**Other**) — without forcing the Study Coordinator to guess between fine-grained distinctions that would not survive downstream aggregation anyway.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

End Disconnection Reason Options | **Hash:** af779b05

CAL-PRD-participant-not-participating-reason-options: Mark as Not Participating Reason Options

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-participant-mark-not-participating

Assertions

- A. The reason SHALL be selected from a predefined list.
- B. The predefined reason options SHALL be: **Subject Withdrawal**, **Death**, **Protocol Treatment/Study Complete**, and **Other**.

Rationale

End-of-trial reporting is sensitive to the categorical distinction between participant-driven exit (**Subject Withdrawal**), participant death (**Death**, which carries distinct safety-reporting implications), planned protocol completion (**Protocol Treatment/Study Complete**), and the residual case (**Other**). Encoding these as a predefined list ensures the audit trail and downstream analyses can aggregate exit reasons consistently and meet the reporting obligations Callisto's regulatory submissions depend on. The categories are exhaustive enough that **Other** should be rare; when it does occur, the audit trail's free-text reason narrative carries the specific explanation alongside the categorical tag.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

End Mark as Not Participating Reason Options | **Hash:** ad090ba1

CAL-GUI-participant-dashboard-configuration: Participant Dashboard Configuration

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-GUI-participant-dashboard

Overview

This requirement defines the Callisto-specific configuration of the **Participant Dashboard** — the tab set, the per-status action availability, and the Show Linking Code and Link Participant flows. The platform-side dashboard scaffolding (tabs, search, modal) is established by DIARY-GUI-participant-dashboard; this requirement specifies how the Callisto deployment fills it in.

Definitions

Participant Summary: The participant list section of the **Sponsor Portal** dashboard displaying all participants across the Study Coordinator's assigned sites.

Assertions

Tabs

A. The interface SHALL display three tabs: **Not Connected**, **Active**, and **Inactive**.

B. The **Not Connected** tab SHALL display participants with status **Not Connected**, **Pending**, or **Expired**.

C. The **Active** tab SHALL display participants with status **Linked - Awaiting Start** or **Trial Active**.

D. The **Inactive** tab SHALL display participants with status **Disconnected** or **Not Participating**.

E. The interface SHALL display the **Not Connected** tab by default.

Action Availability Table

F. The interface SHALL display the primary action in the Primary Action column and the secondary actions in the **Participant Actions Modal** according to the following table:

Status	Primary Action Column	Participant Actions Modal
Not Connected	Link Participant	Link Participant
Pending	Show Linking Code	Show Linking Code
Expired	Regenerate Code	Regenerate Code
Linked - Awaiting Start	Start Trial	Show Linking Code, Disconnect Participant
Trial Active	Manage Questionnaires	Show Linking Code, Disconnect Participant
Disconnected	Reconnect Participant	Reconnect Participant, Mark as Not Participating, Show Linking Code
Not Participating	Reactivate Participant	Reactivate Participant, Show Linking Code

Linking Code Display

G. When a Study Coordinator selects Show Linking Code for a Participant with **Pending** status, the interface SHALL display the **Mobile Linking Code** with a Copy action and a Save as PDF action.

H. The Save as PDF action SHALL generate a PDF containing the **Mobile Linking Code** and participant instructions.

I. When a Study Coordinator selects Show Linking Code for a Participant with any status other than **Pending**, the interface SHALL display the **Participant Linking Code**.

Link Participant Flow

J. When a Study Coordinator selects Link Participant, the interface SHALL display a Confirmation Dialog showing the **Participant** ID and the configured **Mobile Linking Code** expiry duration.

K. When the Study Coordinator confirms, the interface SHALL display an Acknowledgement Dialog showing the generated **Mobile Linking Code**, a Copy action, and the remaining time until expiry.

L. When the Study Coordinator dismisses the Acknowledgement Dialog, the interface SHALL update the **Participant's Status Badge** to **Pending**.

Rationale

The three-tab structure (**Not Connected**, **Active**, **Inactive**) groups participants by operational phase rather than by individual status, which keeps the dashboard scannable as the participant population grows: a Study Coordinator looking for a participant who is “somewhere in onboarding” finds them under one tab without first guessing whether they are Pending, Expired, or Not Connected. The action availability table is the operational core of the Coordinator's workflow — it encodes which actions are valid in which state, and the primary-action column makes the next-expected action reachable in a single click for the common path. Show Linking Code's split behavior (live **Mobile Linking Code** for Pending, retained **Participant Linking Code** for everyone else) reflects the linking-code lifecycle: Pending participants need the active code to complete linking; everyone else gets the historical reference for troubleshooting per DIARY-PRD-linking-code-lifecycle assertion H. The Save-as-PDF affordance supports the common workflow of giving the participant a physical copy of their linking code at a site visit. The Confirmation/Acknowledgement two-step on Link Participant captures both the Coordinator's confirmation of the action and the visible delivery of the generated code with a copy affordance and visible expiry timer.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

End Participant Dashboard Configuration | **Hash:** 15d19d33

1.8 Callisto Portal Authentication Overlays

This file holds the Callisto sponsor overlays for the **Sponsor Portal** authentication surface: the chosen second-factor method and code expiry, and the login rate-limiting threshold and cooldown values. The platform-side authentication foundation (password rules, 2FA, forgot-password, session management) lives in `hht_diary/spec/prd-portal-auth` and is referenced cross-repo via Refines.

CAL-PRD-two-factor-authentication-configuration: Two-Factor Authentication Configuration

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-two-factor-authentication

Assertions

- A. The **Second Factor** method SHALL be configured to email delivery to the **User Account** owner's registered **Email Address**.
- B. The **Code Expiry** SHALL be configured to 10 minutes.

Rationale

Email delivery of the **Verification Code** is the Callisto deployment's chosen **Second Factor** because the registered **Email Address** is already established as the authoritative communication channel for every **User Account** (it is the activation channel, the password-reset channel, and the username), and using it as the 2FA channel keeps the authentication surface coherent without introducing additional credential infrastructure (authenticator apps, SMS gateway). The 10-minute **Code Expiry** is short enough that a leaked code becomes useless quickly while long enough to absorb realistic email-delivery latency (typical clinical-environment email lag is under 2 minutes; the 10-minute window accommodates the long-tail cases).

End Two-Factor Authentication Configuration | **Hash:** d8e3f0ab

CAL-PRD-login-rate-limiting-configuration: Login Attempt Rate Limiting Configuration

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-session-management

Assertions

- A. The **Login Rate Limit Threshold** SHALL be configured to 5 consecutive failed login attempts.

- B. The **Login Rate Limit Cooldown** SHALL be configured to 60 seconds.
- C. The Login Rate Limit error message SHALL display the text: "Too many failed attempts. Please wait 60 seconds before trying again."

Rationale

Five consecutive failures is well above the rate at which a legitimate user mistyping their **Password** would trigger the rate limit (most legitimate users notice the mistake within two or three attempts), but well below the rate at which an automated brute-force attempt would be useful. The 60-second cooldown forces an attacker to amortize attempts across many waits, raising the cost of any brute-force campaign to impractical levels while imposing only a minor inconvenience on a legitimate user who has genuinely forgotten their password (they would typically initiate a Forgot Password reset rather than retry). The exact-text error message is specified so that the user-visible behavior is consistent across the deployment and matches the cooldown duration.

End Login Attempt Rate Limiting Configuration | **Hash:** 2a54a478

1.9 Callisto Questionnaire Management Overlays

This file holds the Callisto sponsor overlays for the questionnaire management surface: the Start Trial workflow and its interface, the NOSE HHT / HHT-QoL workflow that anchors the clinical questionnaire experience, the Manage Questionnaires Modal and Finalization Workflow GUI surfaces, and the per-deployment cycle-tracking and session-timing values. The platform-side questionnaire foundation (Clinical Questionnaire System, sponsor configuration, submission control, change control) lives in `hht_diary/spec/prd-questionnaire-management.md` and is referenced cross-repo via `Refines`.

CAL-PRD-trial-start-workflow: Start Trial Workflow

Level: prd | **Status:** Legacy | **Implements:** -

Overview

Before the trial is started, the diary entries remain on the **Participant's** device and are not transmitted to the **Sponsor Portal** or **Rave EDC**. This requirement defines the **Start Trial** action that activates **Diary Data Synchronization** and gates the **NOSE HHT** and **HHT-QoL Questionnaires** for the Callisto deployment.

Definitions

Start Trial: The action available to a Study Coordinator for a participant with **Linked - Awaiting Start** status that activates **Diary Data Synchronization** and begins the participant's active trial participation.

Assertions

- A. When a Study Coordinator confirms **Start Trial** for a participant with **Linked - Awaiting Start** status, the System SHALL change the Participant Status to **Trial Active** and activate **Diary Data Synchronization**.
- B. The System SHALL NOT activate **Diary Data Synchronization** for a participant before **Start Trial** is confirmed.
- C. The System SHALL NOT permit **NOSE HHT** or **HHT-QoL** Questionnaires to be sent to a participant before **Start Trial** is confirmed.

Rationale

Start Trial is the boundary at which a **Participant** transitions from pre-trial (their device is linked but their diary data and clinical questionnaires remain device-local)

to active trial (their data flows to the **Sponsor Portal** and **Rave EDC**, and clinical questionnaires can be administered). Anchoring synchronization activation and clinical questionnaire eligibility to a single, Study-Coordinator-confirmed event ensures that the trial-active state is an explicit clinical decision rather than an emergent consequence of device linking. The gate on clinical questionnaire delivery (NOSE HHT and HHT-QoL) is necessary because those questionnaires generate trial-grade clinical data; sending them before Start Trial would either produce data with no trial context or force a retroactive trial-start backdate that the audit trail cannot defensibly represent.

End Start Trial Workflow | **Hash:** d1421617

CAL-GUI-trial-start-workflow: Start Trial Workflow Interface

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** CAL-PRD-trial-start-workflow

Assertions

- A. The interface SHALL present a **Start Trial** button to the Study Coordinator in the Action column for participants with **Linked - Awaiting Start** status.
- B. When the Study Coordinator selects **Start Trial**, the interface SHALL display a Confirmation Dialog.
- C. The Confirmation Dialog SHALL present a **Send EQ** button and a cancel action.
- D. When the Study Coordinator selects **Send EQ**, the interface SHALL update the participant's status to **Trial Active** and replace the **Start Trial** button with a **Manage Questionnaires** button in the Action column.
- E. If the Study Coordinator cancels the Confirmation Dialog, the participant's status SHALL remain unchanged.

Rationale

The **Start Trial** button is placed in the Action column of the participant row so it is reachable in a single click from the **Participant Dashboard**, matching the daily workflow of a Study Coordinator who steps a series of newly-linked participants into trial-active state. The Confirmation Dialog is gated by an explicit **Send EQ** confirmation rather than a yes/no toggle because the action is consequential (it commits the **Participant** to clinical questionnaire administration and trial-grade data synchronization) and the labelling makes the next operational step (send the first clinical questionnaire) visible. Replacing **Start Trial** with **Manage Questionnaires** on confirmation is the visual cue that the **Participant** has moved into the trial-active workflow and the next set of actions is the per-questionnaire workflow.

End Start Trial Workflow Interface | **Hash:** a3809678

CAL-PRD-nose-hht-qol-workflow: NOSE HHT and HHT-QoL Questionnaire Workflow

Level: prd | **Status:** Legacy | **Implements:** -

Overview

NOSE HHT and **HHT-QoL** questionnaires are administered to participants on Day 1 of each treatment **Cycle**. A Study Coordinator initiates each cycle by sending the questionnaire(s) to the **Participant's Mobile Application**. This requirement governs the per-questionnaire workflow from send through finalization.

Assertions

Sending

- A. The System SHALL enforce a limit of one active **Questionnaire** per **Cycle** per **Questionnaire Type**.
- B. When a **Questionnaire** fails to reach the participant's mobile application, the System SHALL change its status to **Delivery Failed**.
- C. After the first **Cycle** has been finalized, the System SHALL make the **Start Next Cycle** available in place of **Send Now** action for that **Questionnaire Type**.

Deleting

- D. The System SHALL allow a Study Coordinator to call back a **Questionnaire** with **Sent**, **Delivery Failed** or **Ready to Review** status.
- E. The System SHALL NOT allow a Study Coordinator to call back a **Questionnaire** when it is already completed by the participant and finalized by the Study Coordinator.

Participant Completion

- F. The System SHALL allow the participant to edit their answers at any time before Finalization (i.e., while the **Questionnaire** has **Sent** or **Ready to Review** status). There is no limit on how many times a participant may edit their answers.
- G. When a participant submits a **Questionnaire**, the System SHALL change its status to **Ready to Review**.

Site Visit Review

- H. The System SHALL allow a Study Coordinator to review submitted **Questionnaire** answers before finalization.

Finalizing

- I. The System SHALL allow a Study Coordinator to finalize a **Questionnaire** with **Ready to Review** status.
- J. Upon finalization, the System SHALL lock the answers, calculate the **Questionnaire**

score, and transmit answers to **Rave EDC**.

K. When the finalization operation fails, the System SHALL automatically retry the operation in the background. The **Questionnaire** SHALL remain in **Ready to Review** status throughout.

L. While a finalization retry is in progress, the System SHALL display the following message to the Study Coordinator: "Submission is taking longer than expected. The system is retrying. No action is required."

M. Upon successful finalization, the System SHALL change the **Questionnaire** status to **Not Sent**.

Rationale

NOSE HHT and HHT-QoL are administered once per treatment **Cycle**, and the workflow encoded here reflects that cadence: at most one active **Questionnaire** per **Cycle** per **Type**, with the **Cycle** advancing only after finalization completes. The send/edit/submit/review/finalize sequence mirrors the clinical workflow — the participant fills out the instrument at home (with unlimited edits before submission to support honest answer revision), the Study Coordinator reviews the submitted answers during a site visit, and the Coordinator finalizes only after the review. Finalization is the locking event: it transmits to Rave EDC, calculates the score, and freezes the answers, after which call-back is no longer available. The retry-on-failure pattern with the participant-facing message is operationally important because Rave EDC connectivity can be transient, and a failure during finalization should not block the workflow or force the Coordinator to wait at the dialog. Returning the card to **Not Sent** on successful finalization is the signal that the next **Cycle** can be initiated.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

End NOSE HHT and HHT-QoL Questionnaire Workflow | **Hash:** 0efabf1b

CAL-GUI-manage-questionnaires-modal: Manage Questionnaires Modal

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** CAL-PRD-nose-hht-qol-workflow

Definitions

Questionnaire Card: The UI component displayed within the **Manage Questionnaires Modal** representing a single **Questionnaire Type** for a participant, showing its current status and available actions.

Select Starting Cycle Dialog: The dialog displayed when a Study Coordinator selects **Send Now** for the first time for a given **Questionnaire Type**, allowing selection of the **Starting Cycle** before sending.

Assertions

Modal

- A. The interface SHALL display the **Manage Questionnaires Modal** with the participant ID in the modal header.
- B. The interface SHALL display one **Questionnaire Card** per **Questionnaire Type**.
- C. The interface SHALL present a close action that dismisses the modal without making any changes.

Questionnaire Card — Display and Actions

- D. Each **Questionnaire Card** SHALL display the **Questionnaire Type** name and the current **Questionnaire Status**. Date and time information SHALL be paired inline with the **Cycle** it describes (e.g., "Cycle 1 . Apr 24, 2026, 11:34 AM"), not as a standalone field.
- E. The interface SHALL present cycle information, **Status Badge** placement, and available actions on each **Questionnaire Card** according to the following table:

Questionnaire Status	Cycle Fields Displayed	Status Badge Placement	Actions Available
Not Sent — never sent	None	Below Questionnaire Type name	Send Now
Not Sent — after finalize or call back	Finalized Cycle (with date), Next Cycle	Inline with Finalized Cycle row	Start Next Cycle
Sent	Current Cycle	Inline with Current Cycle row	Call Back (Trash icon)
Delivery Failed	Current Cycle	Inline with Current Cycle row, with information icon	Finalize, Call Back (Trash icon)
Ready to Review	Current Cycle (with completion date)	Inline with Current Cycle row	Finalize, Call Back (Trash icon)

Questionnaire Status	Cycle Fields Displayed	Status Badge Placement	Actions Available
Closed	Finalized Cycle (with date)	Inline with Finalized Cycle row, as combined badge ("Closed . End of Treatment" or "Closed . End of Study")	None

Delete Interaction

F. When a Study Coordinator selects the call back (Trash icon) action, the interface SHALL display a Reason Dialog — Free Text before proceeding.

G. When the Study Coordinator confirms the call back action, the interface SHALL update the **Questionnaire Card** status to **Not Sent**.

H. When the Study Coordinator cancels the Reason Dialog, the **Questionnaire** SHALL remain unchanged.

Send Now

I. The **Send Now** action is displayed only when the first **Questionnaire** of each **Questionnaire Type** will be sent to the participant. When a Study Coordinator selects **Send Now**, the interface SHALL display the **Select Starting Cycle Dialog**.

J. The **Select Starting Cycle Dialog** SHALL present a **Confirm and Send** button and a Cancel action.

K. When the Study Coordinator confirms the **Select Starting Cycle Dialog**, the interface SHALL update the **Questionnaire Card** status to **Sent**.

L. If the Study Coordinator cancels the **Select Starting Cycle Dialog**, the **Questionnaire Card** SHALL remain unchanged.

Start Next Cycle

M. When a Study Coordinator selects **Start Next Cycle**, the interface SHALL update the **Questionnaire Card** status to **Sent** without displaying the **Select Starting Cycle Dialog**.

Delivery Failed Troubleshooting

N. When the **Questionnaire Status** is **Delivery Failed**, the interface SHALL display an information icon adjacent to the **Status Badge**. When the **Study Coordinator** selects this icon, the interface SHALL display a **Troubleshooting Popover** with guidance on resolving delivery issues.

O. The **Troubleshooting Popover** SHALL dismiss when the **Study Coordinator** clicks outside its bounds or selects a close action within it.

Rationale

The **Manage Questionnaires Modal** is the per-participant control surface for the clinical questionnaire workflow, and its design favors a card-per-type layout because the Study Coordinator’s mental model is “what is the state of NOSE HHT for this participant, and what is the state of HHT-QoL?” rather than a flat list of every questionnaire ever sent. The status-driven action table collapses the workflow’s branch logic into a visible matrix: each status has exactly the actions it can take, no others appear, and the status badge sits next to the cycle it describes so the **Cycle** context is never ambiguous. The Select Starting Cycle Dialog appears only on the first send because that is the only moment at which the **Starting Cycle** is a Coordinator choice — every subsequent cycle is auto-incremented per CAL-PRD-questionnaire-cycle-tracking. The information-icon Troubleshooting Popover handles the **Delivery Failed** state in-line without requiring a separate screen, because the Coordinator’s diagnostic loop typically completes (try resending, check connectivity) without leaving the modal.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

End Manage Questionnaires Modal | **Hash:** 9cb05bc8

CAL-GUI-questionnaire-finalization-workflow: Questionnaire Finalization Workflow

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** CAL-PRD-nose-hht-qol-workflow

Definitions

Finalization Dialog: The Confirmation Dialog displayed to the Study Coordinator when the **Finalize** button is selected, containing a **Cycle** dropdown for selecting the **Cycle** value before confirming finalization.

Assertions

Finalization Dialog

A. When a Study Coordinator selects **Finalize**, the interface SHALL display the **Finalization Dialog** containing a **Cycle** dropdown.

B. The **Cycle** dropdown SHALL present the following selectable options: the **Current Cycle N Day 1** value, **End of Treatment**, and **End of Study**.

C. The **Finalization Dialog** SHALL present a **Finalize Questionnaire** button and a Cancel action.

Finalization Outcomes

D. When the **Study Coordinator** finalizes the **Questionnaire** with a **Cycle N Day 1** value selected, the interface SHALL update the **Questionnaire Card** status to **Not Sent** and display the **Finalized Cycle** (paired with the finalization date and time) and the **Next Cycle** on the card.

E. When the Study Coordinator finalized the **Questionnaire** with a **Terminal Cycle** value (**End of Treatment** or **End of Study**) selected, the interface SHALL update the **Questionnaire Card** status to **Closed**, display the **Terminal Cycle** value alongside the **Closed** status as a combined badge.

F. When the Study Coordinator cancels the **Finalization Dialog**, the **Questionnaire** SHALL remain unchanged.

G. When the **Study Coordinator** cancels the **Terminal Cycle Warning Dialog**, the **Questionnaire** SHALL remain unchanged and the **Study Coordinator** SHALL be returned to the **Finalization Dialog**.

Rationale

Finalization is the action that locks a questionnaire's answers and commits its score to Rave EDC, and the **Finalization Dialog** captures the two pieces of information that must be set at lock time: the **Cycle** value being finalized and the Coordinator's confirmation. Presenting the **Current Cycle N Day 1** plus the two terminal options (**End of Treatment, End of Study**) covers every legitimate finalization case — most cycles finalize to their own N Day 1, and the two terminal cases mark the **Participant's** endpoint for this **Questionnaire Type**. The Terminal Cycle Warning Dialog (referenced as a cancel path here) exists because terminal finalization is irreversible — no further **Questionnaires** of this type can be sent — and warrants a confirmation step distinct from the standard finalization confirmation. Cancelling either dialog returns the **Questionnaire** to its prior state so the Coordinator can revisit the choice without unintended side effects.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

End Questionnaire Finalization Workflow | **Hash:** 79ad946c

CAL-PRD-questionnaire-cycle-tracking: Questionnaire Cycle Tracking

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** CAL-PRD-nose-hht-qol-workflow

Overview

Each questionnaire must be traceable to a specific protocol **Cycle** for regulatory compliance and data integrity. Automatic cycle incrementing reduces manual error. Because the protocol allows only one questionnaire of each type per cycle, no two completed questionnaires of the same type for a given participant may share the same **Cycle** value. Some participants start the study using paper records, meaning their first electronic questionnaire may not be **Cycle 1 Day 1**. A Study Coordinator may select the starting cycle when sending the first questionnaire of each type. Cycle tracking is independent per **Questionnaire Type**.

Definitions

Cycle: The protocol event identifier assigned to a **Questionnaire** representing the treatment cycle during which it was administered. Valid values are **Cycle N Day 1** (where N is a positive whole number), **End of Treatment**, and **End of Study**.

Current Cycle: The **Cycle** value assigned to the most recently sent **Questionnaire** of a given **Questionnaire Type** for a participant.

Finalized Cycle: The **Cycle** value of the most recently finalized **Questionnaire** of a given **Questionnaire Type** for a participant.

Next Cycle: The **Cycle** value that will be assigned when the next **Questionnaire** of that **Questionnaire Type** is sent. Always the next sequential **Cycle N Day 1** value following the **Finalized Cycle** (the **Finalized Cycle** N incremented by 1).

Starting Cycle: The **Cycle** value assigned to the first **Questionnaire** of a given **Questionnaire Type** sent to a participant.

Terminal Cycle: A **Cycle** value of **End of Treatment** or **End of Study** indicating no further **Questionnaires** of that **Questionnaire Type** will be sent to the participant.

Cycle N Day 1: A **Cycle** value identifying the first day of the Nth treatment cycle, where N is a positive whole number (Cycle 1 Day 1, Cycle 2 Day 1, etc.).

Assertions

Cycle Assignment

- A. The System SHALL assign a **Cycle** value to each **Questionnaire** at the time it is sent.
- B. The System SHALL NOT allow two **Questionnaires** of the same **Questionnaire Type** for the same participant to share the same **Cycle** value.
- C. When a Study Coordinator sends the first **Questionnaire** of a given **Questionnaire Type** to a participant, the System SHALL require the Study Coordinator to select a **Starting Cycle** before the **Questionnaire** is sent.
- D. For each subsequent **Questionnaire** of a given **Questionnaire Type**, the System

SHALL assign the **Next Cycle** value automatically by incrementing the **Cycle N** value by 1.

E. When a **Questionnaire** is called back before finalization, the System SHALL reassign the same **Cycle** value to the next **Questionnaire** of that **Questionnaire Type** for that participant.

Terminal Cycles

F. When a **Questionnaire** is finalized with a **Terminal Cycle** value, the System SHALL change the **Questionnaire** status to **Closed**.

G. The System SHALL NOT allow a **Terminal Cycle** value to be assigned to more than one **Questionnaire** of the same **Questionnaire Type** per participant.

H. When the **Questionnaire** status is **Closed**, the System SHALL NOT permit a new **Questionnaire** of that **Questionnaire Type** to be sent to that participant.

Configuration

I. The System SHALL support per-sponsor configuration request of whether cycle tracking is enabled for a given deployment.

J. When cycle tracking is enabled, the System SHALL support per-sponsor configuration request of whether the Study Coordinator is required to select a **Starting Cycle** when sending the first **Questionnaire** of a given **Questionnaire Type** to a participant.

K. When the **Starting Cycle** selection is disabled, the System SHALL assign **Cycle 1 Day 1** as the **Starting Cycle** for all participants.

Rationale

The **Cycle** value is the protocol coordinate that lets Rave EDC and downstream analyses know which treatment cycle a given **Questionnaire** answer corresponds to. Two **Questionnaires** of the same **Questionnaire Type** sharing a **Cycle** value would break that coordinate (which cycle did the participant actually answer for?), so the uniqueness rule is structural rather than advisory. Automatic incrementing of **Next Cycle** removes a category of manual data-entry error from the Coordinator's workflow; the **Starting Cycle** Coordinator-selection at first-send accommodates the paper-then-electronic pattern in which the first electronic cycle is not Cycle 1. Reassigning the same **Cycle** value to the next **Questionnaire** after a call-back preserves the protocol coordinate across the call-back/resend round-trip — the call-back is operationally an "undo and redo" rather than a "skip and advance". Terminal cycles end the **Questionnaire Type** stream for the **Participant**; uniqueness of terminal cycles per **Type** per **Participant** prevents the ambiguous case of two End-of-Treatment markers for the same instrument.

End Questionnaire Cycle Tracking | **Hash:** 8fba1131

CAL-PRD-questionnaire-session-resume-timeout: Questionnaire Session Resume and Timeout

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-questionnaire-session-timeout

Overview

Session resume, expiry behavior, state preservation, and clock semantics are defined by the platform-side DIARY-PRD-questionnaire-session-timeout requirement. This requirement specifies only the sponsor portal-specific configuration values for the Callisto deployment.

Assertions

A. The **NOSE HHT** and **HHT-QoL** questionnaires SHALL be configured with a **Session Timeout** of 30 minutes.

B. The **NOSE HHT** and **HHT-QoL** questionnaires SHALL be configured with a **Timeout Warning Notification** threshold of 5 minutes before expiry.

Rationale

The 30-minute **Session Timeout** matches the typical clinical-portal completion time for the NOSE HHT and HHT-QoL instruments (10-20 minutes for a Participant working through the questions, with comfortable headroom for interruption). The 5-minute warning threshold gives the Participant enough time to either complete the session or actively dismiss the warning before expiry without being so early that the warning becomes background noise. Both values are sponsor-configurable at the platform level (DIARY-PRD-questionnaire-session-timeout); this Callisto overlay pins the specific values that the deployment is operating under.

End Questionnaire Session Resume and Timeout | **Hash:** 9431d100

1.10 Callisto Participant Questionnaire Workflow Overlays

This file holds the Callisto sponsor overlay for the participant questionnaire workflow: the **Portal-Sent Questionnaire Call-Back — Participant Experience** that defines what the participant sees when a **Study Coordinator** calls back an assigned questionnaire. The platform-side workflow foundation (DIARY-PRD-questionnaire-portal-sent-rules, DIARY-GUI-questionnaire-portal-sent-workflow, DIARY-PRD-questionnaire-session-timeout, DIARY-GUI-questionnaire-session-expiry) lives in `hht_diary/spec/prd-questionnaire-part` and is referenced cross-repo via Refines.

CAL-GUI-questionnaire-call-back-participant: Portal-Sent Questionnaire Call-Back — Participant Experience

Level: GUI | **Status:** Legacy | **Implements:** - **Refines:** DIARY-GUI-questionnaire-portal-sent-workflow

Overview

Participants need clear, immediate feedback when a Portal-Sent Questionnaire is called back so they do not waste effort on an inactive questionnaire or become confused about its disappearance.

Definitions

Call-Back Notice: An Acknowledgement Dialog informing the **Participant** that a **Portal-Sent Questionnaire** assigned to them has been called back by the study team.

Assertions

Open Questionnaire Interruption

- A. When a **Portal-Sent Questionnaire** is called back while the **Participant** has it open, the interface SHALL display the Call-Back Notice.
- B. The Call-Back Notice SHALL NOT be dismissible by tapping outside the dialog.
- C. When the **Participant** acknowledges the Call-Back Notice, the interface SHALL navigate the **Participant** to the **Main Screen**.

Task List

- D. When a **Portal-Sent Questionnaire** has been called back, the interface SHALL remove the corresponding Questionnaire Task from the Task List.

Next Session Notification

E. When a **Portal-Sent Questionnaire** has been called back and the **Participant** did not have it open at the time of call-back, the interface SHALL display the Call-Back Notice on the **Participant's** next Application session.

Offline Submission Attempt

F. When the **Participant** attempts to submit a **Portal-Sent Questionnaire** that has been called back, the interface SHALL display the Call-Back Notice.

Rationale

A call-back removes the questionnaire from the participant's active task set, but if the participant has the questionnaire open at the moment of call-back (or attempts to submit one whose call-back happened while they were offline), they need explicit feedback that the questionnaire is no longer assigned to them. The Call-Back Notice provides that feedback in three contexts: live interruption (open now, called back now), deferred notification (called back while not open, surface on next session), and offline submission attempt (called back while offline, surface when submission tries to land). The non-dismissible-by-outside-tap rule (assertion B) prevents the participant from accidentally clearing the notice without reading it. Removing the Questionnaire Task from the Task List immediately on call-back closes the entry point on the **Main Screen** so the participant does not navigate into the called-back questionnaire from there. Routing acknowledgement to the **Main Screen** rather than back into the now-defunct questionnaire flow is the only sensible destination — the questionnaire is no longer interactable, so returning the participant to it would be a dead end.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

Screen reference

See: Call Back Notice

End Portal-Sent Questionnaire Call-Back — Participant Experience | **Hash:** 7bc60d78

1.11 CAL-PRD-questionnaire-score-display-prohibition: Questionnaire Score Display Prohibition

Level: prd | **Status:** Legacy | **Implements:** -

Overview

The platform does not permit calculated questionnaire scores to be displayed to any user.

Assertions

A. The System SHALL NOT display calculated Questionnaire Score to any user.

Rationale

For the Callisto deployment, calculated questionnaire scores are clinical research outputs that belong in the analysis pipeline, not the user-facing interface. Displaying a score to a Study Coordinator or to a participant could bias subsequent participant responses (priming effects in patient-reported outcome instruments are well documented), undermine the blinding required by the trial protocol, and create the impression that the score is itself a clinical finding for which the portal is the authoritative surface. The score is computed and recorded for downstream analysis; the display surface stays free of it across every role and every screen.

Follow-up — configurability: This requirement currently encodes the only option implemented in code. Future sponsors may require different rules; introduce a configurable seam (e.g. a parameter on the CAL-PRD-* parent, or a new platform-side template the CAL- REQ Satisfies) when the need arises. Until that seam exists, this REQ is normative for the Callisto deployment.

End Questionnaire Score Display Prohibition | **Hash:** 23d84761

1.12 Callisto RBAC Overlays

This file holds the Callisto sponsor overlays for role-based access control: the permissions table binding roles to actions, and the per-role site-view scope rule. The platform-side RBAC foundation (customizable RBAC, Action Inventory, Role Definitions, role-switching GUI) lives in `hht_diary/spec/prd-rbac.md` and is referenced cross-repo via `Refines`.

CAL-PRD-permissions-table: Permissions Table

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-action-inventory, DIARY-PRD-rbac-customizable, DIARY-PRD-role-definitions

Overview

The permissions table defines which roles are granted each Action from the Action Inventory (DIARY-PRD-action-inventory). It is the sponsor-level binding from platform-defined roles (DIARY-PRD-role-definitions) to platform-defined Actions for the Callisto deployment.

Assertions

A. The System SHALL enforce the following permission rules in the sponsor portal: In the following tables, Y indicates the role is granted the Action and N indicates the role is denied the Action.

User Account Management

End Permissions Table | **Hash:** b59776a6

Action ID	Action	Study Coordinator	CRA	Administrator
ACT-USR-001	Create User Account	N	N	Y
ACT-USR-002	Edit User Account	N	N	Y
ACT-USR-003	Deactivate User Account	N	N	Y
ACT-USR-004	Reactivate User Account	N	N	Y
ACT-USR-005	Unlock User Account	N	N	Y
ACT-USR-006	Resend activation email	N	N	Y
ACT-USR-007	Assign role to User Account	N	N	Y
ACT-USR-008	Assign Site to User Account	N	N	Y

Participant Management

Action ID	Action	Study Coordinator	CRA	Administrator
ACT-PAT-001	Link participant	Y	N	N
ACT-PAT-002	Disconnect participant	Y	N	N
ACT-PAT-003	Reconnect participant	Y	N	N
ACT-PAT-004	Mark participant as not participating	Y	N	N
ACT-PAT-005	Reactivate participant	Y	N	N
ACT-PAT-006	View participant	Y	Y	N

Questionnaire Management

Action ID	Action	Study Coordinator	CRA	Administrator
ACT-QST-001	Send questionnaire	Y	N	N
ACT-QST-002	Call back questionnaire	Y	N	N
ACT-QST-003	Finalize questionnaire	Y	N	N

Sites and Audit Log

Action ID	Action	Study Coordinator	CRA	Administrator
ACT-SIT-001	View Sites	Y	Y	Y
ACT-AUD-001	View audit log	Y	Y	Y

Rationale

The Callisto permissions matrix encodes a strict separation between operational roles (Study Coordinator), monitoring roles (CRA), and administrative roles (Administrator). Study Coordinators perform all participant- and questionnaire-facing actions because the operational workflow is theirs alone; CRAs are read-only by design, with view access to participants and the audit log at their assigned sites so they can monitor compliance without altering trial state; Administrators are confined to User Account lifecycle because their accountability is account governance, not clinical operations. View permissions for Sites and Audit Log are granted to all three roles because each role needs site context and audit visibility scoped to its own responsibility (the scope itself is enforced by CAL-PRD-site-view-scope and CAL-PRD-audit-log-view-scope, not by this table). Every cell in the matrix is defensible against the FDA 21 CFR Part 11 principle of separation of duties and against the operational principle that a single mistake by any role should not cascade across boundaries.

End Permissions Table | **Hash:** 00000000

CAL-PRD-site-view-scope: Site View Scope

Level: prd | **Status:** Legacy | **Implements:** - **Refines:** DIARY-PRD-role-definitions

Overview

This requirement defines which **Sites** each role may view within the Callisto sponsor portal. Permissions to perform Actions at a Site are governed by CAL-PRD-permissions-table; this requirement governs visibility scope.

Assertions

- A. A **Study Coordinator** SHALL only view **Sites** to which they are assigned.
- B. A **Clinical Research Associate** SHALL only view **Sites** to which they are assigned.
- C. An **Administrator** SHALL view all **Sites** within the sponsor portal.

Rationale

Site-level visibility is the operational analogue of role-based action permissions: a Study Coordinator should not see participants or activity at Sites they do not staff, both because those records are not their responsibility and because broad cross-site visibility leaks operational information across investigative teams. CRAs are similarly scoped to their assigned Sites; their job is to monitor those Sites, not to oversee the entire study from a single console. Administrators see all Sites because their accountability — User Account management, role and Site assignment — is study-wide by definition; restricting Administrators to a Site subset would prevent them from performing the very operations the role exists to perform.

End Site View Scope | **Hash:** 6ea645cb

Operational Requirements

2.1 Callisto CI Validation Placeholders

Forward-looking obligations for the Callisto repository's PR-time CI validation surface: requirement traceability, change-appropriate validation, secret and vulnerability scanning, and code quality / static analysis. These requirements are not yet implemented in canonical form — they coexist with the active CI infrastructure in the `hht_workflows` and `hht_admin` repositories and will be reconciled against that infrastructure as the canonical Callisto pipeline takes shape.

CAL-OPS-ci-traceability-pipeline: CI/CD Pipeline for Requirement Traceability (placeholder)

Level: ops | **Status:** Legacy | **Implements:** -

Rationale

TODO: canonical rationale not yet authored. Referenced by the PR Health workflow chain (`pr-health.yml`, `ci-gate.yml`) introduced by CUR-1246.

Assertions

A. The pull-request validation pipeline SHALL surface requirement traceability errors that block merge to main.

End CI/CD Pipeline for Requirement Traceability (placeholder) | **Hash:** e90b3229

CAL-OPS-ci-change-appropriate-validation: Change-Appropriate CI Validation (placeholder)

Level: ops | **Status:** Legacy | **Implements:** -

Rationale

TODO: canonical rationale not yet authored. Referenced by the change-detection job in `pr-health.yml` and the rollup in `ci-gate.yml`.

Assertions

A. The CI Gate SHALL run validation steps appropriate to the categories of files changed in a pull request, skipping irrelevant validation when no files in the relevant category changed.

End Change-Appropriate CI Validation (placeholder) | **Hash:** 339feadd

CAL-OPS-ci-secret-vulnerability-scanning: Secret and Vulnerability Scanning (placeholder)

Level: ops | **Status:** Legacy | **Implements:** -

Rationale

TODO: canonical rationale not yet authored. Referenced by security-check (gitleaks) and security-scan (Trivy) jobs in pr-health.yml.

Assertions

A. PR-time CI SHALL run automated security scanning that blocks merge on detected secrets and surfaces vulnerability / misconfiguration findings via GitHub Code Scanning.

End Secret and Vulnerability Scanning (placeholder) | **Hash:** 8273f96a

CAL-OPS-ci-code-quality-static-analysis: Code Quality and Static Analysis (placeholder)

Level: ops | **Status:** Legacy | **Implements:** -

Rationale

TODO: canonical rationale not yet authored. Referenced by validate-code-headers, validate-migrations, and doc-lint jobs in pr-health.yml.

Assertions

A. PR-time CI SHALL run automated code-quality and static-analysis gates covering implementation-header presence, migration-header format, and documentation linting.

End Code Quality and Static Analysis (placeholder) | **Hash:** 9e617957

2.2 Callisto Deployment Secrets

Operations-level obligations for the per-environment Doppler service tokens that Cloud Run revisions use to authenticate against Doppler at startup. Defines the routing path from the Doppler vault, through GitHub Actions deployment environments, onto Cloud Run, with secret values scoped per environment and sponsor identity sourced from `deployment/base-config.json`.

CAL-OPS-secrets-doppler-cloud-run: Per-environment Doppler service tokens for Cloud Run runtime

Level: ops | **Status:** Legacy | **Implements:** -

Rationale

Cloud Run revisions authenticate against Doppler at container startup using a service token passed in via the `DOPPLER_TOKEN` environment variable. A single shared token across environments would let any compromise in a lower environment leak secrets from higher environments (and vice-versa: a leak in a high environment would force rotation across all environments). Per-env service tokens contain blast radius and let rotation operate at one-environment granularity. Tokens are minted with no expiry so an automated job does not silently break on token aging; rotation is operator-initiated and verified before the prior token is revoked.

Assertions

A. The System SHALL maintain one Doppler service token per environment (dev, qa, uat, prod), each scoped read-only to that environment's Doppler config.

B. The System SHALL name each per-env service token using the format `cloudrun-<service>-<env>` where `<service>` identifies the Cloud Run service and `<env>` identifies the target environment.

C. Per-env Doppler service tokens SHALL be configured with no expiry.

D. The System SHALL never log, echo, persist to disk, or expose on a process command line the value of any per-env Doppler service token.

E. When a per-env Doppler service token is rotated, the System SHALL retain the prior token in Doppler until the new token is verified working in the corresponding Cloud Run revision; the prior token SHALL be revoked once verification succeeds.

End Per-environment Doppler service tokens for Cloud Run runtime | **Hash:** 6ccecb17

CAL-OPS-secrets-github-actions-routing: GitHub Actions deployment environments route per-env secrets

Level: ops | **Status:** Legacy | **Implements:** - **Refines:** CAL-OPS-secrets-doppler-cloud-run

Rationale

GitHub Actions deployment environments scope secret resolution to the environment selected at deploy time: a workflow job declaring environment: <env> reads same-named secrets from the matching deployment-environment secret store rather than from repository-level secrets. This routes the per-env Doppler service token (REQ-CAL-o00006) through standard CI infrastructure without per-env secret-name conventions in workflow code, and unlocks per-env protection rules (required reviewers, wait timers) as a future opt-in.

Assertions

- A. The System SHALL maintain one GitHub Actions deployment environment per Cloud Run target environment (dev, qa, uat, prod).
- B. Each GitHub Actions deployment environment SHALL hold a DOPPLER_TOKEN secret containing the corresponding per-env Doppler service token defined by REQ-CAL-o00006.
- C. The Cloud Run deploy workflow SHALL declare environment: <env> on the deploy job, where <env> is the target deployment environment, so secret resolution routes to the matching deployment-environment secret store.
- D. The System SHALL NOT depend on a repository-level DOPPLER_TOKEN secret for any production-bound deploy.
- E. Caller workflows that invoke the Cloud Run deploy workflow via uses: SHALL declare secrets: inherit. Empirically, environment-scoped secret resolution in a called workflow's job is gated on the calling workflow propagating its secrets context: without secrets: inherit, secrets.DOPPLER_TOKEN resolves to the empty string in the called workflow even when the deployment environment's DOPPLER_TOKEN secret is correctly populated.

End GitHub Actions deployment environments route per-env secrets | **Hash:** aca006c3

CAL-OPS-secrets-sponsor-agnostic-automation: Sponsor-agnostic deployment automation reads sponsor name from base-config.json

Level: ops | **Status:** Legacy | **Implements:** -

Rationale

The Callisto sponsor repo is one of several planned sponsor repos using the same multi-sponsor platform. Hardcoding the literal string `callisto` in deployment automation creates a maintenance burden when onboarding new sponsors and obscures which scripts are sponsor-portable vs sponsor-specific. The single source of truth for sponsor identity in this repo is `deployment/base-config.json`, which already declares "sponsor" for the build pipeline. Deployment automation SHALL read sponsor identity from that file rather than hardcoding it, so that bringing up a new sponsor requires no per-script edits.

Assertions

- A. Scripts under `deployment/scripts/` that need a sponsor identifier SHALL read `.sponsor` from `deployment/base-config.json` rather than hardcoding the literal string.
- B. The System SHALL fail loudly with a clear error message if `deployment/base-config.json` is missing or does not contain a non-empty `.sponsor` value, when invoked by an automation script that requires it.

End Sponsor-agnostic deployment automation reads sponsor name from base-config.json
| **Hash:** ace2c978

2.3 Callisto Deployment Operations

Operations-level obligations for the end-to-end Cloud Run deployment system: verification gates, provenance, permission scoping, notification routing, rollback and canary controls, concurrency locks, cross-repo image refresh, regulatory audit trail, and run-book coverage.

See also: `ops-deployment-secrets.md` for the per-env Doppler routing SOP that this document builds on.

CAL-OPS-deploy-cloud-run-verification: Cloud Run deploy verification gates

Level: ops | **Status:** Legacy | **Implements:** -

Overview

Today the deploy workflow's only post-deploy verification is a `/health` probe. That probe stays green even when Doppler injection has silently failed (CUR-1194), because `DatabaseConfig.fromEnvironment` defaults `DB_HOST` to `localhost` when the variable is unset and `/health` does not actually run a database query. This requirement establishes the verification contract: liveness, Doppler-injection probe per service, database-connectivity probe, and version assertion against the workflow's input. Any gate that fails fails the deploy.

Rationale

The CUR-1194 incident took hours to diagnose because the deploy verify step gave a false positive. A verification step that exercises the actual fault domain — env-var injection and database access — would have caught the bad rotation within seconds of the first deploy. Version assertion closes a related gap: today there is no machine check that the revision running matches the version label the operator typed into the workflow form.

Assertions

- A. The Cloud Run deploy workflow SHALL fail the workflow run when the deployed revision's `/health` endpoint does not return HTTP 200 within `{{health_probe_timeout_seconds}}` seconds of the new revision becoming ready.
- B. When the Cloud Run deploy workflow updates a `portal-service`, the workflow SHALL fail the run if the deployed revision's `/api/v1/portal/config/identity` endpoint does not return HTTP 200 within `{{verification_timeout_seconds}}` seconds.

C. When the Cloud Run deploy workflow updates a portal-service, the workflow SHALL fail the run if the response body of `/api/v1/portal/config/identity` does not contain a non-empty `projectId` field.

D. When the Cloud Run deploy workflow updates a diary-service, the workflow SHALL fail the run if the deployed revision's designated Doppler-injection-sensitive endpoint does not return HTTP 200 within `{{verification_timeout_seconds}}` seconds.

E. The Cloud Run deploy workflow SHALL fail the workflow run when the deployed revision's designated database-connectivity endpoint does not return HTTP 200 indicating a successful round-trip query against the bound database.

F. When the Cloud Run deploy workflow is invoked with a non-empty version input, the workflow SHALL fail the run if the deployed revision's reported component versions do not match the input value.

End Cloud Run deploy verification gates | **Hash:** 31b96204

CAL-OPS-deploy-provenance-traceability: Deploy provenance and traceability

Level: ops | **Status:** Legacy | **Implements:** -

Overview

Every Cloud Run revision is linkable bidirectionally to the triggering pull request, the GitHub Actions workflow run, the actor, the image digest, the embedded base-image commit SHAs (sponsor-ci, portal-server, diary-server), and the Doppler config and token slug used at deploy time. A documented one-command query produces the complete deploy record from any of those keys.

Rationale

CUR-1194 diagnosis required ad-hoc archaeology — manifest blob inspection, OCI label parsing, and Doppler token introspection — to answer “what code is running and from what config?” Cloud Run revision annotations are the natural pivot for this metadata; today they are unused.

Assertions

A. When the Cloud Run deploy workflow updates a service, the workflow SHALL set an annotation on the new revision containing the URL of the GitHub pull request that authorized the deploy.

B. When the Cloud Run deploy workflow updates a service, the workflow SHALL set an annotation on the new revision containing the GitHub Actions workflow-run identifier.

- C. When the Cloud Run deploy workflow updates a service, the workflow SHALL set an annotation on the new revision containing the GitHub username of the actor that triggered the workflow.
- D. When the Cloud Run deploy workflow updates a service, the workflow SHALL set an annotation on the new revision containing the immutable image digest of the deployed container image.
- E. When the Cloud Run deploy workflow updates a service, the workflow SHALL set an annotation on the new revision containing the slug of the Doppler service token used to inject runtime secrets.
- F. The image-build workflow SHALL embed the source commit SHAs of every base image consumed (sponsor-ci, portal-server, diary-server) as OCI labels on the resulting image.
- G. The repository SHALL contain an executable script that, given any of {revision name, image digest, pull-request number, ticket identifier}, outputs the complete deploy record as JSON within {{record_query_timeout_seconds}} seconds.

End Deploy provenance and traceability | **Hash:** 06ad47b1

CAL-OPS-deploy-permission: Per-environment permission scoping

Level: ops | **Status:** Legacy | **Implements:** - **Refines:** CAL-OPS-secrets-doppler-cloud-run

Overview

Permissions across GCP IAM, Doppler config write, GitHub Actions environment protection, and deploy-time service tokens follow a tiered model: dev least restrictive, prod requiring multi-party approval. Automation tokens are functionally distinct from human credentials, run from dedicated service identities, and hold the minimum scope required for their job. The token-rotation script runs from a dedicated service identity, not from individual operator CLI authentication.

This template captures env-agnostic permission obligations. Per-environment gating (which envs require pre-deploy approval, wait timers, etc.) lives on the satisfying REQs CAL-OPS-deploy-permission-{dev,qa,uat,prod}; elspais clones each template assertion into per-env instance nodes so coverage flows per environment.

Rationale

CUR-1194's root cause was unrecoverable in part because the rotation script that minted the bad tokens ran from an individual operator's CLI auth, leaving no audit trail of which credential state produced the mis-scope. Running automation from a dedicated service identity eliminates operator-state ambiguity. The tier model codifies access conventions that today are informal.

Assertions

- A. The repository SHALL contain a permission-matrix document at `spec/ops-deployment-permission` covering, for each of {dev, qa, uat, prod}, the access levels for GCP IAM, Doppler config write, GitHub Actions environment protection, and deploy-time service tokens.
- B. The Doppler service-token rotation script SHALL execute under a dedicated rotation service identity, not under any individual operator's personal credentials.
- C. The image-build pipeline SHALL NOT have read access to any Doppler service token.

End Per-environment permission scoping | **Hash:** 68ee2a74

CAL-OPS-deploy-permission-dev: Permission gating — dev

Level: ops | **Status:** Legacy | **Satisfies:** CAL-OPS-deploy-permission

Rationale

dev is the least-restrictive tier. Any developer can dispatch a dev deploy; no approval gate. Permission specifics follow the matrix referenced by the template.

Assertions

- A. The GitHub Actions deployment environment named dev SHALL NOT require pre-deploy approval.

End Permission gating — dev | **Hash:** 37e59910

CAL-OPS-deploy-permission-qa: Permission gating — qa

Level: ops | **Status:** Legacy | **Satisfies:** CAL-OPS-deploy-permission

Rationale

qa carries no pre-deploy approval gate; QA-team review happens post-deploy via the test-execution loop, not via deployment gating.

Assertions

- A. The GitHub Actions deployment environment named qa SHALL NOT require pre-deploy approval.

End Permission gating — qa | **Hash:** 9e6df3e3

CAL-OPS-deploy-permission-uat: Permission gating — uat

Level: ops | **Status:** Legacy | **Satisfies:** CAL-OPS-deploy-permission

Rationale

uat requires single-reviewer approval because uat is the staging surface where sponsors validate releases. uat does not yet require a wait timer (vs prod's reviewer + wait-timer combination).

Assertions

A. The GitHub Actions deployment environment named uat SHALL require approval from at least one designated reviewer before any job in that environment runs.

End Permission gating — uat | **Hash:** 0f9ccabb

CAL-OPS-deploy-permission-prod: Permission gating — prod

Level: ops | **Status:** Legacy | **Satisfies:** CAL-OPS-deploy-permission

Rationale

prod requires both reviewer approval and a wait timer between approval and execution, so an inadvertent or coerced approval has a window in which it can be cancelled before any side-effect lands. The wait-timer value is parameterized in deployment configuration.

Assertions

A. The GitHub Actions deployment environment named prod SHALL require approval from at least one designated reviewer before any job in that environment runs.

B. The GitHub Actions deployment environment named prod SHALL impose a wait timer of at least {{prod_wait_minutes}} minutes between approval and job execution.

End Permission gating — prod | **Hash:** 68860323

CAL-OPS-deploy-notification: Per-environment notification routing

Level: ops | **Status:** Legacy | **Implements:** -

Depends on: PagerDuty service configured with a signed HIPAA BAA, plus a PagerDuty integration credential available to the deploy workflow. Tracked under the Observability Phase 5 (On-Call & Escalation) work — Linear view: linear.app/cure-hht-diary/team/CUR/view/observability-fb50ea08fb2f. The PagerDuty assertions on the uat and prod satisfying REQs cannot be satisfied until that prerequisite lands.

Overview

Deploy events, secret rotations, and database resets emit push notifications matched to the audience for each target environment. Volume scales with blast radius: dev events post to a low-noise channel, qa to a QA-team channel, uat broadcasts to the full Anspar Slack and emails the sponsor representative, and prod additionally pages on-call. Every notification carries a one-click Cloud Logging query link pre-filtered to the service, environment, and time window of the event.

This template captures env-agnostic notification obligations (the Cloud Logging URL convention, the documented log-query inventory). Per-environment routing (Slack channels, email recipients, paging) lives on the satisfying REQs CAL-0PS-deploy-notification-`{de`

Slack destinations are referenced by role-keyed configuration variables, not by hard-coded channel names; the actual channel each key resolves to is set in the deployment configuration (and may be re-pointed without a spec change).

The full routing matrix (the satisfying REQs are normative; this matrix is a quick-reference):

Trigger	Slack channel (config key)	Sponsor email	On-call page
dev deploy success	<code>\${SLACK_DEV_DEPLOY_CH}</code> (summary)		no
dev deploy failure	<code>\${SLACK_DEV_DEPLOY_CH}</code> (@here)		no
qa deploy success	<code>\${SLACK_QA_DEPLOY_CH}</code> (summary)		no
qa deploy failure	<code>\${SLACK_QA_DEPLOY_CH}</code> and <code>\${SLACK_DEVOPS_CH}</code> (@here)		no
uat deploy success	<code>\${SLACK_BROADCAST_CH}</code> (summary)		no
uat deploy failure	<code>\${SLACK_BROADCAST_CH}</code> (@channel)		yes
prod deploy any outcome	<code>\${SLACK_BROADCAST_CH}</code>		yes

The configuration keys, in role terms:

- SLACK_DEV_DEPLOY_CH — low-noise channel for dev deploy traffic.
- SLACK_QA_DEPLOY_CH — channel watched by the QA team.
- SLACK_DEVOPS_CH — channel watched by DevOps for cross-team escalation when a non-prod deploy fails.
- SLACK_BROADCAST_CH — wide-audience channel used for uat and prod deploy events.

Rationale

CUR-1194's bad rotation went unnoticed for ~16 hours because no signal left GitHub Actions. Notification volume tracks blast radius: a dev deploy hitting the broadcast channel would create noise; a prod deploy without paging is regulatory exposure. Routing by role-keyed config variables means the volume/blast-radius mapping is testable independently of any specific channel name, and the same spec applies across sponsors with different Slack workspaces. Embedding pre-filtered log links in notifications eliminates the "30 minutes to find the right log" friction that surfaced repeatedly during CUR-1194 diagnosis.

Assertions

- A. The Cloud Run deploy workflow SHALL emit a notification on every deploy completion (success or failure) to the destinations defined by the satisfying REQ for the target environment.
- B. Every Slack notification posted by a deploy workflow SHALL contain a Cloud Logging URL pre-filtered to the target service, target environment, and a time window that includes the deploy event.
- C. The repository SHALL contain a documented Cloud Logging query for portal-service startup environment banner inspection.
- D. The repository SHALL contain a documented Cloud Logging query for portal-service /api/v1/portal/config/identity non-200 response.
- E. The repository SHALL contain a documented Cloud Logging query for database-connection failure across services.
- F. The repository SHALL contain a documented Cloud Logging query for Doppler authentication or fetch errors.

End Per-environment notification routing | **Hash:** 2078e9c4

CAL-OPS-deploy-notification-dev: Notification routing — dev

Level: ops | **Status:** Legacy | **Satisfies:** CAL-OPS-deploy-notification

Rationale

dev is the lowest-blast-radius tier. Success and failure both go to a single dev-team channel; no sponsor email, no on-call page.

Assertions

A. A successful dev deploy SHALL post a deploy-summary message to the Slack channel `${SLACK_DEV_DEPLOY_CH}`.

B. A failed dev deploy SHALL post an @here-tagged failure message to the Slack channel `${SLACK_DEV_DEPLOY_CH}`.

End Notification routing — dev | **Hash:** ab29f35b

CAL-OPS-deploy-notification-qa: Notification routing — qa

Level: ops | **Status:** Legacy | **Satisfies:** CAL-OPS-deploy-notification

Rationale

qa is QA-team-facing. Success goes to the QA channel; failure additionally escalates to DevOps so a broken qa pipeline doesn't silently block test execution.

Assertions

A. A successful qa deploy SHALL post a deploy-summary message to the Slack channel `${SLACK_QA_DEPLOY_CH}`.

B. A failed qa deploy SHALL post an @here-tagged failure message to the Slack channel `${SLACK_QA_DEPLOY_CH}`.

C. A failed qa deploy SHALL also post an @here-tagged failure message to the Slack channel `${SLACK_DEVOPS_CH}`.

End Notification routing — qa | **Hash:** 0bd1fa75

CAL-OPS-deploy-notification-uat: Notification routing — uat

Level: ops | **Status:** Legacy | **Satisfies:** CAL-OPS-deploy-notification

Rationale

uat is the sponsor-validation surface; sponsors must be aware of every deploy. Success notifies broadly and emails the sponsor; failure additionally pages on-call so a

stuck uat doesn't block sponsor validation overnight.

Assertions

- A. A successful uat deploy SHALL post a deploy-summary message to the Slack channel `${SLACK_BROADCAST_CH}`.
- B. A successful uat deploy SHALL send a deploy-notification email to the sponsor representative configured for the target sponsor.
- C. A failed uat deploy SHALL post an @channel-tagged failure message to the Slack channel `${SLACK_BROADCAST_CH}`.
- D. A failed uat deploy SHALL send a failure-notification email to the sponsor representative.
- E. A failed uat deploy SHALL trigger a PagerDuty incident on the configured service.

End Notification routing — uat | **Hash:** 2d3f7eba

CAL-OPS-deploy-notification-prod: Notification routing — prod

Level: ops | **Status:** Legacy | **Satisfies:** CAL-OPS-deploy-notification

Rationale

prod has the highest blast radius. Every prod deploy outcome (success or failure) goes to the broadcast Slack channel, emails the sponsor, and pages on-call — the symmetric volume reflects that any prod state change is operationally significant.

Assertions

- A. Any prod deploy outcome SHALL post a deploy-result message to the Slack channel `${SLACK_BROADCAST_CH}`.
- B. Any prod deploy outcome SHALL send a deploy-result email to the sponsor representative.
- C. Any prod deploy outcome SHALL trigger a PagerDuty incident on the configured service.

End Notification routing — prod | **Hash:** 88b52b13

CAL-OPS-deploy-rollback-canary: One-command rollback and canary traffic split

Level: ops | **Status:** Legacy | **Implements:** -

Overview

A `rollback.yml` workflow takes (`service`, `env`, `target-revision`) and shifts 100% traffic to the named revision, firing the same notification fan-out as a forward deploy. For `uat` and `prod`, deploys support a canary phase: configurable percentage routes to the new revision for a configurable duration before promoting to 100%, gated on the verification checks defined elsewhere in this specification. Image references at the deploy boundary are immutable digests; mutable tags such as `:main-latest` are rejected at workflow input.

Rationale

Today rollback is “open the Cloud Run console and click Manage Traffic” — unscriptable in an emergency, no audit trail, no notification. Canary traffic split exists in Cloud Run natively but is not wired into the deploy workflow, so every release is a 100% cutover. Mutable tags at the deploy boundary were the proximate cause of “two builds from the same callisto SHA produced different runtime behavior” in CUR-1194’s image investigation.

Assertions

- A. The repository SHALL contain a workflow named `rollback.yml` that accepts inputs `service`, `sponsor-env`, and `target-revision`, and shifts 100% of traffic on the named service in the named environment to the named revision.
 - B. When the `rollback.yml` workflow runs, the workflow SHALL emit a `rollback-result` notification to the Slack channels and email recipients designated for `deploy-result` notifications in the target environment.
 - C. When the Cloud Run deploy workflow targets `uat` or `prod` and a non-zero canary percentage is configured, the workflow SHALL route the configured percentage of traffic to the new revision for the configured duration before promoting to 100%.
 - D. When a post-deploy verification check fails during a canary window in the Cloud Run deploy workflow, the workflow SHALL terminate the run with the prior revision still receiving 100% of traffic.
 - E. The Cloud Run deploy workflow SHALL reject any `image_path` input value that does not reference an immutable image digest of the form `<host>/<repo>/<image>@sha256:<64-hex>`.
- End One-command rollback and canary traffic split* | **Hash:** a5a19b96

CAL-OPS-deploy-concurrency-locks: Deploy concurrency and safety locks

Level: ops | **Status:** Legacy | **Implements:** -

Overview

Each deploy workflow declares a concurrency group keyed by (service, environment) so only one deploy runs per pair at a time. Destructive workflows (database resets) declare per-environment concurrency. A repo-level deploy-paused signal SHALL exist; deploy workflows refuse to run when it is set for the target environment.

Rationale

CUR-1194 saw four revisions deployed in twelve hours via auto-replay and manual retries. Concurrency control would not have caused this incident but would prevent a future one — parallel rollback racing a forward deploy, two database-reset jobs racing on the same environment. A panic-stop signal is cheap to add and expensive when needed without it.

Assertions

- A. The Cloud Run deploy workflow SHALL declare a concurrency group keyed by the combination of service-name input and sponsor-env input, with cancel-in-progress set to false.
- B. The database-reset workflows SHALL declare a concurrency group keyed by sponsor-env input, with cancel-in-progress set to false.
- C. The Cloud Run deploy workflow SHALL fail the run at its first job step if the repository variable named DEPLOY_PAUSED_<ENV> is set to the string "true", where <ENV> is the uppercase target environment identifier.

End Deploy concurrency and safety locks | **Hash:** 7a20af36

CAL-OPS-deploy-cross-repo-image-trigger: Cross-repo image-refresh trigger

Level: ops | **Status:** Legacy | **Implements:** -

Overview

When the parent repository (hht_diary) publishes a new portal-server, diary-server, or sponsor-ci image, it dispatches a repository_dispatch event to this repository carrying the new image digest. The receiving workflow gates auto-rebuild and auto-deploy-to-dev on a label allowlist (e.g. auto-deploy-dev) on the originating pull request. Production environments never auto-promote from a parent-image refresh; they require explicit operator action. At build time, the callisto image-build workflow pins by SHA, never by mutable tag.

Rationale

Today `:main-latest` floats silently. The CUR-1194 image investigation surfaced the consequence: two callisto builds produced from the same source SHA can have different runtime behavior because their parent base-image digests changed between builds. The trigger gap also means dev environments drift behind parent main without anyone noticing until a manual deploy. Closing the trigger gap is also a prerequisite for the future cross-repo PR-label-gated auto-deploy described in the internal CI/CD overview.

Assertions

- A. The repository SHALL contain a workflow that listens for a `repository_dispatch` event of type `parent-image-published` from the `hht_diary` repository.
- B. When the `parent-image-published` workflow receives a dispatch event, the workflow SHALL trigger an image rebuild only if the originating pull request carries the label `auto-deploy-dev`.
- C. The `parent-image-published` workflow SHALL only auto-deploy to the dev environment; deploys to `qa`, `uat`, and `prod` SHALL require a separate manual trigger.
- D. The `image-build` workflow SHALL resolve every base-image input to an immutable digest before passing it to the Docker build step.

End Cross-repo image-refresh trigger | **Hash:** 88449aa6

CAL-OPS-deploy-audit-trail: Regulatory audit trail for deploys, secrets, and schema

Level: ops | **Status:** Legacy | **Implements:** -

Overview

An append-only audit trail exists for every Cloud Run deploy, every Doppler service-token rotation, and every database schema migration. Each entry carries a timestamp, the executor, the approver where the environment requires one, the originating pull request, the related ticket identifier, and the artifact identifier (image digest for deploys, token slug for rotations, migration identifier for schema). The trail is exportable as CSV scoped to a target environment and date range, and is retained for the regulatory retention window applicable to clinical-trial systems, parameterized by `audit_retention_years` and resolved from the deployment configuration set by the sponsor compliance officer.

Rationale

Cloud Audit Logs cover the raw events but are not structured to produce a regulator-ready record. A documented composer that joins deploy events, secret rotations, and schema changes back to PRs and tickets is required for FDA 21 CFR Part 11 traceability obligations.

Assertions

A. The repository SHALL contain a documented Cloud Logging query that produces a CSV of every Cloud Run deploy event for a given environment within a given date range.

B. The repository SHALL contain a documented Cloud Logging query that produces a CSV of every Doppler service-token rotation event for a given environment within a given date range.

C. The repository SHALL contain a documented Cloud Logging query that produces a CSV of every database schema-migration event for a given environment within a given date range.

D. Each row of every deployment-system audit CSV that the System produces SHALL contain a timestamp in ISO 8601 format, the executor identifier, the approver identifier when the target environment requires one, the originating pull-request URL, the related ticket identifier, and the relevant artifact identifier.

E. Cloud Logging retention for the projects `callisto4-uat` and `callisto4-prod` SHALL be configured to a minimum of `{{audit_retention_years}}` years.

End Regulatory audit trail for deploys, secrets, and schema | **Hash:** 560d57d5

CAL-OPS-deploy-runbooks-failure-modes: Runbooks and failure-mode documentation

Level: ops | **Status:** Legacy | **Implements:** -

Overview

Operational procedures live as runbooks under `deployment/runbooks/`. Each runbook covers a single procedure (normal deploy, hot-fix, rollback, database reset, secret rotation, token-leak response, sponsor onboarding, incident response) and links to the workflow it invokes, the relevant Cloud Logging queries, and the Slack channels for status updates. A separate failure-mode-coverage document enumerates each external dependency and the documented response when that dependency is unavailable.

Rationale

CUR-1187 introduced the per-env rotation automation but no incident-response runbook; the D-18 token leak inside that PR was handled ad-hoc. CUR-1194's diagnosis was lengthened by the absence of a "what to check when portal returns 503" runbook entry. Documented procedures make these handle-able by anyone on rotation, not just the original author.

Assertions

- A. The repository SHALL contain a runbook at `deployment/runbooks/deploy.md` covering the normal deploy procedure for portal-service and diary-service across the four target environments.
- B. The repository SHALL contain a runbook at `deployment/runbooks/rollback.md` covering the rollback procedure.
- C. The repository SHALL contain a runbook at `deployment/runbooks/database-reset.md` covering the destructive database-reset procedure.
- D. The repository SHALL contain a runbook at `deployment/runbooks/secret-rotation.md` covering the Doppler service-token rotation procedure.
- E. The repository SHALL contain a runbook at `deployment/runbooks/token-leak-response.md` covering the response procedure when a Doppler service-token value is exposed.
- F. The repository SHALL contain a runbook at `deployment/runbooks/sponsor-onboarding.md` covering the procedure for onboarding a new sponsor to the deployment system.
- G. The repository SHALL contain a runbook at `deployment/runbooks/incident-response.md` covering the on-call response procedure for production incidents.
- H. The repository SHALL contain an onboarding document at `deployment/runbooks/onboarding.md` that walks a new engineer through the access scopes granted at each environment tier.
- I. The repository SHALL contain a failure-mode-coverage document at `deployment/runbooks/failure-mode-coverage.md` that enumerates each external dependency (Doppler, GHCR, GAR, GitHub Actions, GCP region, Cloud SQL) and documents the response when that dependency is unavailable.
- J. Every runbook stored under `deployment/runbooks/` SHALL contain a section linking to the GitHub Actions workflow it invokes, the relevant Cloud Logging queries, and the Slack channels used for status updates.

End Runbooks and failure-mode documentation | **Hash:** 27b81f29

Development Requirements

3.1 Callisto Database Conventions

Implementation-level obligations governing how the Callisto deployment shapes data in its local databases. Defines state-transition rules, field-preservation invariants, response-storage format, and proxy-database scoping.

TODO (URS-Phase-3 reconciliation): Every REQ in this file originally Implements-ed one or more of: REQ-p00004 (Immutable Audit Trail), REQ-p00011 (ALCOA+ Data Integrity Principles), REQ-p00012 (Clinical Data Retention Requirements), REQ-p00013 (Complete Data Change History). All four parents are URS-replaced or deprecated. Re-parent each REQ to its URS-derived counterpart when those land.

CAL-DEV-database-minimize-deleted-records: Minimize Deleted Records

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

This requirement defines state transition rules for daily record management to optimize data storage and maintain clean audit trails. By reusing existing record UUIDs and transitioning states rather than creating new deleted records, the system reduces storage overhead while preserving complete event history through the event sourcing model. The state transition matrix ensures logical consistency from a user experience perspective while maintaining FDA 21 CFR Part 11 compliance through immutable event records.

Allowable State Transitions

From State	To: isDeleted	To: isNoNosebleed	To: isUnknown	To: Nosebleed
isDeleted	N/A	Yes	Yes	Yes
isNoNosebleed	Yes	N/A	Yes (a)	Yes
isUnknown	No (b)	Yes	N/A	Yes

Notes:

- (a) Requires providing user the option to directly interact with a “NoNosebleeds” event and change it to “Unknown” (e.g., “Actually, I don’t remember if that’s true”)
- (b) This transition does not make UX sense: changing “I don’t know” to “I don’t know whether it’s NoNosebleeds or Unknown” is effectively still “Unknown”

- (c) Cure HHT will maintain the append-only data model and ensure all state changes are event-based

Assertions

- A. The system SHALL interpret user actions during a given day to minimize the number of records marked with isDeleted status.
- B. The system SHALL allow records to transition from isDeleted state to isNoNosebleed state.
- C. The system SHALL allow records to transition from isDeleted state to isUnknown state.
- D. The system SHALL allow records to transition from isDeleted state to Nosebleed state.
- E. The system SHALL allow records to transition from isNoNosebleed state to isDeleted state.
- F. The system SHALL allow records to transition from isNoNosebleed state to isUnknown state when the user explicitly chooses to change a NoNosebleeds event to Unknown.
- G. The system SHALL allow records to transition from isNoNosebleed state to Nosebleed state.
- H. The system SHALL NOT allow records to transition from isUnknown state to isDeleted state.
- I. The system SHALL allow records to transition from isUnknown state to isNoNosebleed state.
- J. The system SHALL allow records to transition from isUnknown state to Nosebleed state.
- K. The system SHALL prevent state transitions that are not defined in the allowable state transition matrix at the application level.
- L. The system SHALL record all state changes as events in the EventStore.
- M. The system SHALL preserve the record UUID when transitioning between states.
- N. The system SHALL maintain a complete audit trail showing the history of all state transitions for each record.

End Minimize Deleted Records | **Hash:** 5bc520a6

CAL-DEV-database-no-implicit-field-changes: No Implicit Field Changes

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

This requirement ensures that audit trails remain complete and accurate by preventing unintended data loss during record state transitions. When a record undergoes a state change (such as deletion or modification), preserving all unmodified fields allows auditors and regulators to view the complete historical state of the record at each transition point. This approach supports FDA 21 CFR Part 11 compliance by maintaining tamper-evident records that show exactly what changed and what remained constant, rather than presenting sanitized or reconstructed versions of historical data.

Example: When a user deletes a nosebleed event:

- The `isDeleted` flag SHALL be set to `true`
- All other fields (Start Time, Severity, etc.) SHALL retain their values

Assertions

A. The system SHALL NOT modify any fields except those explicitly required to effect the user's requested action.

B. The system SHALL set the `isDeleted` flag to `true` when a user deletes an event.

C. The system SHALL retain all original field values when the `isDeleted` flag is set to `true`.

D. The system SHALL preserve unrelated field values during state transitions.

E. The audit trail SHALL record only those fields that actually changed during an operation.

F. The system SHALL NOT perform automatic cleanup or modification of fields during state changes.

End No Implicit Field Changes | **Hash:** e54e9d7a

CAL-DEV-database-survey-response-storage: Survey Response Storage Format

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

Plain-text storage ensures future readability of archival data for regulatory compliance and long-term preservation. Lookup table indices require maintaining the lookup table indefinitely and correctly correlating versions across system changes. Plain text is self-documenting and readable without additional context, supporting data interpretability after system decommissioning and during regulatory audits.

Assertions

- A. The system SHALL store survey response values for multiple choice questions as plain-text strings.
- B. The system SHALL NOT store survey response values as numeric indices into lookup tables.
- C. Survey response data SHALL be readable as human-readable text without requiring lookup tables.
- D. Survey response data SHALL be readable without requiring code or additional software context.
- E. Survey response data SHALL remain interpretable after system decommissioning.
- F. Data exports SHALL include response text values, not numeric indices.

End Survey Response Storage Format | **Hash:** 1701c36a

CAL-DEV-database-proxy-purpose: Proxy Database Purpose

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

Separating synchronization concerns from analysis ensures the proxy database remains focused on data transfer reliability and security. Analysis should be performed on dedicated systems with appropriate controls. This architectural separation supports system reliability, performance optimization, and security by preventing analytical workloads from interfering with time-sensitive synchronization operations.

Assertions

- A. The proxy database SHALL serve as an intermediary for synchronization operations only.
- B. The proxy database SHALL NOT be used for analytical purposes.
- C. The system SHALL NOT execute analytical queries against the proxy database.
- D. The proxy database schema SHALL be optimized for synchronization operations.
- E. Analysis operations SHALL be performed on separate database systems with appropriate controls.

End Proxy Database Purpose | **Hash:** a4a5f2f2

3.2 Callisto EDC Schema — Field Formats and Audit Conventions

Implementation-level obligations governing how the Callisto deployment shapes data for export to the sponsor's EDC (Rave). Defines field formats, audit-trail conventions, and transformation rules between the internal event model and the EDC-friendly schema.

TODO (URS-Phase-3 reconciliation): Every REQ in this file originally Implements-ed one or more of: REQ-p00004 (Immutable Audit Trail), REQ-p00011 (ALCOA+ Data Integrity Principles), REQ-p00012 (Clinical Data Retention Requirements), REQ-p00013 (Complete Data Change History). All four parents are URS-replaced or deprecated. Re-parent each REQ to its URS-derived counterpart when those land.

CAL-DEV-edc-uuid-field-format: UUID Field Format

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

UUID v7 provides chronological ordering while maintaining uniqueness across all tables in the system. The timestamp-based design allows changes to be ordered globally regardless of where they are recorded. The standard hyphenated format ensures human readability and compatibility with common UUID libraries and databases. Immutable identifiers ensure record traceability across modifications, supporting audit trail requirements for clinical trial systems.

Assertions

- A. The system SHALL use UUID v7 per IETF RFC 9562 for all id fields.
- B. UUID v7 values SHALL be created by allocating a Unix timestamp in milliseconds in the most significant 48 bits.
- C. UUID v7 values SHALL fill the remaining 74 bits, excluding the required version and variant bits, with random bits.
- D. When not stored as a binary number, UUIDs SHALL be represented in the standard format of 32 hexadecimal characters with four hyphens (xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx).
- E. The system SHALL assign a UUID v7 to all diary entries at creation time.
- F. The system SHALL NOT modify or change a UUID once assigned to a record.
- G. The system SHALL export UUIDs as 36-character hyphenated strings.

H. The system SHALL validate UUID format on creation.

I. UUIDs SHALL be chronologically sortable by their timestamp component.

End UUID Field Format | **Hash:** 0fef1aa4

CAL-DEV-edc-date-field-format: Date Field Format

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

Human-readable date format ensures long-term data accessibility without requiring date parsing logic for clinical trial data archival. Consistent format across all date fields simplifies processing and reduces implementation errors. This supports FDA 21 CFR Part 11 requirements for electronic record readability and long-term preservation.

Assertions

A. The system SHALL format all date fields (startDate, endDate, lastModifiedDate) as text in the format "DD MMM YYYY".

B. The system SHALL represent the day component as zero-padded two digits (01-31).

C. The system SHALL represent the month component as a three-letter uppercase abbreviation (JAN, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC).

D. The system SHALL represent the year component as four digits.

E. The system SHALL separate date format components with a single space character.

F. The system SHALL use identical formatting for all date fields without exception.

End Date Field Format | **Hash:** 58b2f49d

CAL-DEV-edc-time-field-format: Time Field Format

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

This requirement establishes a standard time format for the Callisto clinical trial system. The 24-hour format eliminates AM/PM ambiguity that could lead to data entry errors or misinterpretation of clinical event timing. Using text format rather than native time types ensures consistent representation across different database systems and client applications without timezone conversion issues. This standardization is critical for accurate temporal data analysis in multi-site clinical trials.

Assertions

- A. The system SHALL represent time fields (startTime, endTime, lastModifiedTime) as text in 24-hour format.
- B. Time field values SHALL follow the pattern "HH:MM" where HH represents hours and MM represents minutes.
- C. The hours component SHALL be zero-padded to exactly two digits with valid range 00-23.
- D. The minutes component SHALL be zero-padded to exactly two digits with valid range 00-59.
- E. The system SHALL use a colon (':') as the separator between hours and minutes.
- F. All time fields within the system SHALL use identical formatting.

End Time Field Format | **Hash:** de99e024

CAL-DEV-edc-duration-calculation: Duration Field Calculation

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

Standardized duration calculation ensures consistency across the clinical trial system. A minimum duration of 1 minute prevents zero-duration records that could complicate data analysis. Even rounding (banker's rounding) follows standard statistical practice to minimize cumulative rounding bias in large datasets.

Assertions

- A. The system SHALL calculate the duration field as (endTime - startTime) in minutes.
- B. The system SHALL express duration values in whole minutes.
- C. The system SHALL enforce a minimum duration value of 1 minute.
- D. The system SHALL round durations less than 1 minute to 1 minute.
- E. The system SHALL round seconds to the nearest even minute using banker's rounding.
- F. The system SHALL recalculate the duration field when the start time is modified.
- G. The system SHALL recalculate the duration field when the end time is modified.

End Duration Field Calculation | **Hash:** 0c76ed17

CAL-DEV-edc-severity-field-format: Severity Field Format

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

Plain-text severity values are self-documenting and readable without lookup tables, supporting long-term data archival requirements. This approach ensures that data remains interpretable over time without requiring access to external code mappings or lookup tables, which is critical for regulatory compliance and long-term clinical data preservation.

Assertions

- A. The system SHALL store the severity field as literal English text of the severity level.
- B. The system SHALL transmit the severity field as literal English text of the severity level.
- C. The system SHALL NOT use numeric codes to represent severity values.
- D. The system SHALL validate severity values against a predefined list of allowed text values.
- E. The system SHALL export severity data as text values, not as numeric indices.
- F. Severity field values SHALL be formatted in all lowercase characters.
- G. The allowed severity values SHALL be: "trivial", "mild", "moderate", "severe", and "steady-stream".
- H. The "steady-stream" severity value SHALL be hyphenated as a single compound word.

End Severity Field Format | **Hash:** 9bfdd635

CAL-DEV-edc-notes-field-handling: Notes Field Handling

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

Free-text notes may inadvertently contain PHI (Protected Health Information). Using coded justifications maintains audit trail requirements while protecting patient privacy and ensuring compliance with data protection regulations. This approach allows internal tracking of modification reasons while preventing exposure of sensitive information to external Sponsor EDC systems.

Assertions

- A. The system SHALL NOT synchronize the notes field to the Sponsor EDC.
- B. The system SHALL replace notes fields with coded justifications to avoid PHI exposure.
- C. The system SHALL use predefined justification codes for notes when required for old entry modifications.
- D. The system SHALL NOT use free text for justifications.
- E. The system SHALL NOT synchronize any free-text fields to the EDC.
- F. The system SHALL store justification reasons as codes.
- G. The internal audit trail SHALL contain justification codes.
- H. The system SHALL maintain a mapping from justification codes to descriptions separately from synchronized data.

End Notes Field Handling | **Hash:** c162113d

CAL-DEV-edc-no-nosebleeds-flag: No Nosebleeds Event Flag

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

This requirement distinguishes between three states in nosebleed recording: explicit confirmation of no nosebleeds, presence of nosebleeds, and missing/incomplete data. This distinction is critical for clinical trial data quality, as it allows researchers to differentiate between days where patients confirmed no symptoms versus days where data was not recorded. This supports accurate assessment of patient compliance and symptom patterns in HHT clinical trials.

Assertions

- A. The system SHALL implement the isNoNosebleedsEvent field as a boolean data type.
- B. The system SHALL set isNoNosebleedsEvent to true when the user explicitly selects that there were no nosebleeds that day.
- C. The system SHALL set isNoNosebleedsEvent to false as the default value when the field is not applicable.
- D. The system SHALL NOT allow isNoNosebleedsEvent to be true on the same day as actual nosebleed events are recorded.

End No Nosebleeds Event Flag | **Hash:** c278cf17

CAL-DEV-edc-unknown-event-flag: Unknown Event Flag**Level:** dev | **Status:** Legacy | **Implements:** -**Rationale**

Distinguishing “unknown” from “no entry” supports data completeness analysis and demonstrates patient engagement with the diary. The system encourages at least one entry per day, allowing users to explicitly indicate when they cannot recall information rather than leaving the diary incomplete.

Assertions

- A. The system SHALL implement an isUnknownEvent field as a boolean data type.
- B. The isUnknownEvent field SHALL be set to true when the user explicitly selects that they do not recall if there were any nosebleeds that day.
- C. The isUnknownEvent field SHALL be set to false when the user does not select the “unknown/don’t recall” option.
- D. The isUnknownEvent field SHALL default to false when the field is not applicable.
- E. The isUnknownEvent flag SHALL be mutually exclusive with other day status flags.

End Unknown Event Flag | **Hash:** 40380c25**CAL-DEV-edc-data-source-trace: Data Source Traceability****Level:** dev | **Status:** Legacy | **Implements:** -**Rationale**

Data source tracking supports audit trail requirements for FDA 21 CFR Part 11 compliance and enables debugging of platform-specific issues. Recording the device type and software version provides traceability for data provenance, which is essential for identifying and resolving technical issues that may be specific to certain platforms or application versions.

Assertions

- A. The system SHALL record the type of device/software used to input data in the dataSource field.
- B. The system SHALL record the software version used to input data in the dataSource field.

- C. The dataSource field SHALL include platform type at minimum (iOS/Android).
 - D. The dataSource field SHALL include application version.
 - E. The dataSource field format SHALL be consistent across all data entries.
 - F. The dataSource field format SHALL be parseable for programmatic extraction.
 - G. The system SHALL populate the dataSource field automatically without user input.
- End Data Source Traceability* | **Hash:** 5efe8e1c

CAL-DEV-edc-incomplete-entry-flag: Incomplete Entry Flag

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

Tracking incomplete entries enables targeted reminders and data quality monitoring. This requirement ensures users are prompted to complete partially-entered nosebleed events, improving data completeness for clinical trial accuracy.

Assertions

- A. The system SHALL implement isIncomplete as a boolean field.
- B. The system SHALL set isIncomplete to true when a user creates a nosebleed event without completing all required fields.
- C. The system SHALL set isIncomplete to true when any required field is missing from a nosebleed entry.
- D. The system SHALL generate reminders to the user when isIncomplete is true.
- E. The system SHALL continue to remind the user to complete the entry while isIncomplete remains true.
- F. The system SHALL set isIncomplete to false when all required fields are populated.
- G. The system SHALL update the isIncomplete flag automatically as fields are completed.

End Incomplete Entry Flag | **Hash:** b1e9f94f

CAL-DEV-edc-change-history-via-eventstore: Change History via EventStore

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

Event sourcing provides a complete, immutable audit trail by design. Maintaining a separate changeHistory field would create redundancy and introduce the risk of inconsistency between the field and the EventStore. This approach ensures a single source of truth for all historical data while meeting FDA 21 CFR Part 11 requirements for audit trails and data integrity.

Assertions

- A. The system SHALL NOT include a changeHistory field in the synchronized schema.
- B. The system SHALL derive all change history exclusively from the EventStore.
- C. The EventStore SHALL provide a complete audit trail for all data modifications.
- D. The system SHALL enable reconstruction of historical states from stored events.

End Change History via EventStore | **Hash:** d9ee256c

CAL-DEV-edc-requester-id: Requester ID Field

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

Tracking the survey initiator supports audit trail and accountability requirements for clinical trial data. This field enables traceability between survey records and the authenticated portal users who created them, which is essential for regulatory compliance and data integrity validation.

Assertions

- A. The system SHALL include a requesterID field for all survey records.
- B. The requesterID field SHALL indicate which person initiated the survey.
- C. The requesterID field SHALL identify the portal user who initiated the survey.
- D. The requesterID SHALL be traceable to an authenticated user.
- E. The requesterID field SHALL be required for all survey records.
- F. The requesterID format SHALL be consistent with the user ID scheme.
- G. The system SHALL NOT allow surveys to be initiated from within the app.

End Requester ID Field | **Hash:** 45bef36a

CAL-DEV-edc-requester-name: Requester Name Field**Level:** dev | **Status:** Legacy | **Implements:** -**Rationale**

Human-readable names alongside structured identifiers support long-term readability of archived data, particularly when user accounts may be modified or deactivated. This practice aligns with FDA 21 CFR Part 11 requirements for maintaining readable and accessible electronic records throughout their retention period, ensuring that archived survey data remains interpretable without requiring active system lookups.

Assertions

- A. The system SHALL store the requesterName field as plain text.
- B. The requesterName field SHALL contain the name corresponding to the requesterID at the time of survey creation.
- C. The requesterName field SHALL be readable without requiring system lookup of the requesterID.

End Requester Name Field | **Hash:** 95edc730**CAL-DEV-edc-survey-name: Survey Name Field****Level:** dev | **Status:** Legacy | **Implements:** -**Rationale**

This requirement defines the data type and validation rules for the surveyName field, which identifies the type of survey instrument being administered. This field enables consistent survey categorization and supports data analysis by allowing grouping and filtering of responses by survey type. Proper identification of survey instruments is essential for clinical trial data integrity and regulatory compliance.

Assertions

- A. The surveyName field SHALL be a string data type.
- B. The surveyName field SHALL have a defined maximum length.
- C. The surveyName field SHALL identify the survey instrument used.
- D. Survey naming SHALL be consistent across all instances of the same survey type.

End Survey Name Field | **Hash:** 0e4fa43a

CAL-DEV-edc-survey-version: Survey Version Field**Level:** dev | **Status:** Legacy | **Implements:** -**Rationale**

Version tracking ensures correct interpretation of responses when survey instruments are updated. As survey instruments evolve over time, the version field provides a critical link between response data and the survey definition that was in use when the response was collected. This enables accurate historical data interpretation and maintains data integrity across system boundaries (application, proxy, and sponsor EDC).

Assertions

- A. The surveyVersion field SHALL be an integer data type.
- B. The surveyVersion field SHALL have a minimum value of 1.
- C. The surveyVersion field SHALL NOT accept values of 0 or negative integers.
- D. The system SHALL increment the surveyVersion when any change is made to the survey instrument.
- E. The surveyVersion value SHALL be consistent across the application schema, proxy schema, and Sponsor EDC for the same survey.
- F. The system SHALL associate each response record with the specific surveyVersion that was active when the response was collected.
- G. The system SHALL enable interpretation of historical responses using the stored surveyVersion identifier.

End Survey Version Field | **Hash:** 3d9457ea**CAL-DEV-edc-survey-response-storage: Survey Response Storage****Level:** dev | **Status:** Legacy | **Implements:** -**Rationale**

This requirement ensures long-term readability and regulatory compliance of clinical trial survey data. FDA 21 CFR Part 11 requires that electronic records remain readable and accessible throughout their retention period. Storing survey responses as human-readable text eliminates dependencies on lookup tables, code mappings, or specialized software that may become unavailable over time. This approach supports both archival requirements and data migration scenarios where original database structures may not be preserved.

Assertions

- A. The system SHALL store survey responses as text values.
- B. The system SHALL implement text storage using either pre-defined fields (one per question) or structured data format (JSON) with full-text question and answer fields.
- C. Survey responses SHALL be stored in human-readable text format.
- D. The system SHALL preserve the full question text with each corresponding answer.
- E. Survey responses SHALL be readable without requiring lookup tables or code interpretation.
- F. The storage format SHALL maintain consistency with the App and ProxyDB implementations.

End Survey Response Storage | **Hash:** 2f4e6520

CAL-DEV-edc-uuid-table-prefix: UUID Table Prefix Format

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

Table-prefixed UUIDs provide immediate context about the record's origin, simplifying debugging, logging, and cross-table reference resolution without requiring database lookups. This convention is optional but enables developers to identify record sources at a glance in logs, error messages, and API responses without consulting the database schema.

Assertions

- A. The system SHALL support UUIDs with an optional table name prefix using the format {table_name}_{uuid}.
- B. The system SHALL use an underscore character as the separator between table name prefix and UUID.
- C. When a table prefix is used, the prefix SHALL match the actual table name containing the record.
- D. The system SHALL accept both prefixed and non-prefixed UUID formats in all operations.
- E. The system SHALL strip the table prefix before performing database operations on prefixed UUIDs.

End UUID Table Prefix Format | **Hash:** 189341f5

CAL-DEV-edc-site-id-format: Site ID Format**Level:** dev | **Status:** Legacy | **Implements:** -**Rationale**

Standardized site identifiers enable consistent identification across systems while supporting hierarchical sub-site structures. ISO country codes ensure international compatibility and facilitate global clinical trial operations. This requirement provides the technical specification for site identifier formatting in development and data validation contexts.

Assertions

- A. Site identifiers SHALL follow the format CCC-SSS[a-z] where CCC is the country code, SSS is the site number, and [a-z] is an optional sub-site designator.
- B. The country code component SHALL be a valid ISO 3166-1 numeric country code.
- C. The country code component SHALL be exactly 3 digits with leading zeroes as needed.
- D. The site number component SHALL be exactly 3 digits with leading zeroes as needed.
- E. The optional sub-site designator SHALL be a single lowercase letter from a through z.
- F. Site identifiers SHALL NOT exceed 8 characters in total length.
- G. The system SHALL validate site identifier format on input.
- H. Site identifiers with sub-site designators SHALL be exactly 8 characters (e.g., 840-001a).
- I. Site identifiers without sub-site designators SHALL be exactly 7 characters (e.g., 840-001).

End Site ID Format | **Hash:** f9398120

CAL-DEV-edc-patient-id-format: Patient ID Format**Level:** dev | **Status:** Legacy | **Implements:** -**Rationale**

Immutable patient IDs ensure traceability throughout the trial lifecycle and maintain audit trail integrity even when patients transfer between sites. The structured format embeds the original enrollment location, enabling regulatory compliance and data provenance tracking across multi-site clinical trials.

Assertions

- A. The system SHALL generate patient identifiers using the format CCC-SSS[a-z]-NNN where CCC is the ISO 3166-1 numeric country code, SSS is the enrollment site number, [a-z] is an optional sub-site designator, and NNN is a sequential patient number.
- B. The system SHALL include the original enrollment site in the patient identifier.
- C. The system SHALL NOT change the Site ID component of the patient identifier when a patient transfers to a different site.
- D. The system SHALL NOT allow modification of a patient identifier after assignment.
- E. The country code component SHALL be exactly 3 digits with leading zeroes.
- F. The enrollment site number component SHALL be exactly 3 digits with leading zeroes.
- G. The sub-site designator component SHALL be a single lowercase letter when present.
- H. The patient number component SHALL be exactly 3 digits with leading zeroes.
- I. The patient number SHALL be unique within the enrollment site.
- J. The complete patient identifier SHALL NOT exceed 12 characters in length.
- K. The system SHALL validate the patient identifier format upon creation.

End Patient ID Format | **Hash:** d28af884

CAL-DEV-edc-audit-context-structure: Audit Context Field Structure

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

Complete audit metadata ensures FDA 21 CFR Part 11 compliance by capturing who made each change, from where, when, and why. The AuditRecord structure within the Context ItemData provides the standardized format for transmitting audit trail information to the RAVE EDC system, enabling tamper-evident record keeping of all data modifications.

Assertions

- A. The system SHALL include an AuditRecord element within the Context ItemData for each transaction to the RAVE EDC.
- B. The AuditRecord element SHALL set the EditPoint attribute to "DataManagement".
- C. The AuditRecord element SHALL contain a UserRef child element with a UserID attribute identifying the user performing the transaction.

D. The AuditRecord element SHALL contain a LocationRef child element with a LocationOID attribute identifying the location of the transaction.

E. The AuditRecord element SHALL contain a DateTimeStamp child element with the timestamp of the transaction.

F. The AuditRecord element SHALL contain a ReasonForChange child element with the reason for the data modification.

G. The XML structure of the Context ItemData and AuditRecord elements SHALL validate against the RAVE EDC schema.

End Audit Context Field Structure | **Hash:** cff75d38

CAL-DEV-edc-audit-user-values: Audit User Field Values

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

Distinguishing patient-initiated changes from automated system changes supports audit trail analysis and regulatory compliance. This requirement standardizes the UserID field values to enable clear identification of the source of data changes in the audit trail.

Assertions

A. The UserID field SHALL contain exactly one of two standardized values: "patient" or "automation".

B. The system SHALL use the value "patient" in the UserID field for all events resulting from patient data entry.

C. The system SHALL use the value "automation" in the UserID field for all events resulting from system-initiated changes.

D. The system SHALL NOT permit any values other than "patient" or "automation" in the UserID field.

End Audit User Field Values | **Hash:** a54dee5a

CAL-DEV-edc-audit-location-format: Audit Location Field Format

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

Tracking the specific source system and installation enables debugging, audit analysis, and identification of platform-specific issues. This requirement defines the standardized format for the LocationOID field to distinguish between patient-facing applications (iOS, Android, Web) and backend automation systems (mobile backend, portal backend).

Assertions

- A. The LocationRef LocationOID field SHALL identify the source system that generated the data change.
- B. The LocationRef LocationOID field SHALL identify the installation that generated the data change.
- C. The system SHALL use the format 'iOS- $\{x\}$ ' for LocationOID values when the source is an iOS app installation, where $\{x\}$ is the unique identifier of that app installation.
- D. The system SHALL use the format 'Android- $\{x\}$ ' for LocationOID values when the source is an Android app installation, where $\{x\}$ is the unique identifier of that app installation.
- E. The system SHALL use the format 'Web- $\{x\}$ ' for LocationOID values when the source is a web application instance, where $\{x\}$ is the unique identifier of that web application instance.
- F. The system SHALL use the format 'mobile- $\{y\}$ ' for LocationOID values when the source is automation from the mobile backend subsystem, where $\{y\}$ is the name of the subsystem generating the event.
- G. The system SHALL use the format 'portal- $\{y\}$ ' for LocationOID values when the source is automation from the portal backend subsystem, where $\{y\}$ is the name of the subsystem generating the event.
- H. The installation identifier $\{x\}$ SHALL be unique per app installation.
- I. The location format SHALL match the source type (iOS/Android/Web for apps, mobile/portal for backend systems).
- J. The subsystem name $\{y\}$ SHALL accurately identify the backend component generating the event.
- K. The LocationOID format SHALL be consistent across all transactions from the same source.

End Audit Location Field Format | **Hash:** 39b68d8e

CAL-DEV-edc-audit-timestamp-format: Audit Timestamp Format

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

ISO 8601 format with timezone offset ensures unambiguous timestamp interpretation regardless of the reader's locale. The timezone offset preserves the user's local context at the time of the action, which is critical for audit trail integrity in multi-site clinical trials where users may operate in different time zones. This supports FDA 21 CFR Part 11 requirements for accurate and complete audit trails.

Assertions

- A. The DateTimeStamp field SHALL contain the timestamp in ISO 8601 format with timezone offset.
- B. The DateTimeStamp format SHALL be YYYY-MM-DDTHH:MM:SS±HH:MM.
- C. The DateTimeStamp SHALL include the timezone offset using ±HH:MM notation.
- D. The DateTimeStamp SHALL NOT use Z notation for timezone.
- E. The DateTimeStamp SHALL reflect the user's action time, not server receipt time.
- F. The DateTimeStamp SHALL include seconds precision and SHALL NOT truncate to minutes.

End Audit Timestamp Format | **Hash:** b7604c5a

CAL-DEV-edc-audit-reason-values: Audit Reason Field Values

Level: dev | **Status:** Legacy | **Implements:** -

The ReasonForChange field SHALL describe the cause of the data change.

Allowed Values:

- user input - Direct user data entry or modification
- {automation description} - Description of the automated process that triggered the change

Examples:

- user input
- scheduled sync reconciliation
- duplicate record merge
- data validation correction

Rationale: Documenting the reason for each change supports audit trail analysis and helps distinguish intentional user changes from system-initiated modifications.

Assertions

- A. Patient-initiated changes SHALL include a Reason of “User Input”
- B. Automation-initiated changes SHALL include a Reason
- C. Reason text SHALL be human-readable
- D. Reason SHALL accurately reflect the cause of the change

End Audit Reason Field Values | **Hash:** 8ce83605

CAL-DEV-edc-callisto-transforms: Callisto Transformation Rules

Level: dev | **Status:** Legacy | **Implements:** -

Assertions

- A. The system SHALL support sponsor-specific data transformation rules.
- B. Transformation rules SHALL be stored in a versioned, sponsor-specific configuration.
- C. Transformation rules SHALL be applied after validation but before EDC synchronization.
- D. Transformation rules SHALL be documented with their business rationale.
- E. Transformation rules SHALL NOT modify the original user-entered data stored locally.
- F. Transformation rules SHALL NOT create data that would fail EDC validation.
- G. Transformation rules SHALL NOT operate silently without audit trail.
- H. The transformation process SHALL log the original value, transformed value, rule applied, and timestamp.

Rationale

Some EDC requirements involve business logic that cannot be expressed in declarative schemas. Separating these rules into explicit, versioned configuration ensures they are maintainable, auditable, and clearly distinguished from schema-derived validation.

End Callisto Transformation Rules | **Hash:** 9d3d0997

3.3 CAL-DEV-push-notifications-fcm-delivery: Push Notification Delivery via FCM

Level: dev | **Status:** Legacy | **Implements:** -

Rationale

Questionnaire assignment is a time-sensitive clinical workflow: patients need prompt notification so they can complete instruments within the study protocol window. FCM provides a cross-platform delivery channel for both iOS and Android. Because this is a clinical trial system, every notification attempt must be auditable and delivery failures must be handled predictably. The payload is kept to identifiers only — no clinical content — to limit exposure if a notification is intercepted or logged by the OS.

TODO (URS-Phase-3 reconciliation): Original Implements targets REQ-CAL-p00023, REQ-CAL-p00082 (callisto's *prd-portal-/prd-app-*), and REQ-p00011 (*hht_diary ALCOA+*) are all URS-replaced or deprecated. Re-parent to URS-derived notification/portal REQs when those land.

Assertions

- A. The portal backend SHALL send a push notification via Firebase Cloud Messaging when a Study Coordinator sends a questionnaire to a patient per REQ-CAL-p00023-D.
- B. The notification payload SHALL include the questionnaire record identifier and questionnaire type so the mobile app can identify the assigned questionnaire.
- C. The notification payload SHALL NOT contain Protected Health Information, questionnaire content, or patient-identifying data beyond what FCM requires for routing.
- D. The mobile app SHALL register its FCM device token with the portal backend on first launch and on each token refresh.
- E. The portal backend SHALL store the most recent FCM device token for each registered patient device.
- F. The portal backend SHALL invalidate a stored device token when FCM reports it as unregistered or invalid.
- G. The portal backend SHALL retry delivery up to 3 times with exponential backoff when FCM returns a transient error.
- H. The portal backend SHALL NOT retry delivery when FCM returns a permanent error such as invalid token or unregistered device.
- I. The system SHALL log each notification dispatch attempt in the audit trail, including timestamp, patient identifier, questionnaire record identifier, delivery outcome, and failure reason when applicable.

J. A failed push notification SHALL NOT prevent the questionnaire from being assigned to the patient; the questionnaire SHALL still appear when the mobile app next synchronizes.

End Push Notification Delivery via FCM | **Hash:** 51613c24

Topic Index

account: [DIARY-GUI-administrator-dashboard](#), [DIARY-GUI-user-account-deactivate](#), [DIARY-GUI-user-account-reactivate](#), [DIARY-GUI-user-information-modal](#), [DIARY-GUI-user-management-tabs](#), [DIARY-PRD-reason-field-constraints](#), [DIARY-PRD-user-account-activation-resend](#), [DIARY-PRD-user-account-activation-workflow](#), [DIARY-PRD-user-account-create](#), [DIARY-PRD-user-account-deactivate](#), [DIARY-PRD-user-account-edit](#), [DIARY-PRD-user-account-reactivate](#), [DIARY-PRD-user-account-site-assignment](#)

administrator: [CAL-PRD-administrator-settings-inventory](#), [DIARY-GUI-administrator-settings](#), [DIARY-PRD-administrator-settings](#)

app: [CAL-PRD-application-lock-configuration](#), [CAL-PRD-privacy-policy-configuration](#), [DIARY-PRD-application-lock](#), [DIARY-PRD-diary-start-day](#), [DIARY-PRD-mobile-application](#), [DIARY-PRD-mobile-offline-first](#), [DIARY-PRD-privacy-policy](#)

audit: [CAL-PRD-audit-log-view-scope](#), [DIARY-GUI-audit-log-administrator](#), [DIARY-GUI-audit-log-common](#), [DIARY-GUI-audit-log-cra](#), [DIARY-GUI-audit-log-study-coordinator](#)

auth: [CAL-PRD-login-rate-limiting-configuration](#), [CAL-PRD-two-factor-authentication-configuration](#), [DIARY-GUI-password-forgot-workflow](#), [DIARY-PRD-password-forgot](#), [DIARY-PRD-password-requirements](#), [DIARY-PRD-session-management](#), [DIARY-PRD-two-factor-authentication](#)

backup: [DIARY-PRD-data-backup-and-archival](#)

behavior: [DIARY-PRD-notification-behavior](#)

calculation: [DIARY-PRD-questionnaire-score-calculation](#), [DIARY-PRD-score-hht-qol](#), [DIARY-PRD-score-nose-hht](#)

ci: [CAL-OPS-ci-change-appropriate-validation](#), [CAL-OPS-ci-code-quality-static-analysis](#), [CAL-OPS-ci-secret-vulnerability-scanning](#), [CAL-OPS-ci-traceability-pipeline](#)

configuration: [DIARY-PRD-configuration-precedence](#)

data: [DIARY-PRD-data-backup-and-archival](#)

database: [CAL-DEV-database-minimize-deleted-records](#), [CAL-DEV-database-no-implicit-field-changes](#), [CAL-DEV-database-proxy-purpose](#), [CAL-DEV-database-survey-response-storage](#)

deployment: [CAL-OPS-deploy-audit-trail](#), [CAL-OPS-deploy-cloud-run-verification](#), [CAL-OPS-deploy-concurrency-locks](#), [CAL-OPS-deploy-cross-repo-image-trigger](#), [CAL-OPS-deploy-notification](#), [CAL-OPS-deploy-notification-dev](#), [CAL-OPS-deploy-notification-prod](#), [CAL-OPS-deploy-notification-qa](#), [CAL-OPS-deploy-notification-uat](#), [CAL-OPS-deploy-permission](#), [CAL-OPS-deploy-permission-dev](#), [CAL-OPS-deploy-permission-prod](#), [CAL-OPS-deploy-permission-qa](#), [CAL-OPS-deploy-permission-uat](#), [CAL-OPS-deploy-provenance-traceability](#), [CAL-OPS-deploy-rollback-canary](#), [CAL-OPS-deploy-runbooks-failure-modes](#), [CAL-OPS-secrets-doppler-cloud-run](#), [CAL-OPS-secrets-github-actions-routing](#), [CAL-OPS-secrets-sponsor-agnostic-automation](#)

device: CAL-PRD-linking-code-entry-errors-configuration, DIARY-GUI-join-study-screen, DIARY-GUI-linking-confirmation, DIARY-PRD-linking-code-entry-errors

diary: CAL-PRD-entry-duration-check-configuration, CAL-PRD-entry-time-restrictions-configuration, DIARY-GUI-entry-overlap-resolution, DIARY-PRD-entry-duration-check, DIARY-PRD-entry-overlap-resolution, DIARY-PRD-entry-time-restrictions

display: CAL-PRD-questionnaire-score-display-prohibition

edc: CAL-DEV-edc-audit-context-structure, CAL-DEV-edc-audit-location-format, CAL-DEV-edc-audit-reason-values, CAL-DEV-edc-audit-timestamp-format, CAL-DEV-edc-audit-user-values, CAL-DEV-edc-callisto-transforms, CAL-DEV-edc-change-history-via-eventstore, CAL-DEV-edc-data-source-trace, CAL-DEV-edc-date-field-format, CAL-DEV-edc-duration-calculation, CAL-DEV-edc-incomplete-entry-flag, CAL-DEV-edc-no-nosebleeds-flag, CAL-DEV-edc-notes-field-handling, CAL-DEV-edc-patient-id-format, CAL-DEV-edc-requester-id, CAL-DEV-edc-requester-name, CAL-DEV-edc-severity-field-format, CAL-DEV-edc-site-id-format, CAL-DEV-edc-survey-name, CAL-DEV-edc-survey-response-storage, CAL-DEV-edc-survey-version, CAL-DEV-edc-time-field-format, CAL-DEV-edc-unknown-event-flag, CAL-DEV-edc-uuid-field-format, CAL-DEV-edc-uuid-table-prefix

entry: CAL-PRD-entry-duration-check-configuration, CAL-PRD-entry-time-restrictions-configuration, DIARY-GUI-entry-overlap-resolution, DIARY-PRD-entry-duration-check, DIARY-PRD-entry-overlap-resolution, DIARY-PRD-entry-time-restrictions

epistaxis: DIARY-GUI-epistaxis-delete, DIARY-GUI-epistaxis-record, DIARY-PRD-epistaxis-capture-standard

evidence: DIARY-PRD-evidence-bitcoin-timestamp, DIARY-PRD-evidence-device-fingerprint, DIARY-PRD-evidence-email-identity, DIARY-PRD-evidence-geolocation, DIARY-PRD-evidence-patient-authentication, DIARY-PRD-evidence-timestamp-archival, DIARY-PRD-evidence-timestamp-attestation, DIARY-PRD-evidence-timestamp-verification

help: DIARY-PRD-help-resources

linking: CAL-PRD-linking-code-entry-errors-configuration, DIARY-GUI-join-study-screen, DIARY-GUI-linking-confirmation, DIARY-PRD-linking-code-entry-errors

log: CAL-PRD-audit-log-view-scope, DIARY-GUI-audit-log-administrator, DIARY-GUI-audit-log-common, DIARY-GUI-audit-log-cra, DIARY-GUI-audit-log-study-coordinator

management: CAL-GUI-manage-questionnaires-modal, CAL-GUI-questionnaire-finalization-workflow, CAL-GUI-trial-start-workflow, CAL-PRD-nose-hht-qol-workflow, CAL-PRD-questionnaire-cycle-tracking, CAL-PRD-questionnaire-session-resume-timeout, CAL-PRD-trial-start-workflow, DIARY-PRD-architecture-decision-docs, DIARY-PRD-questionnaire-change-control, DIARY-PRD-questionnaire-sponsor-configuration, DIARY-PRD-questionnaire-submission-control, DIARY-PRD-questionnaire-system, DIARY-PRD-system-validation-traceability

mobile: CAL-PRD-application-lock-configuration, CAL-PRD-notification-historical-gap-configuration, CAL-PRD-notification-incomplete-record-lock-configuration, CAL-PRD-notification-reminder-configuration, CAL-PRD-privacy-policy-configuration, DIARY-GUI-calendar-day-view, DIARY-GUI-main-screen-layout, DIARY-GUI-mobile-navigation, DIARY-GUI-participant-task-list, DIARY-GUI-participation-status-badge, DIARY-PRD-application-lock, DIARY-PRD-diary-start-day, DIARY-PRD-mobile-application, DIARY-PRD-mobile-offline-first, DIARY-PRD-notification-disconnection, DIARY-PRD-notification-historical-gap, DIARY-

PRD-notification-incomplete-record-lock, DIARY-PRD-notification-ongoing-epistaxis, DIARY-PRD-notification-portal-sent-questionnaire, DIARY-PRD-notification-yesterday-entry, DIARY-PRD-privacy-policy

navigation: DIARY-GUI-calendar-day-view, DIARY-GUI-main-screen-layout, DIARY-GUI-mobile-navigation

notification: DIARY-PRD-notification-behavior

notifications: CAL-DEV-push-notifications-fcm-delivery, CAL-PRD-notification-historical-gap-configuration, CAL-PRD-notification-incomplete-record-lock-configuration, CAL-PRD-notification-reminder-configuration, DIARY-GUI-participant-task-list, DIARY-GUI-participation-status-badge, DIARY-PRD-notification-disconnection, DIARY-PRD-notification-historical-gap, DIARY-PRD-notification-incomplete-record-lock, DIARY-PRD-notification-ongoing-epistaxis, DIARY-PRD-notification-portal-sent-questionnaire, DIARY-PRD-notification-yesterday-entry

operations: DIARY-PRD-platform-operations-monitoring

overview: DIARY-PRD-questionnaire-daily-epistaxis, DIARY-PRD-questionnaire-hht-qol, DIARY-PRD-questionnaire-nose-hht

participant: CAL-GUI-participant-dashboard-configuration, CAL-GUI-questionnaire-call-back-participant, CAL-PRD-linking-code-lifecycle-configuration, CAL-PRD-participant-disconnection-reason-options, CAL-PRD-participant-not-participating-reason-options, CAL-PRD-participant-registration-configuration, DIARY-GUI-participant-dashboard, DIARY-GUI-questionnaire-portal-sent-workflow, DIARY-GUI-questionnaire-session-expiry, DIARY-PRD-linking-code-lifecycle, DIARY-PRD-participant-disconnection, DIARY-PRD-participant-link-new, DIARY-PRD-participant-mark-not-participating, DIARY-PRD-participant-reactivate, DIARY-PRD-participant-reconnection, DIARY-PRD-participant-registration, DIARY-PRD-questionnaire-portal-sent-rules, DIARY-PRD-questionnaire-session-timeout

platform: DIARY-PRD-ancillary-platform-services, DIARY-PRD-platform-operations-monitoring

portal: CAL-PRD-login-rate-limiting-configuration, CAL-PRD-two-factor-authentication-configuration, DIARY-GUI-password-forgot-workflow, DIARY-PRD-password-forgot, DIARY-PRD-password-requirements, DIARY-PRD-session-management, DIARY-PRD-two-factor-authentication

push: CAL-DEV-push-notifications-fcm-delivery

questionnaire: CAL-GUI-manage-questionnaires-modal, CAL-GUI-questionnaire-call-back-participant, CAL-GUI-questionnaire-finalization-workflow, CAL-GUI-trial-start-workflow, CAL-PRD-nose-hht-qol-workflow, CAL-PRD-questionnaire-cycle-tracking, CAL-PRD-questionnaire-score-display-prohibition, CAL-PRD-questionnaire-session-resume-timeout, CAL-PRD-trial-start-workflow, DIARY-GUI-questionnaire-portal-sent-workflow, DIARY-GUI-questionnaire-session-expiry, DIARY-PRD-questionnaire-change-control, DIARY-PRD-questionnaire-daily-epistaxis, DIARY-PRD-questionnaire-hht-qol, DIARY-PRD-questionnaire-localization, DIARY-PRD-questionnaire-nose-hht, DIARY-PRD-questionnaire-portal-sent-rules, DIARY-PRD-questionnaire-session-timeout, DIARY-PRD-questionnaire-sponsor-configuration, DIARY-PRD-questionnaire-sponsor-eligibility, DIARY-PRD-questionnaire-submission-control, DIARY-PRD-questionnaire-system, DIARY-PRD-questionnaire-versioning

rbac: CAL-PRD-permissions-table, CAL-PRD-site-view-scope, DIARY-GUI-role-switching, DIARY-PRD-action-inventory, DIARY-PRD-rbac-customizable, DIARY-PRD-role-definitions

records: DIARY-PRD-evidence-bitcoin-timestamp, DIARY-PRD-evidence-device-fingerprint, DIARY-PRD-evidence-email-identity, DIARY-PRD-evidence-geolocation, DIARY-PRD-evidence-patient-authentication, DIARY-PRD-evidence-timestamp-archival, DIARY-PRD-evidence-timestamp-attestation, DIARY-PRD-evidence-timestamp-verification

requirements: DIARY-PRD-architecture-decision-docs, DIARY-PRD-system-validation-traceability

resources: DIARY-PRD-help-resources

rules: CAL-PRD-entry-duration-check-configuration, CAL-PRD-entry-time-restrictions-configuration, DIARY-GUI-entry-overlap-resolution, DIARY-PRD-entry-duration-check, DIARY-PRD-entry-overlap-resolution, DIARY-PRD-entry-time-restrictions

schema: CAL-DEV-edc-audit-context-structure, CAL-DEV-edc-audit-location-format, CAL-DEV-edc-audit-reason-values, CAL-DEV-edc-audit-timestamp-format, CAL-DEV-edc-audit-user-values, CAL-DEV-edc-callisto-transforms, CAL-DEV-edc-change-history-via-eventstore, CAL-DEV-edc-data-source-trace, CAL-DEV-edc-date-field-format, CAL-DEV-edc-duration-calculation, CAL-DEV-edc-incomplete-entry-flag, CAL-DEV-edc-no-nosebleeds-flag, CAL-DEV-edc-notes-field-handling, CAL-DEV-edc-patient-id-format, CAL-DEV-edc-requester-id, CAL-DEV-edc-requester-name, CAL-DEV-edc-severity-field-format, CAL-DEV-edc-site-id-format, CAL-DEV-edc-survey-name, CAL-DEV-edc-survey-response-storage, CAL-DEV-edc-survey-version, CAL-DEV-edc-time-field-format, CAL-DEV-edc-unknown-event-flag, CAL-DEV-edc-uuid-field-format, CAL-DEV-edc-uuid-table-prefix

score: CAL-PRD-questionnaire-score-display-prohibition, DIARY-PRD-questionnaire-score-calculation, DIARY-PRD-score-hht-qol, DIARY-PRD-score-nose-hht

secrets: CAL-OPS-secrets-doppler-cloud-run, CAL-OPS-secrets-github-actions-routing, CAL-OPS-secrets-sponsor-agnostic-automation

services: DIARY-PRD-ancillary-platform-services

settings: CAL-PRD-administrator-settings-inventory, DIARY-GUI-administrator-settings, DIARY-PRD-administrator-settings

sla: DIARY-PRD-sla-chronic-failure-escalation, DIARY-PRD-sla-corrective-preventive, DIARY-PRD-sla-customer-notification, DIARY-PRD-sla-data-recovery, DIARY-PRD-sla-disaster-recovery, DIARY-PRD-sla-incident-response-times, DIARY-PRD-sla-incident-severity, DIARY-PRD-sla-regulatory-event-support, DIARY-PRD-sla-root-cause-analysis, DIARY-PRD-sla-service-availability

user: DIARY-GUI-administrator-dashboard, DIARY-GUI-user-account-deactivate, DIARY-GUI-user-account-reactivate, DIARY-GUI-user-information-modal, DIARY-GUI-user-management-tabs, DIARY-PRD-reason-field-constraints, DIARY-PRD-user-account-activation-resend, DIARY-PRD-user-account-activation-workflow, DIARY-PRD-user-account-create, DIARY-PRD-user-account-deactivate, DIARY-PRD-user-account-edit, DIARY-PRD-user-account-reactivate, DIARY-PRD-user-account-site-assignment

validation: CAL-OPS-ci-change-appropriate-validation, CAL-OPS-ci-code-quality-static-analysis, CAL-OPS-ci-secret-vulnerability-scanning, CAL-OPS-ci-traceability-pipeline

versioning: [DIARY-PRD-questionnaire-localization](#), [DIARY-PRD-questionnaire-sponsor-eligibility](#), [DIARY-PRD-questionnaire-versioning](#)

workflow: [CAL-GUI-questionnaire-call-back-participant](#), [DIARY-GUI-questionnaire-portal-sent-workflow](#), [DIARY-GUI-questionnaire-session-expiry](#), [DIARY-PRD-questionnaire-portal-sent-rules](#), [DIARY-PRD-questionnaire-session-timeout](#)